

SAR-ADC (SKY130)

ISHI会

<https://ishi-kai.org/>

Mail: info@ishi-kai.org

今回のターゲットスペック

シャトル

- TinyTapeoutのSky130のアナログ回路

SAR-ADCスペック

- ビット数：6bit
- クロック：1.2MHz
- リファレンス電圧：1.8V～5.0V（可変）

もくじ： CDAC編

- CDAC
- BGR
- トランスミッションゲート

もくじ： ロジック編

- 制御器
 - ビルド環境構築
 - spiceファイル

もくじ： 比較器編

- ラッチ回路
- 比較器（コンパレータ）
 - クロックドコンパレータ（デジタル）
 - Rail-to-Railクロックドコンパレータ
 - OPAMP型コンパレータ（アナログ）

もくじ：

シミュレーション編

- シミュレーションモデル
 - コーナーモデルシミュレーション
 - モンテカルロシミュレーション
- SAR-ADCのシミュレーション
 - オフセットとノイズ
 - スルーレート
 - ばらつき
 - 全統合シミュレーション
- AI
 - シミュレーション
 - 回路図
 - レイアウト

スペック編



CDAC編

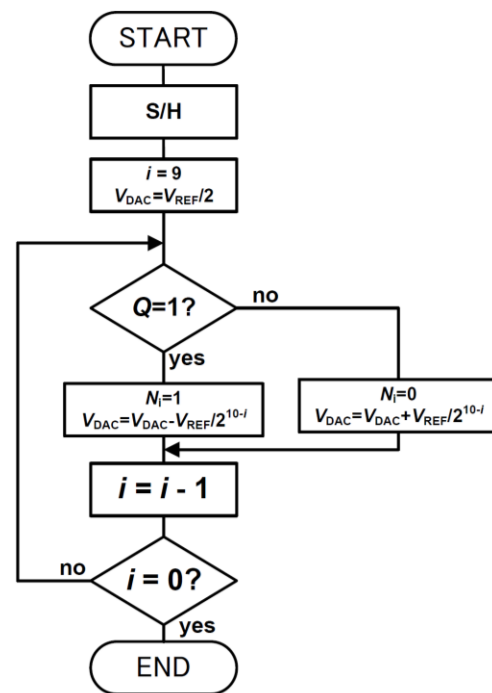
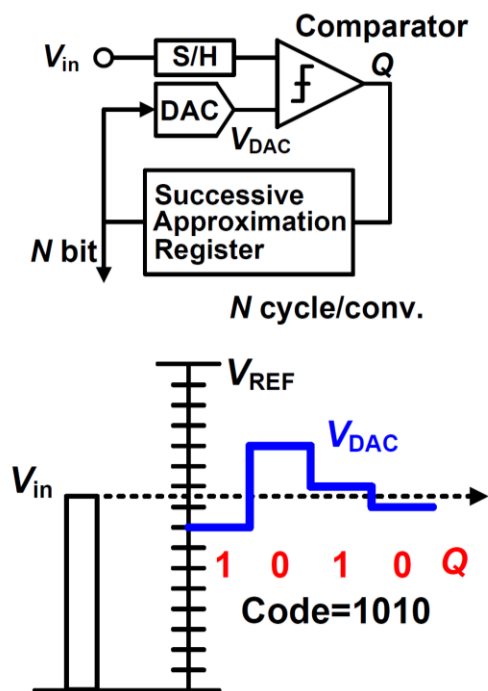




CDAC (Capacitive Digital-to-Analog Converter)

宮原氏の資料ベースに解説します

<https://openit.kek.jp/training/2020/daq/text/analog-digital-miyahara-200729-v1.pdf>



サンプリングモード

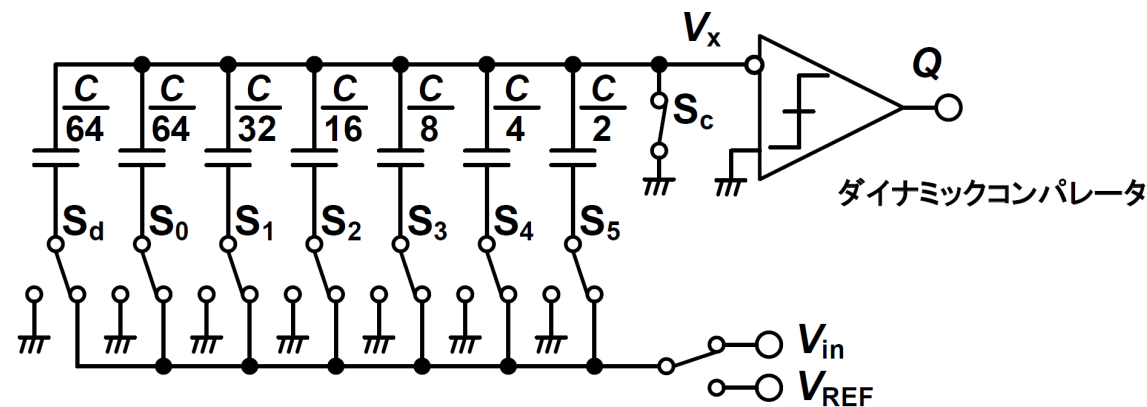
$$Q_S = CV_{in}$$

$$Q_S = Q_{MSB} \text{より、}$$

MSB変換モード

$$Q_{MSB} = \frac{C}{2}(V_{REF} - V_X) - \frac{C}{2}V_X$$

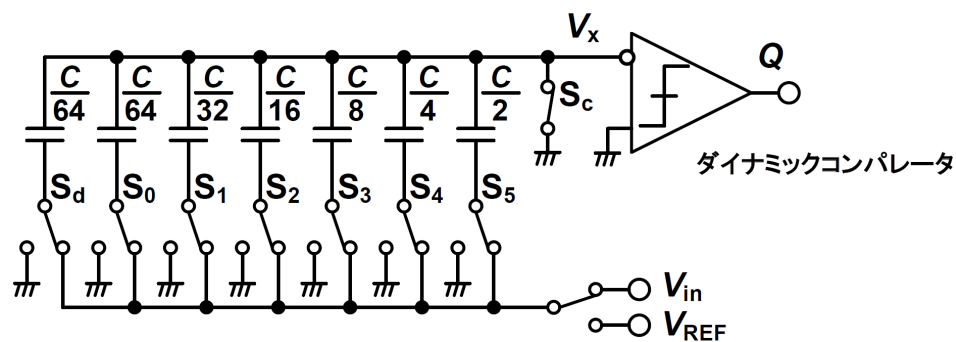
$$V_X = -\left(V_{in} - \frac{1}{2}V_{REF}\right)$$



CDACとは？

バイナリ重み付け（2倍）された容量アレイと、それらを「基準電圧（Vref）」「GND」「入力電圧（Vin）」のいずれかに接続するスイッチ群で構成されます。

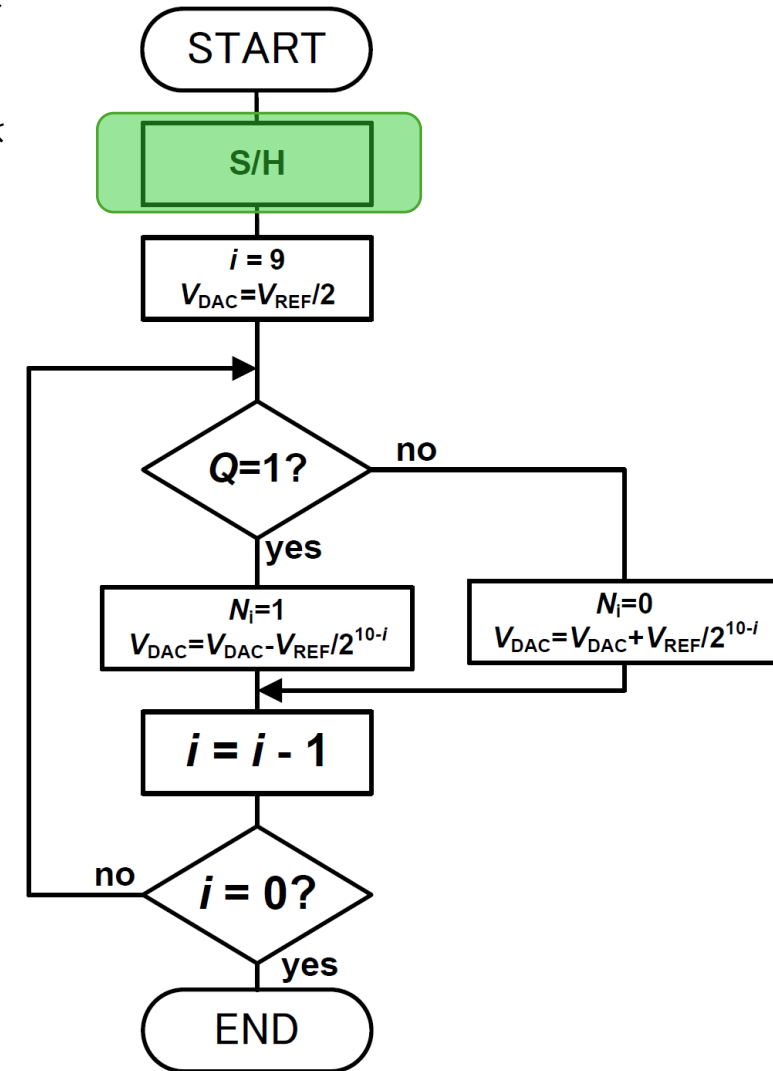
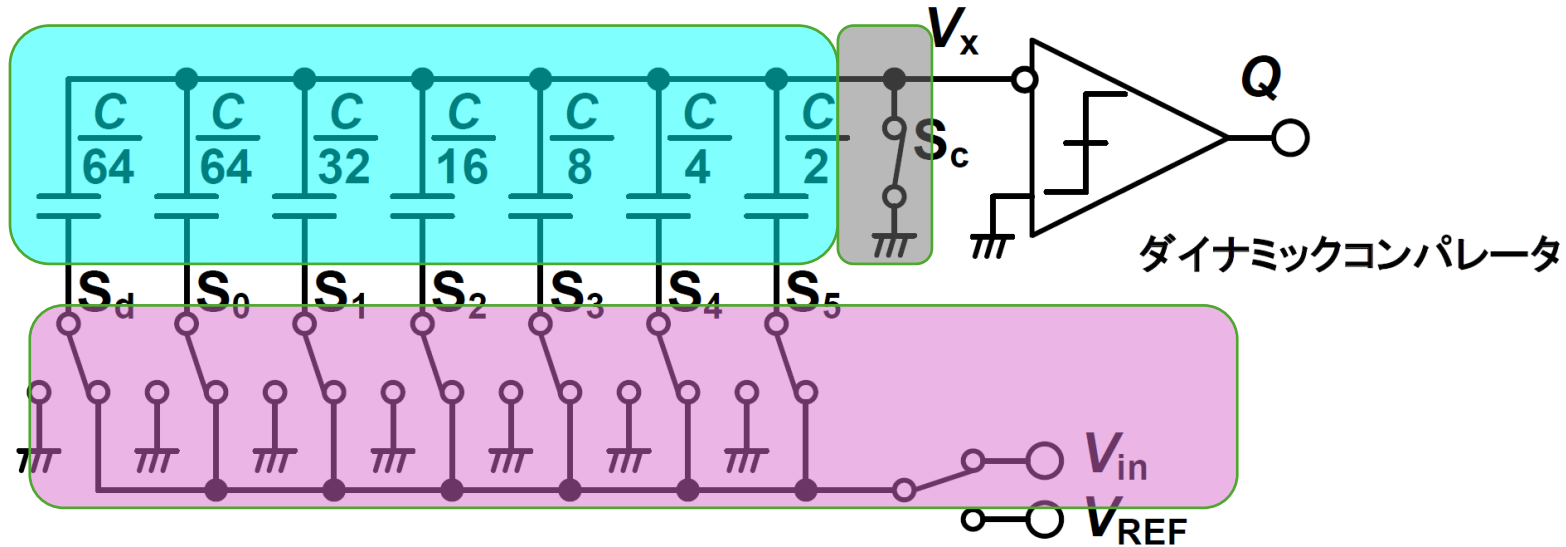
簡単な 動作手順



- サンプリング・フェーズ
 - すべての容量を入力電圧 V_{in} に接続し、電荷を蓄えます。
- ホールド（保持）・フェーズ
 - 入力スイッチを切り離し、電荷を閉じ込めます。このとき、容量アレイの下部プレートの接続を変更することで、上部プレートの電位が変化します。
- 比較・逐次変換フェーズ
 - SARロジックがMSB（最上位ビット）から順にスイッチを操作します。特定の容量を V_{ref} に接続すると、容量分圧によって比較器への入力電圧が上昇/下降します。比較器の結果を見て、そのビットを「1（接続維持）」にするか「0（GNDに戻す）」にするかを決定します。

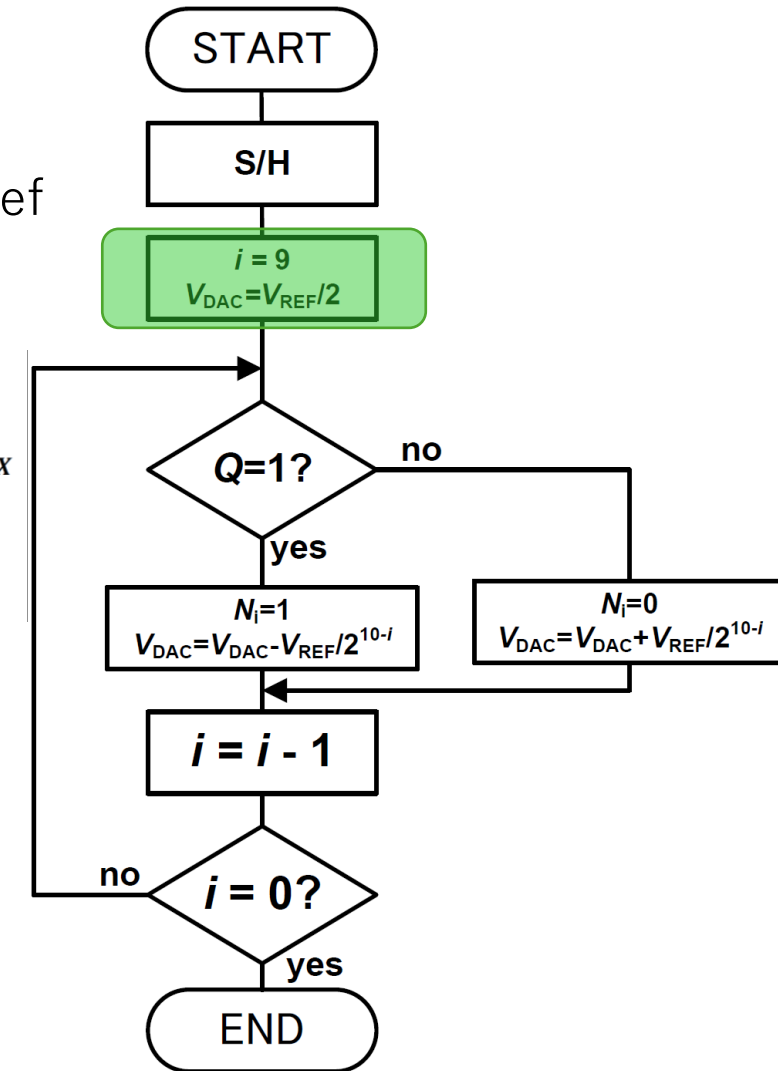
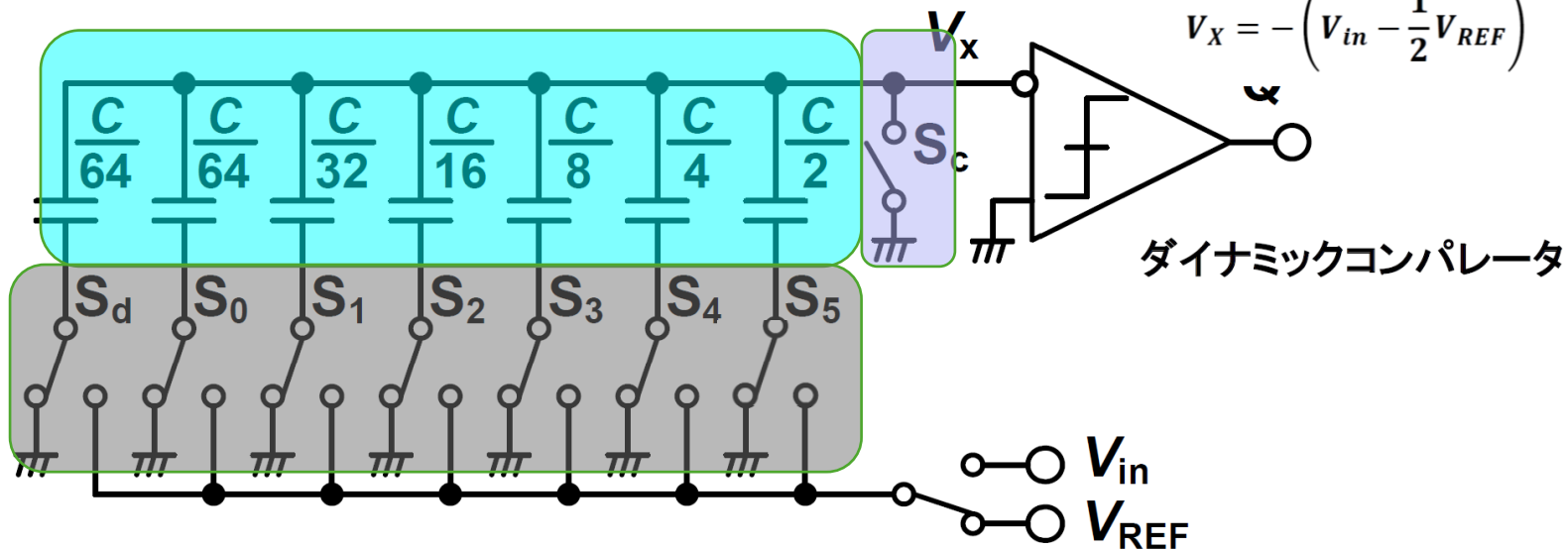
1, サンプリングフェーズ

- V_x 側のスイッチ S_c をGNDに落とし、ボトムプレート側のスイッチ ($S_5 \sim S_d$) をすべて入力 V_{in} に接続する
 - アレイ全体 (合計容量 C) に $Q_s = C \cdot V_{in}$ の電荷が充電 (サンプリング) される
 - 青い領域



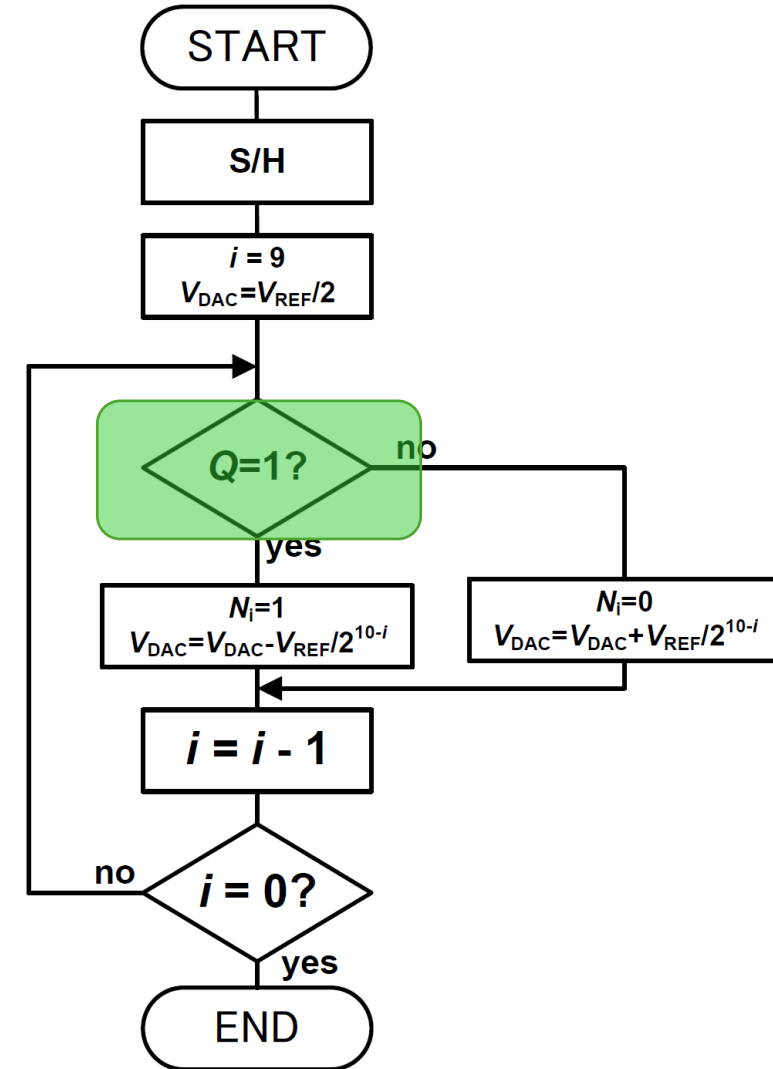
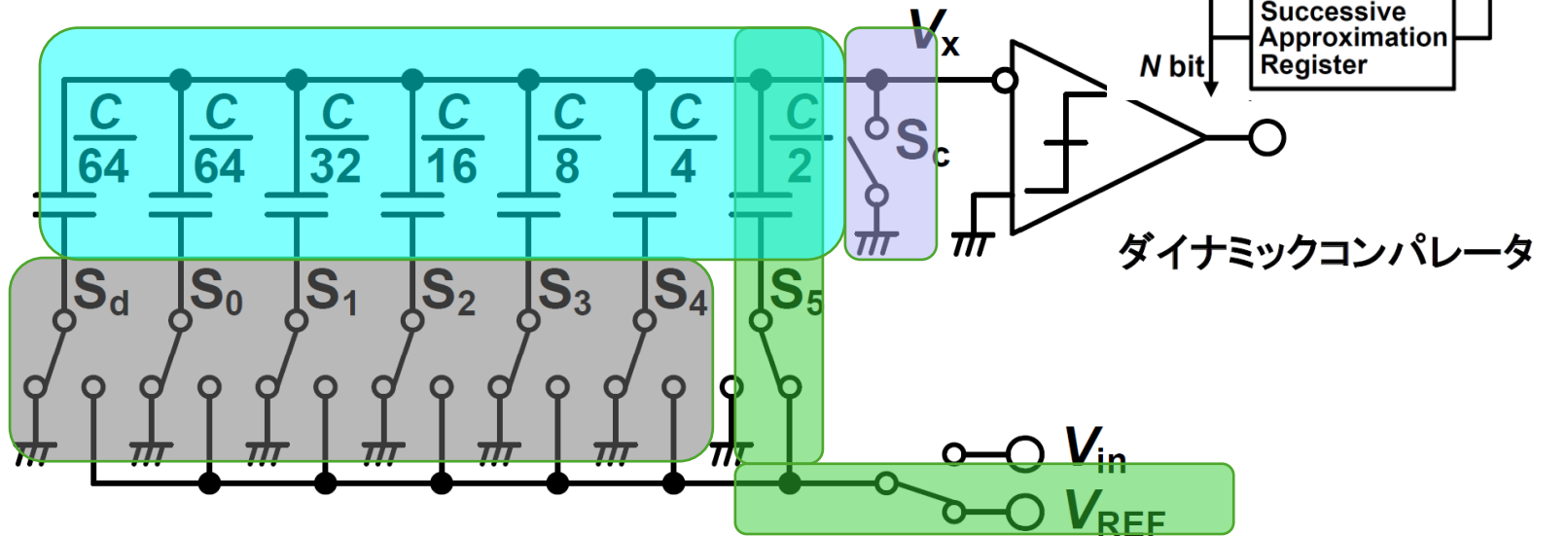
2, MSBの判定準備 (テスト電圧の印加)

- Sc を開いてVxノードをフローティング (ハイインピーダンス) にする
- すべてのボトムプレートを一旦GNDに落とす
 - これにより Vx は $-V_{in}$ (ーに注意) にレベルシフトする (ホールド状態)
- MSBに対応する最大のキャパシタ (C/2) のスイッチ S5 だけを Vref に接続する
 - フローチャートの $V_{dac} = V_{ref} / 2$ をセットする動作相当
 - $V_x = -(V_{in} - 1/2 V_{ref})$ になる



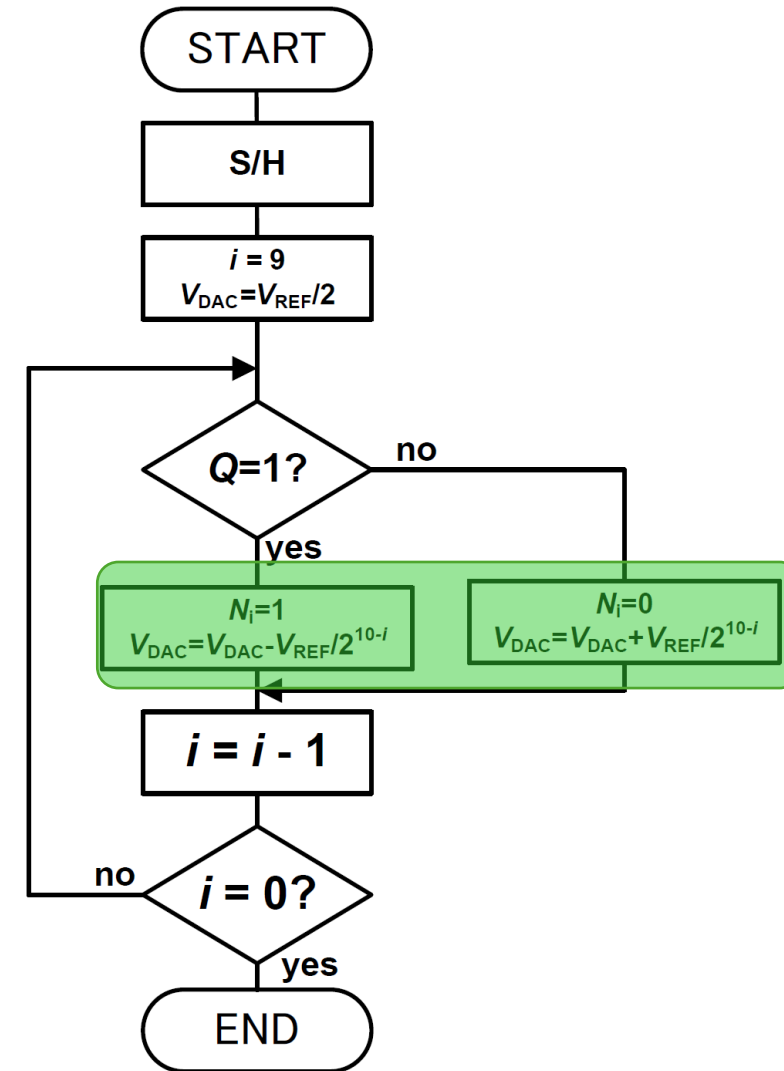
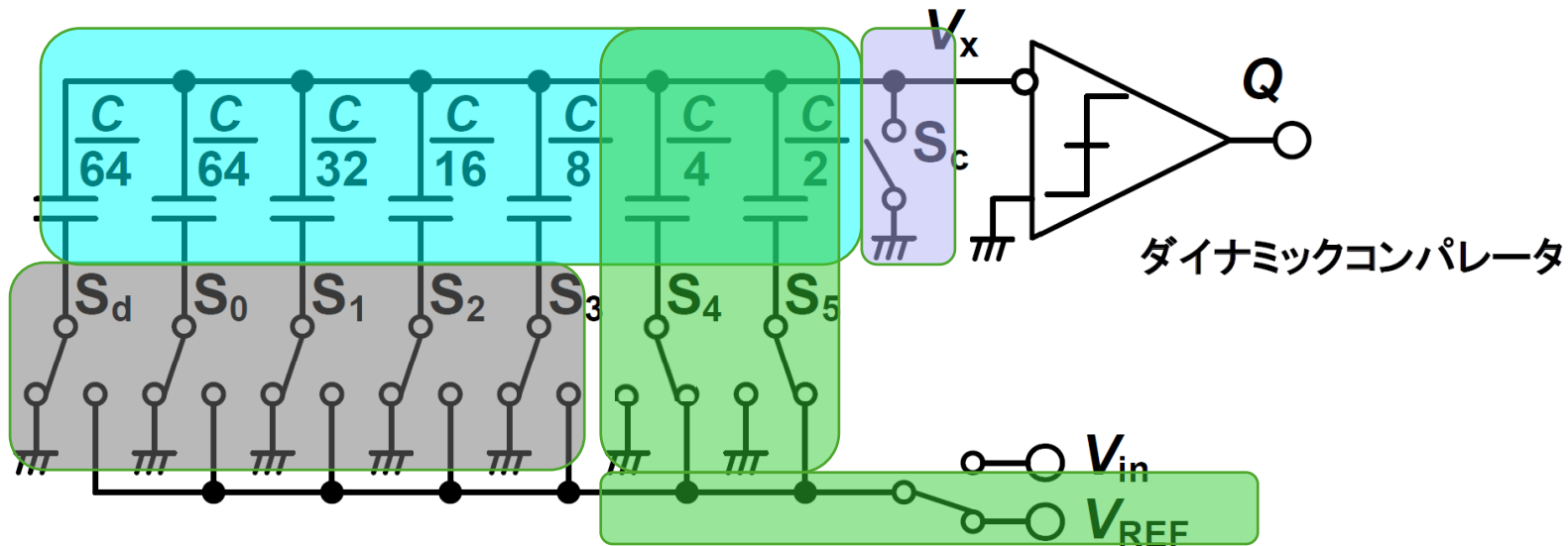
3, コンパレータによる比較判定

- コンパレータが「 V_{in} 」と「 V_{dac} 」という2つの正の電圧を比較する
 - 前段のステップで既に引き算 ($V_{in} - 1/2 V_{ref}$) は実行済み
- $V_x < 0V$ (GND) の場合 = $V_{in} > 1/2 V_{ref}$
 - コンパレータ出力 Q は「1 (Yes)」
- $V_x > 0V$ (GND) の場合 = $V_{in} < 1/2 V_{ref}$
 - コンパレータ出力 Q は「0 (No)」



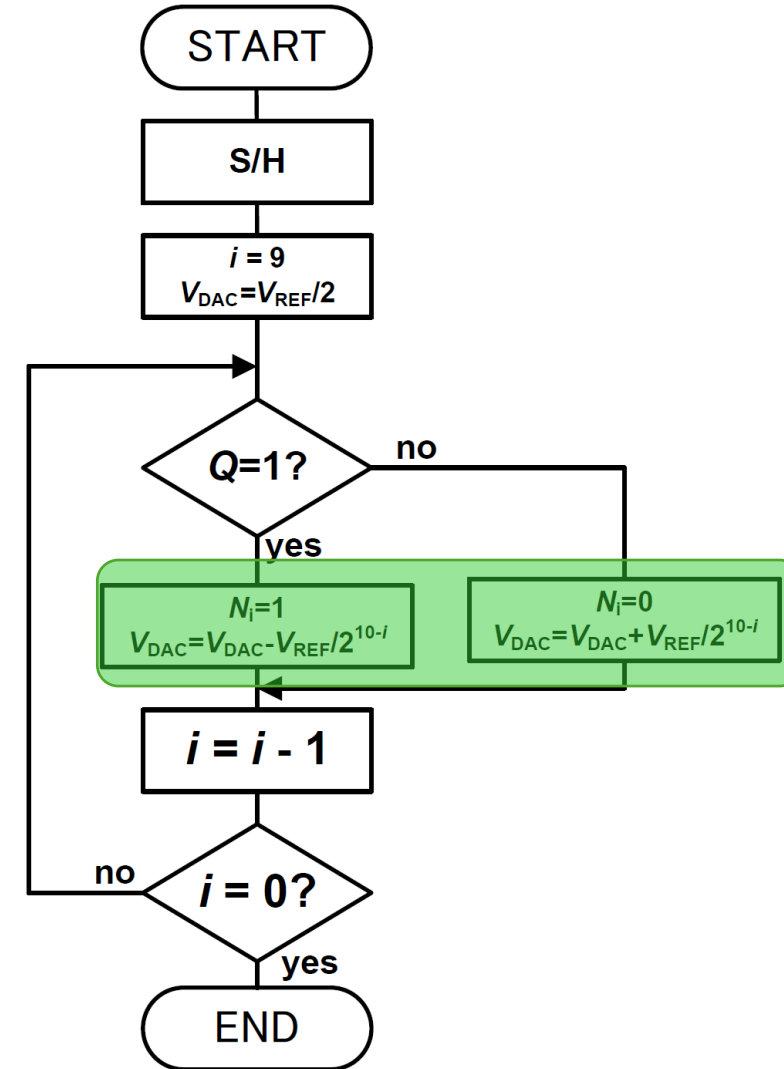
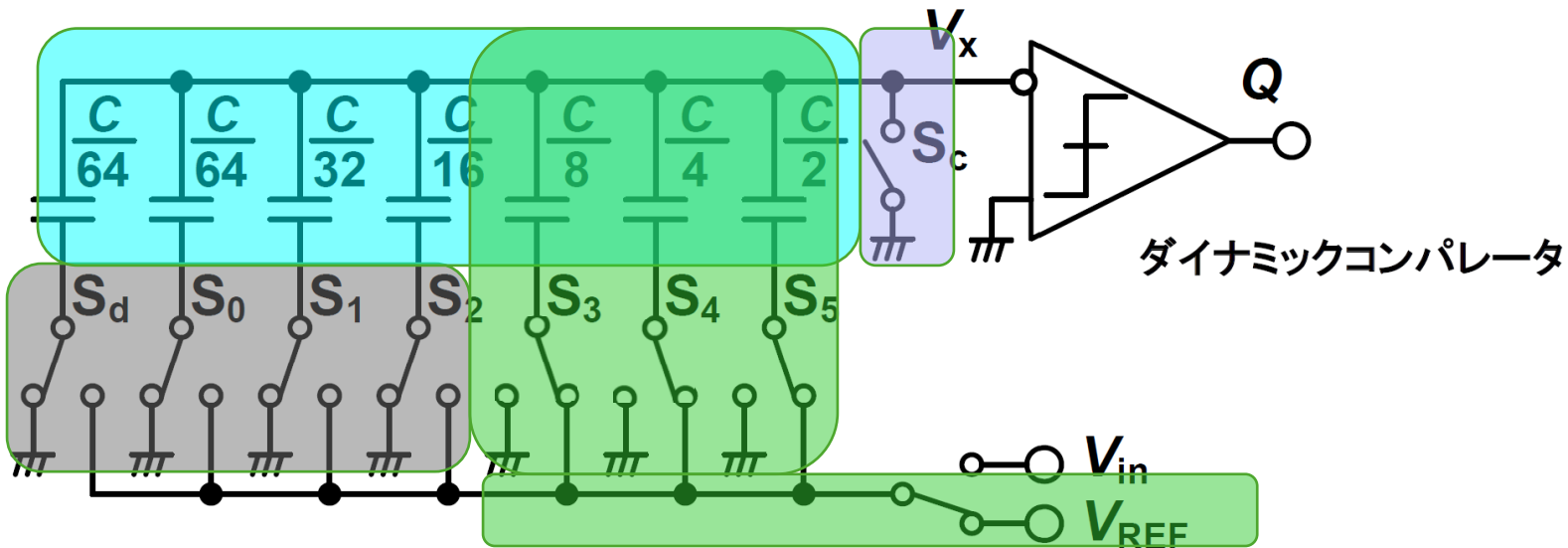
4, 保持と次のビットへの移行

- 判定が終わると、SARロジック（レジスタ）がその結果を記憶し、スイッチの状態を以下のように制御します。
 - Yes ($Q=1$) の場合
 - MSBのスイッチ S_5 は V_{REF} に繋いだまま（固定）にする。（これが $N_i = 1$ の状態）
 - No ($Q=0$) の場合:
 - MSBのスイッチ S_5 を GND に接続する。（これが $N_i = 0$ の状態）
- $i = i - 1$ に進み、次のビット（ S_4 : 容量 $C/4$ ）のスイッチを V_{ref} に接続する
 - V_x の電位は、直前の結果に応じて「 $1/2 V_{ref} + 1/4 V_{ref}$ 」または「 $0 + 1/4 V_{ref}$ 」をテストする状態へ遷移する



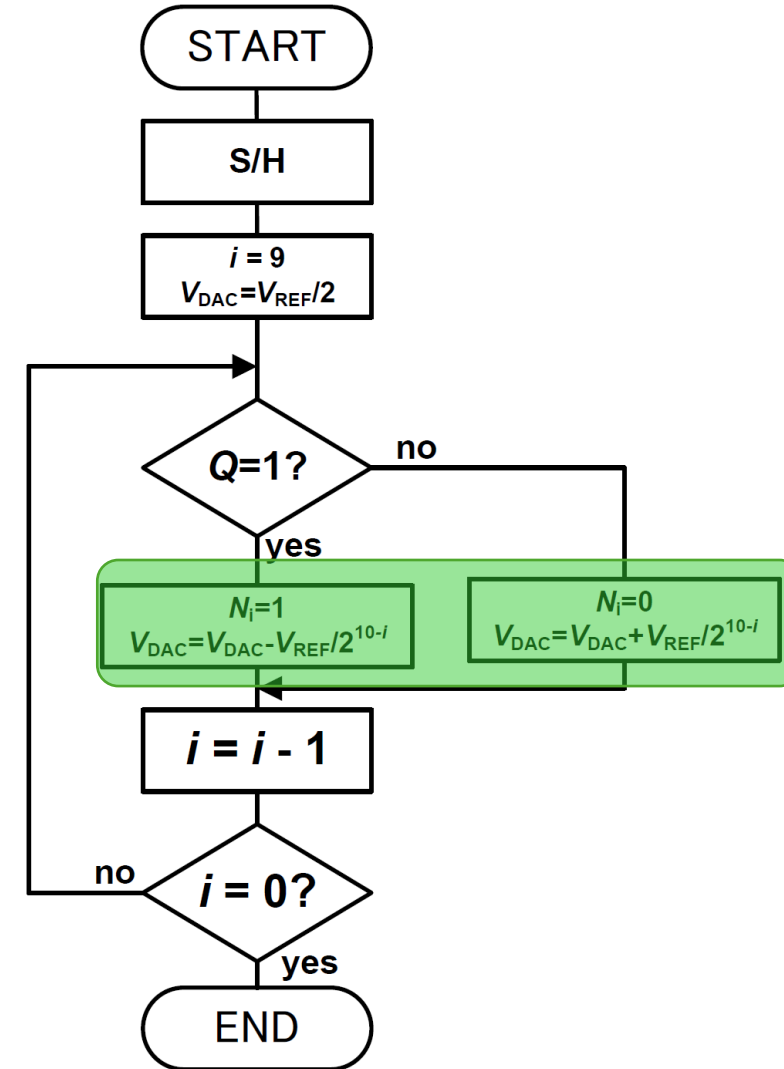
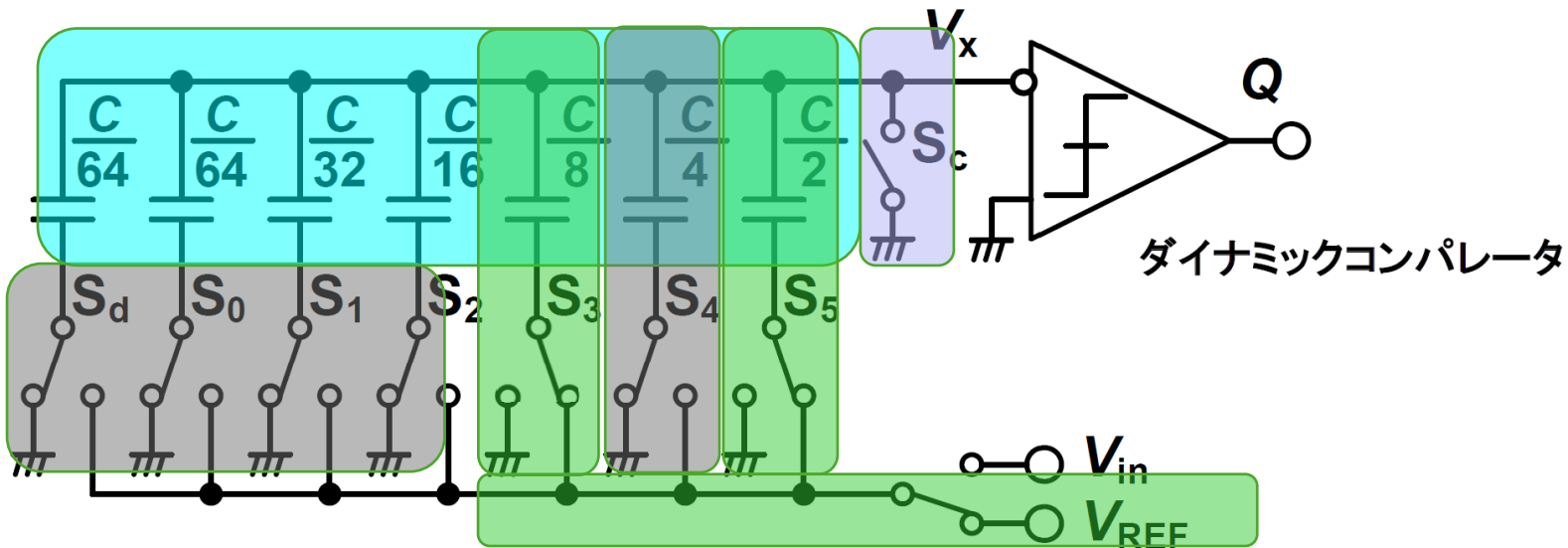
4 - 1, 保持と次のビットへの移行

- 例: $V_x: S_5 > \text{GND}, S_4 > \text{GND}$ だった場合
 - 次のビット (S_3 : 容量 $C/8$) のスイッチを V_{ref} に接続する
 - V_x の電位は
 - $\lceil 1/2 V_{\text{ref}} + 1/4 V_{\text{ref}} + 1/8 V_{\text{ref}} \rceil$
 - MSB
 - 111xxx



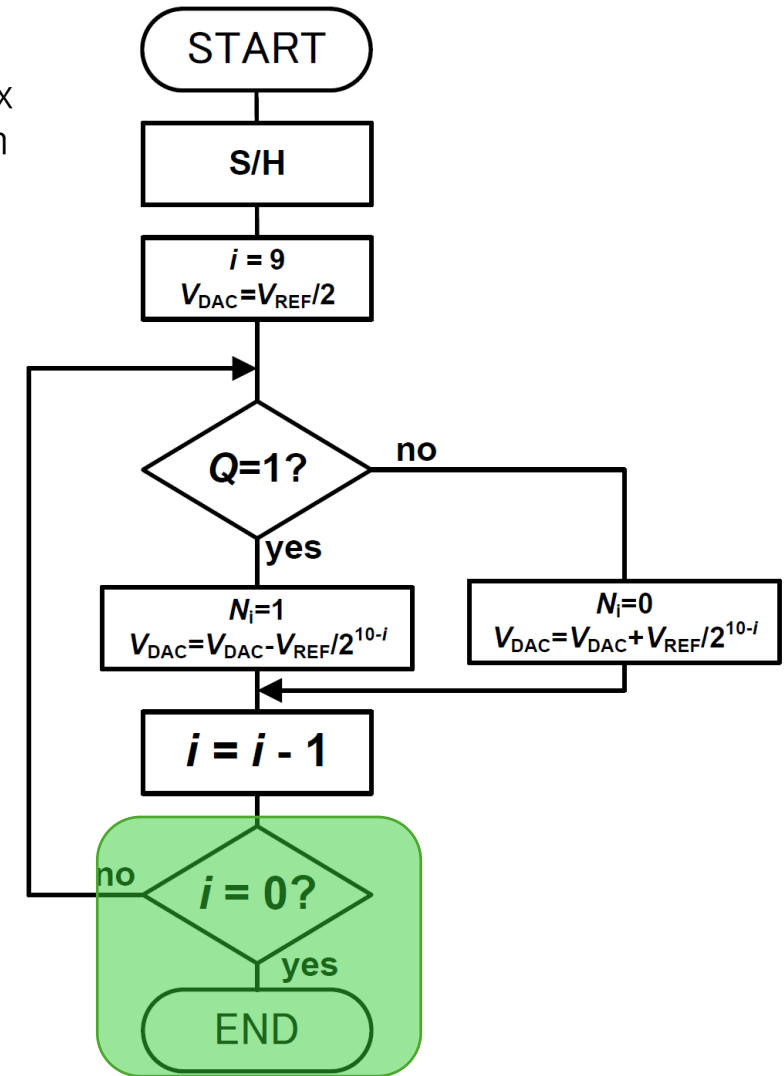
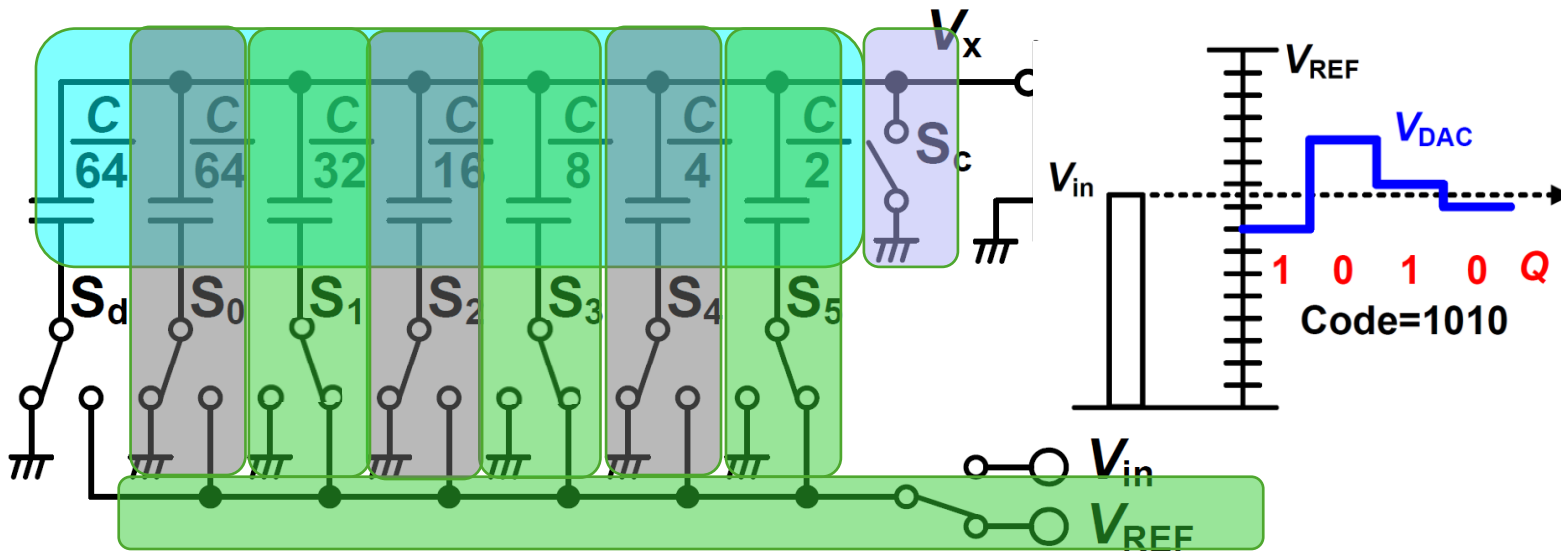
4 - 2, 保持と次のビットへの移行

- 例: $V_x: S_5 > \text{GND}, S_4 < \text{GND}$ だった場合
 - 次のビット (S_3 : 容量 $C/8$) のスイッチを V_{ref} に接続する
 - V_x の電位は
 - $\left[\frac{1}{2} V_{\text{ref}} + 0 + \frac{1}{8} V_{\text{ref}} \right]$
 - MSB
 - 101xxx



5, 変換終了までループ

- 上記 2~4 の操作を、次に大きいキャパシタ ($S_4 \rightarrow S_3 \rightarrow S_2 \rightarrow S_1 \rightarrow S_0$) へと順番に繰り返す
 - グラフの青い線 (V_{DAC}) が示しているように、バイナリサーチによって V_x が最終的に $0V$ に限りなく近づくように (つまり内部の等価DAC電圧が V_{in} に追従するように) スイッチがパタパタと切り替わっていくことになる
- 全ビットの判定が終わったときに V_{ref} 側に残っているスイッチの配列が、そのままデジタル出力コードとなる
 - 下記の例の場合
 - 101010



6, Sdはダミー

- 全容量を「きれいに2のn乗」にするための役割バイナリ重み付けのキャパシタアレイ ($C/2, C/4, \dots, C/64$) をすべて足し合わせると、右図のようになる
- Sdの容量 ($C/64$) を足すことで、全容量 C がちょうど $1.0C$ (すなわち、 $64 \times$ 最小容量の $C = C/64$) となる

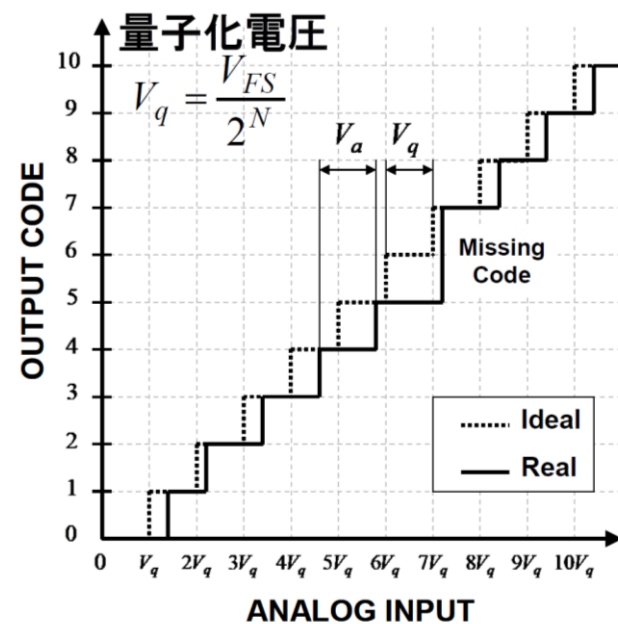
$$\sum_{i=1}^6 \frac{C}{2^i} = \frac{63}{64}C$$

設計方針

- 下記の与えられた条件内で最大容量とする
 - MIM,MOMの選択
 - ばらつきを減らすか、容量を優先するかで適切な方を選択すること
 - 動作クロック
 - 容量が増えると時定数が増加するためクロックが遅くなる
 - 面積
 - 容量が増えると面積が大きくなる
 - 消費電力
 - 容量が増えると電荷が増えるため消費電力が増える

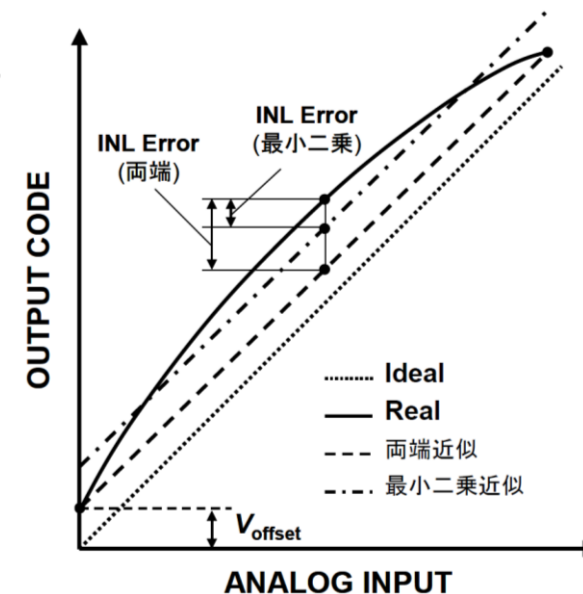
性能指標：DNL, INL

- DNL (Differential non linearity)
 - 1 ステップ毎の誤差量
- INL (Integral non linearity)
 - 理想の入出力直線に対する実際の入出力特性の誤差量



$$DNL(1\text{LSB}) = \frac{V_a - V_q}{V_q}$$

DNLの下限値は-1(ミッシング)



$$INL = \int DNL \Delta code$$

性能指標：電力効率

• 分解能を1bit向上させるために必要となる消費電力の増加量についての考え方の違い

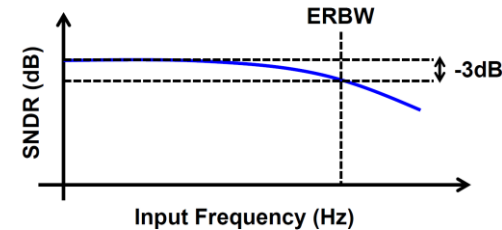
- FoM_W $P_D \propto 2^N$
 - Flash型ADC⇒1bit増で比較器個数2倍
 - 回路規模が2倍になるのだから消費電力も2倍
- FoM_S $P_D \propto 2^{2N}$
 - 1bit増で量子化電圧は1/2, 電力では1/4となる
 - 離散時間系の回路の熱雑音は kT/C に比例する
 - 熱雑音を1/4にするには容量 C を4倍にする
 - 速度一定の場合、消費電力は容量に比例する
⇒消費電力は4倍になる

10bit程度までは FoM_W , それ以降は FoM_S が用いられることが多い

異なる分解能、変換速度を持つADCに対して電力効率を比較するための性能指標の一つ。1変換・ステップあたりに要するエネルギーを表す。

$$FoM_W = \frac{P_D}{2^{ENOB} \cdot \text{Min}(F_s, 2 \times ERBW)} \quad (\text{J/conv. step})$$

ERBWは有効な信号帯域を表す。



FoM_W が低いほど効率の良いADCといえる。
現状最も優れた FoM_W は1fJ/conv.を切る程度

異なる分解能、変換速度を持つADCに対して電力効率を比較するための性能指標一つ。雑音電力によってSNRが決まる領域で有効な指標。

$$FoM_S = SNDR + 10 \log_{10} \left(\frac{BW}{P_D} \right) \quad (\text{dB})$$

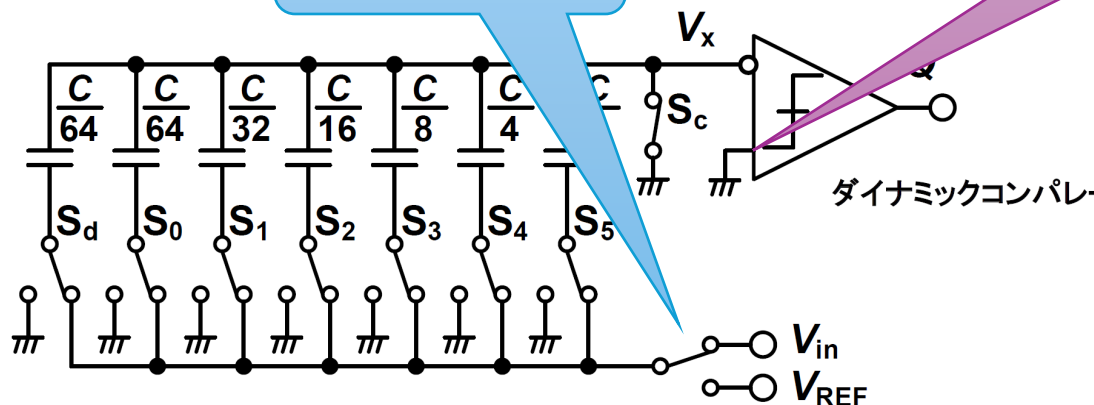
FoM_S が高いほど効率の良いADCといえる。
現状最も優れた FoM_S は180dB程度

サンプルでは . . .

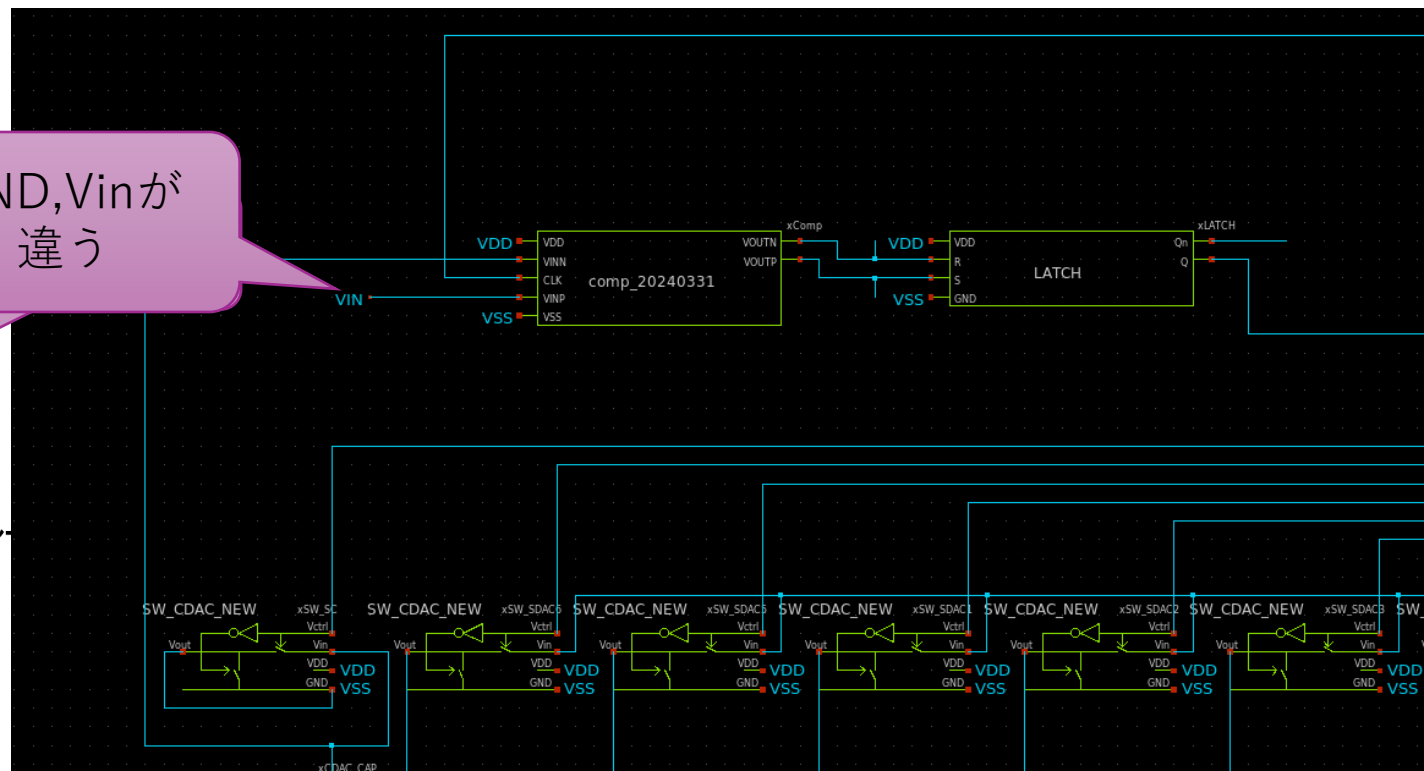
ファイル名：sar_dac.sch

- サンプル：コンパレータの比較用に V_{in} を入力
 - メリット：シンプル
 - V_{ref} , V_{in} 切り替え用のトランSMミッションゲートがない
 - CDACの一括切り替えがない
- 本解説：コンパレータの比較用にGND(VSS)を入力
 - メリット： V_{in} のゆれに強い
 - 最初の充電でしか V_{in} を利用しない

これが必要



GND, V_{in} が
違う

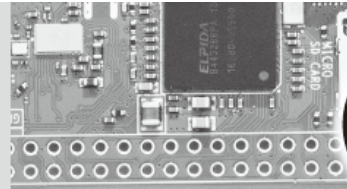




BGR
(Band-Gap Reference)

解説

- Open Source Silicon Magazine Vol.2を参照のこと
 - BGR回路
 - コンパレータ動作の基準用
 - スタートアップ回路
 - ゼロ点防止



[特集] マイコン解体新書

-トランジスタから専用IC設計まで-

すべての基準となる回路

参照電流源回路(CS)・参照電圧源回路(BGR)

■前橋 雄(yu.maehashi@outlook.jp)

様々な環境下でアナログ回路を正確に動作させるには、基準となる参照電流源・参照電圧源が不可欠です。本記事では、基本回路の設計を通して、その仕組みと改善方法を体験していきます。

参照電流(電圧)源回路って何?

アナログ回路には様々な回路がありますが、これらを正確に動作させるにはある決まった電流(定電流)または電圧(定電圧)を供給する必要があります。これらICチップ内でそれらを供給する回路

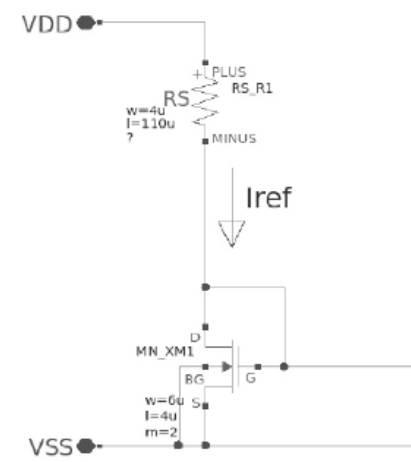


図1 基本の電流源回路

BGR

- 記事を参照してください！

バンドギャップレファレンス回路

前節で参照電流源回路を設計しました。では、一定の電圧を供給する参照電圧源はどのように作るのでしょうか。実は、参照電流源回路を少し変更するだけで実現できます(図21)。

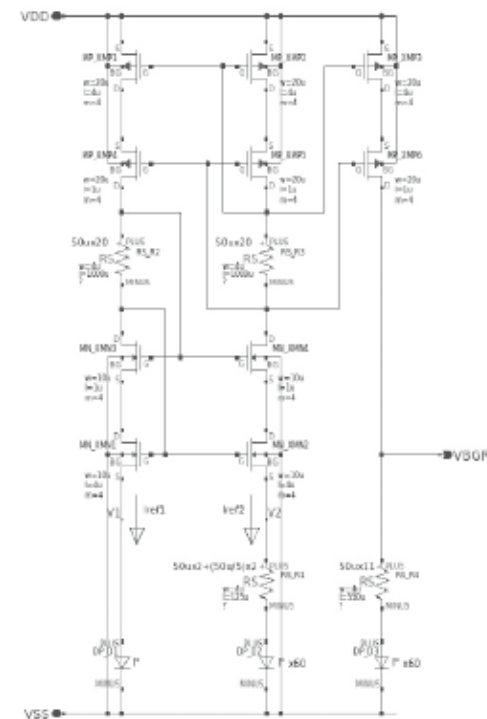


図21 バンドギャップリファレンス回路

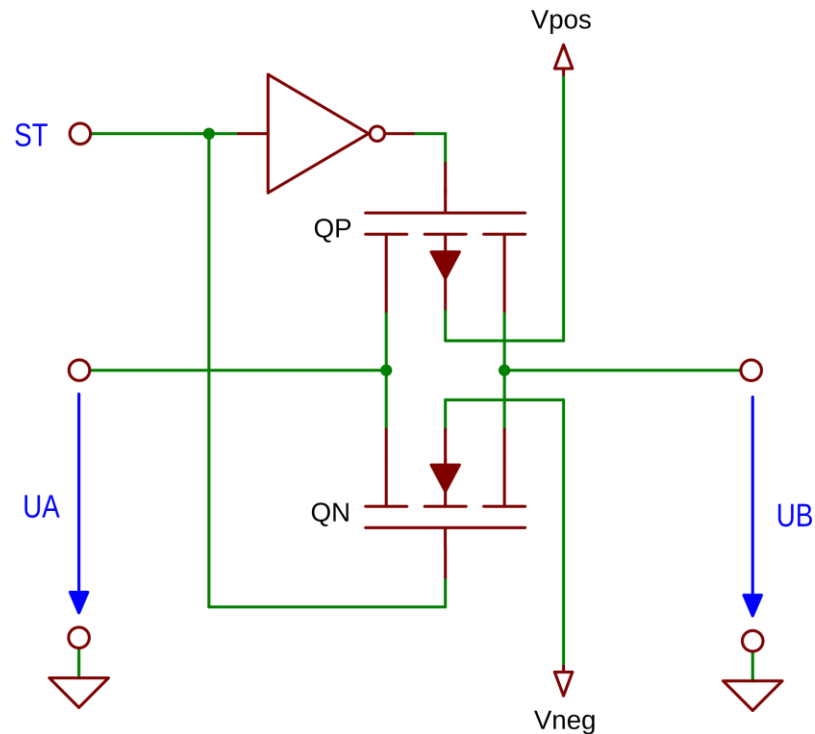
The background features three overlapping teal-colored circles of equal size, arranged horizontally. The circles overlap in the center, creating a darker teal shade. The entire composition is set against a dark gray background. A horizontal white band cuts across the middle of the image, containing the text.

トランスミッションゲート

トランスミッシヨングート

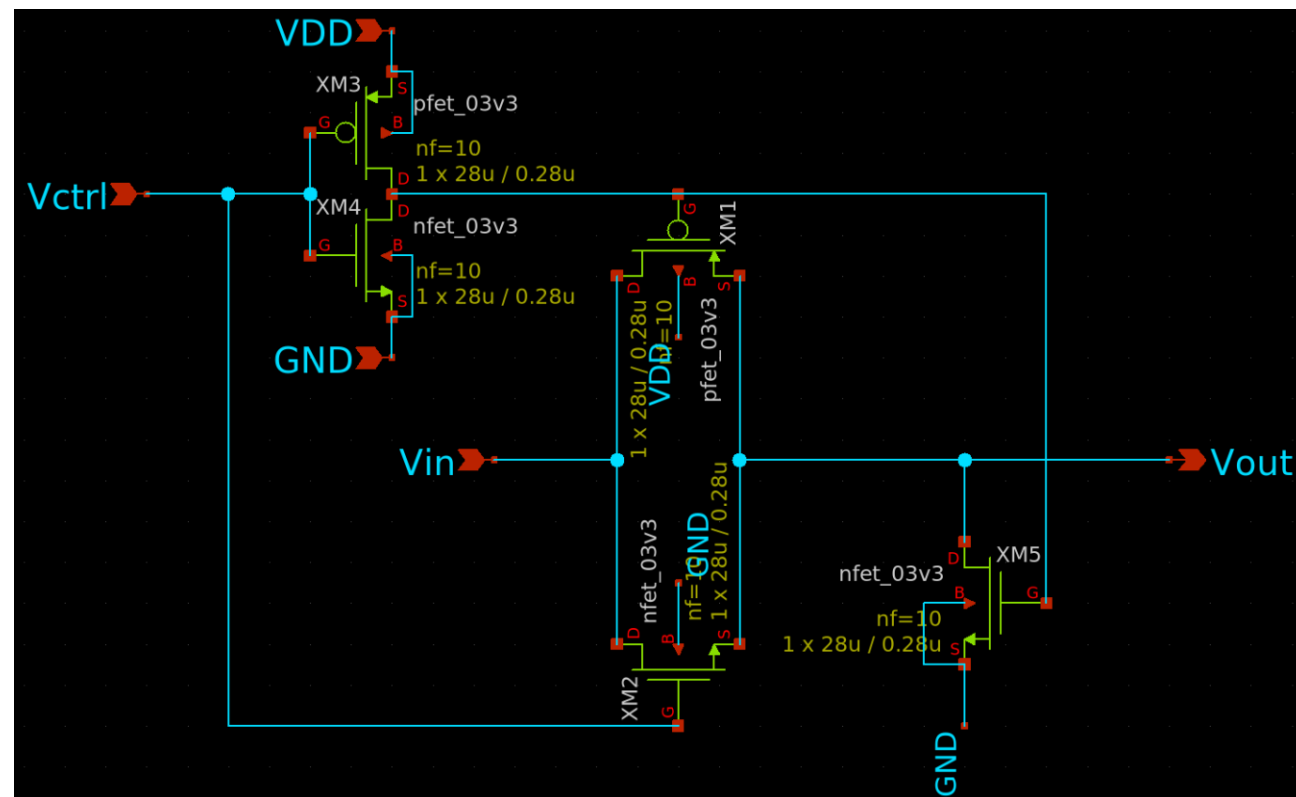
- アナログ用のスイッチ

制御入力 ST	入力 UA	出力 UB	状態
0 (L)	0	Hi-Z	遮断 (OFF)
0 (L)	1	Hi-Z	遮断 (OFF)
1 (H)	0	0	導通 (ON)
1 (H)	1	1	導通 (ON)



動作内容

- Vctrl = High
 - XM2 (NMOS) がオン
 - XM1 (PMOS) は Vctrl の反転でオン
 - Vin が Vout に伝わる
- Vctrl = Low
 - XM1, XM2 とともにオフ
 - Vin と Vout は切り離される
 - XM5が ON になり、Vout を強制的に GNDにする



設計方針

インスタンス	種類	サイズ決定のポイント
XM1	パス用スイッチ (PMOS)	32Cの容量 (電荷) を規定クロック内でVSS (0V) に出来るスルーレート
XM2	パス用スイッチ (NMOS)	P/Nバランスを考慮した値
XM3, XM4	インバータ (P,NMOS)	XM1、XM2のゲートを高速に駆動できるサイズ
XM5	プルダウン (NMOS)	XM2の1/2～1/5程度

トランスミッション設計：スペック例

単位容量：100fF

クロック：1.2MHz

ロジック部電圧：1.8V
アナログ部電圧：5.0V

シミュレーション回路例

COMMANDS
SIM=ngspice

```
.temp 27
.option savecurrent
.control
save all

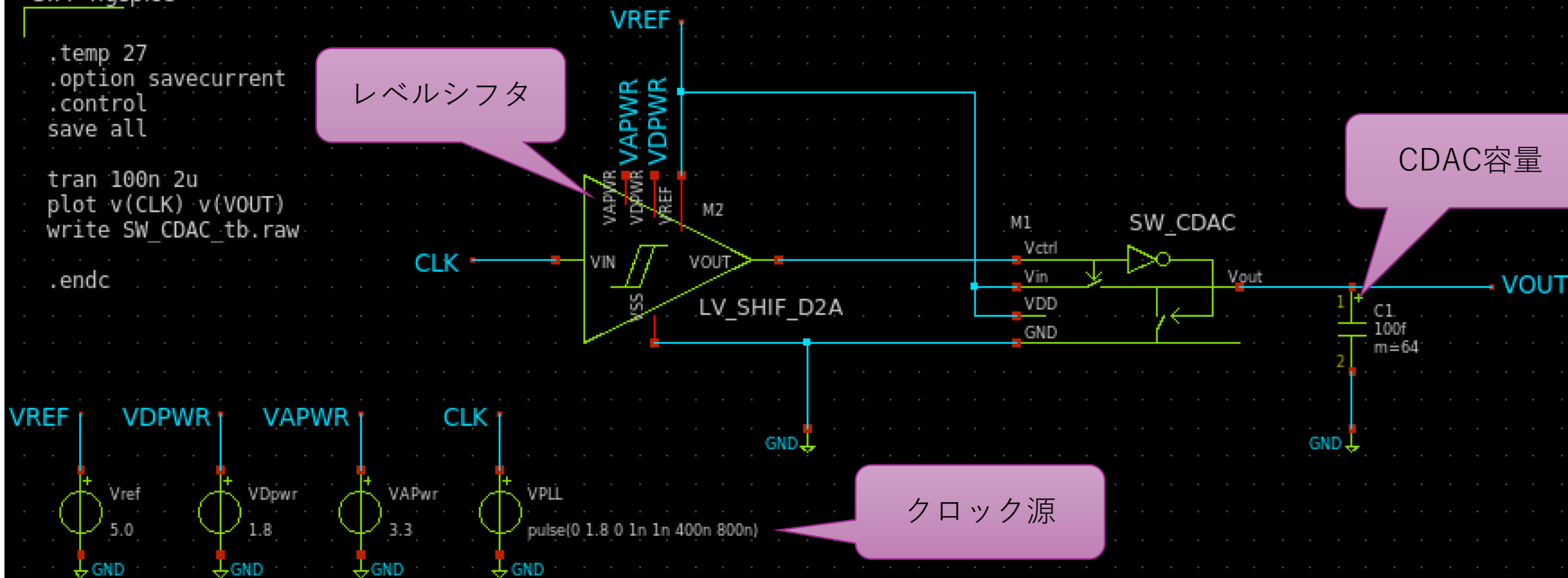
tran 100n 2u
plot v(CLK) v(VOUT)
write SW_CDAC_tb.raw

.endc
```

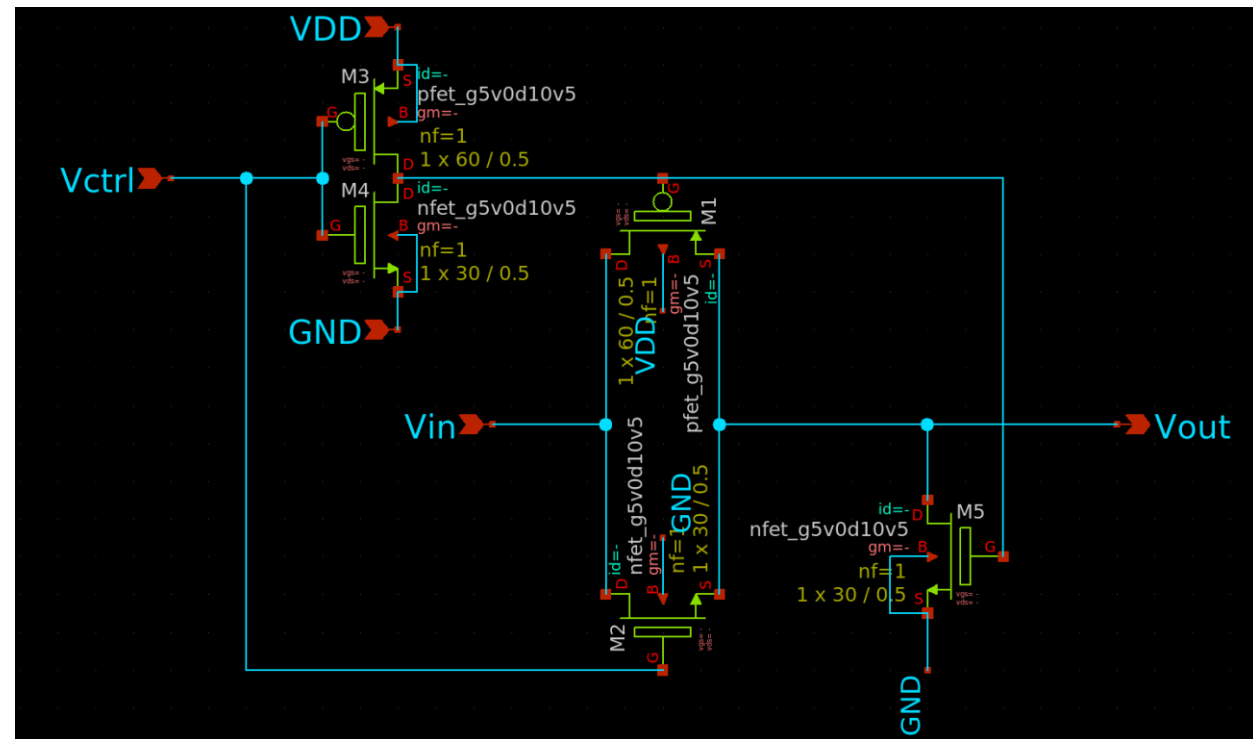
レベルシフタ

CDAC容量

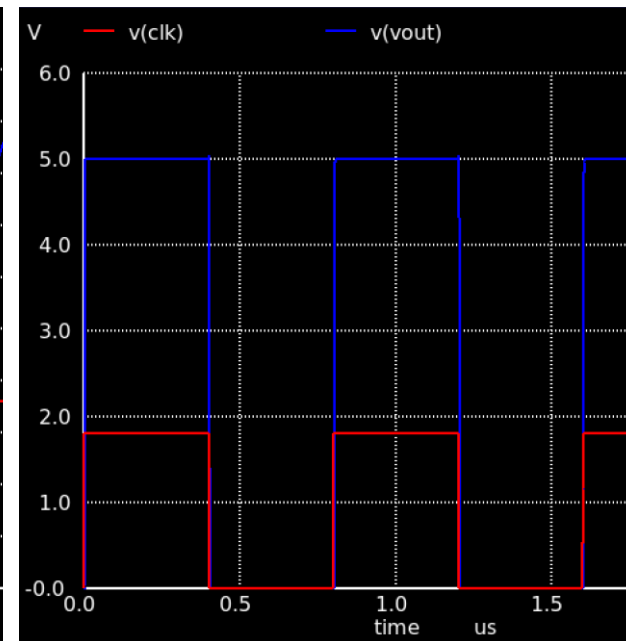
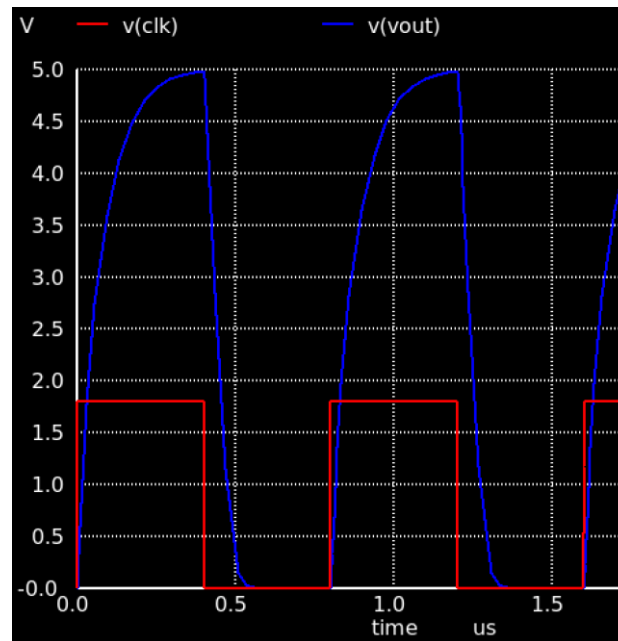
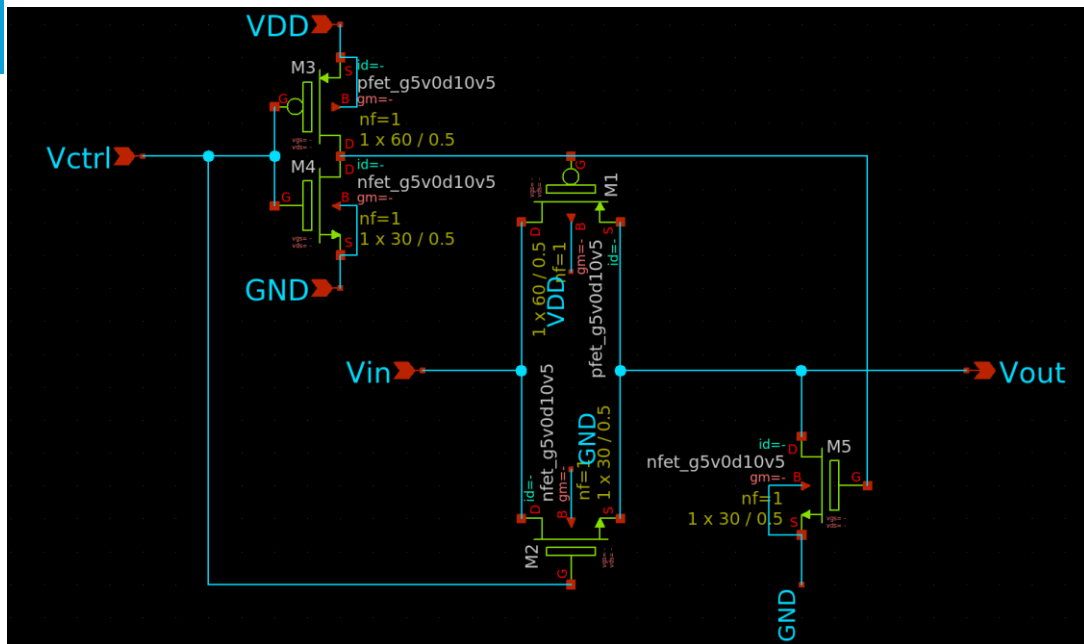
クロック源



設計ポイント：
Wの値



- 時定数： $\tau = R_{on} \times C$ 、目標誤差： $0.1\% = 7\tau$ 、
 - $\tau \leq \frac{200ns}{7} \approx 28.5ns$ (200nsは1.2MHzの半分クロックより)
 - $R_{on} \leq \frac{\tau}{C} = \frac{28.5ns}{6400fF} = 17.8k\Omega$
- PMOSの $R_{on} \approx \frac{1}{\mu_p C_{ox} (\frac{W}{L})(V_{GS} - V_{th})}$
 - W/Lは10～30あれば十分、NMOSは1/2～1/3

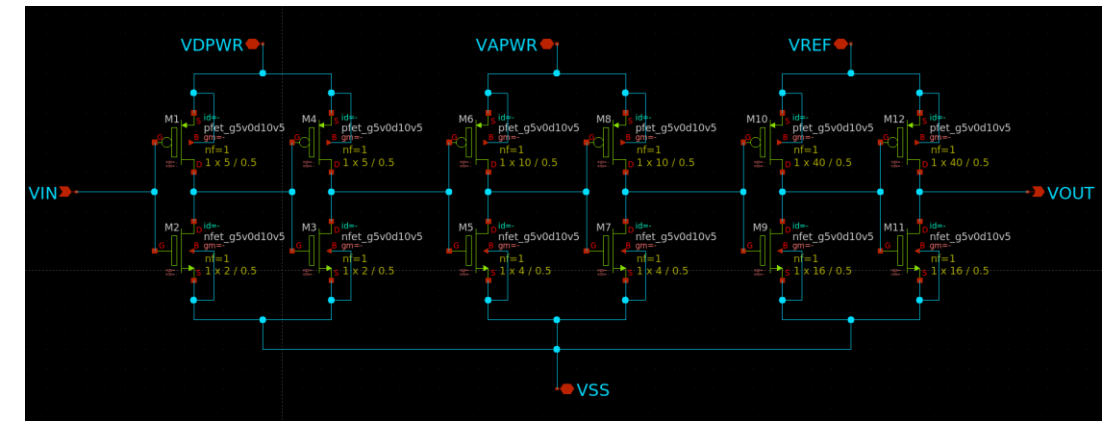
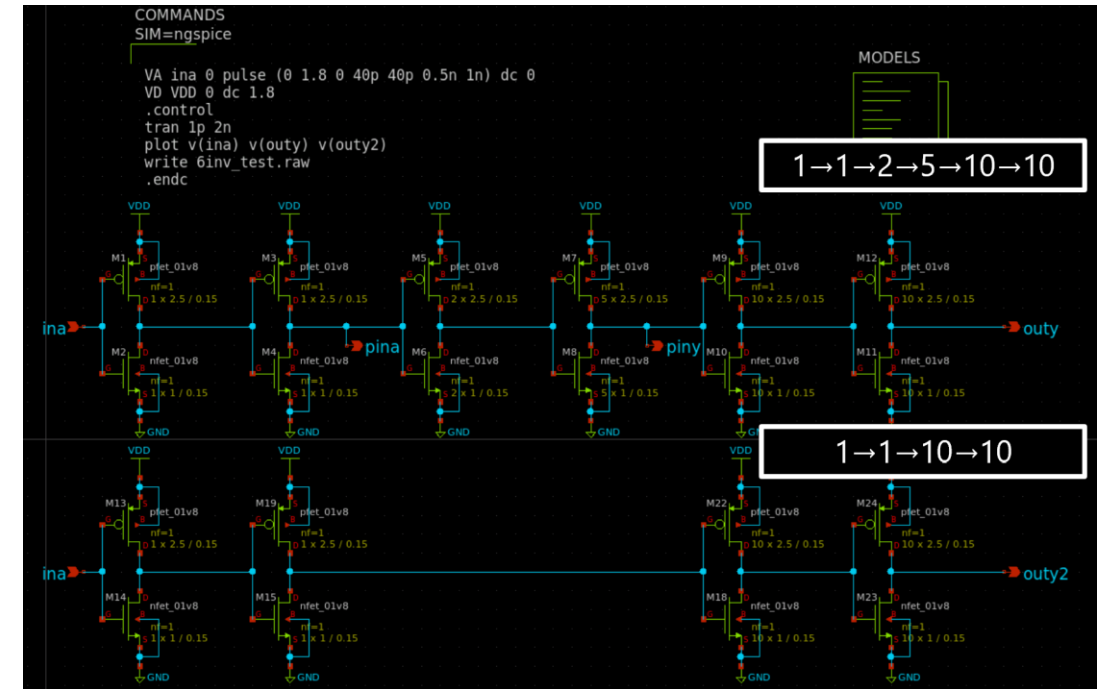


設計ポイント：
Wの値

- 中央の図：PMOS:W=1u, NMOS:W=1u
- 右の図：PMOS:W = 60u, NMOS:W=30u
 - Lは駆動力重視で最小の0.5uとした
 - 回路図とレイアウトでなぜかCapの容量が2倍違い、かなりの寄生容量が載ることを考慮し、必要W/Lの約4倍として設計した

設計のポイント： ファンアウト

- 出力先にどれぐらいの容量負荷が接続されているかを表す単位
- 特性
 - 3倍くらいのサイズにすると一番遅延時間が減る
- 本設計のW
 - 5u -> 10u -> 40u -> 60u
 - 1倍 -> 2倍 -> 4倍 -> 1.5倍





ロジック（デジタル）編

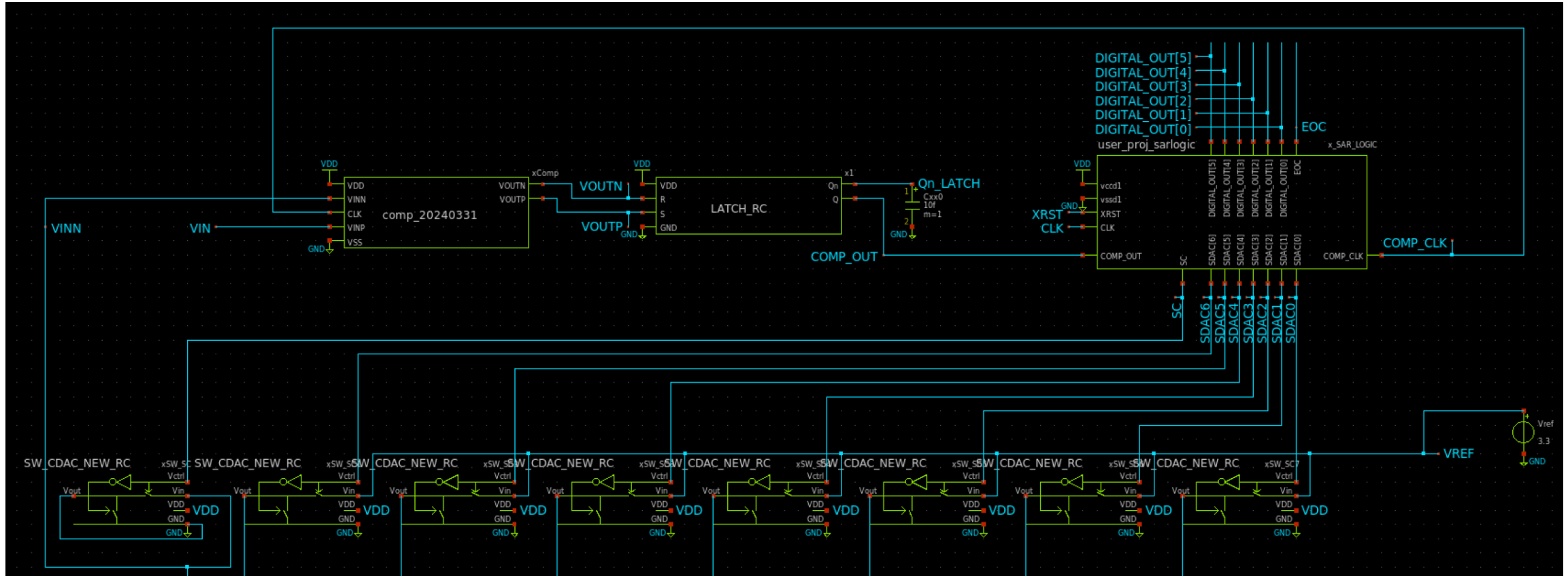




制御器

SAR-LOGIC.vの引数

- module SAR_LOGIC(COMP_OUT, DIGITAL_OUT, COMP_CLK, SC, SDAC, EOC, CLK, XRST);
 - COMP_OUT : ラッチ後のコンパレータ出力
 - DIGITAL_OUT : A/D結果
 - COMP_CLK : コンパレータに入力するCLK
 - SC : CDACの容量のコンパレータ側端子をGNDに落とすSW。HIGHでGNDに落とす想定。
 - SDAC : CDACの容量にVrefを入力するかGNDを入力するか決めるSW。LOWでGNDを入力、HIGHでVrefを入力する想定。
 - BIT_ADC+1個のSW。MSBがCDACの最大の容量、LSBが最小の容量に対応。
 - EOC : End of conversion signal
 - CLK : PLL出力CLK(6の倍数のMHz)
 - XRST : 非同期Reset信号



動作ステップ：1～5を6bit繰り返す

state	動作	内容
1	CDAC リセット	COMP_CLK=0, SC=1, SDAC=0, EOC=0
2	サンプリング解除	SC=0 (Hi-Z)
3	DAC セット	SDAC=SDAC_NEXT
4	コンパレータク ロック High	比較動作開始
5	比較結果取り込み	DIGITAL_OUT_BUF に COMP_OUT をシフト SDAC_NEXTをカウントアップ
6	LSB なら EOC=1	DIGITAL_OUT_BUF を DIGITAL_OUT に反映

```
1 always @(posedge CLK or negedge X_RST) begin
2     if (X_RST == `LOW) begin
3         DIGITAL_OUT <= 0;
4         DIGITAL_OUT_BUF <= 0;
5         COMP_CLK <= `LOW;
6         SC <= `HIGH;
7         EOC <= `LOW;
8         SDAC <= 0;
9         SDAC_NEXT <= 1 << `BIT_ADC;
10    end else if (state == 3'd1) begin
11        // CDAC Reset, CLK@Comparator Low
12        COMP_CLK <= `LOW;
13        SC <= `HIGH;
14        SDAC <= 0;
15        EOC <= `LOW;
16    end else if (state == 3'd2) begin
17        // SC OFF (Vinn = Hi-Z)
18        SC <= `LOW;
19    end else if (state == 3'd3) begin
20        // ref. GEN
21        SDAC <= SDAC_NEXT;
22    end else if (state == 3'd4) begin
23        // CLK@Comparator High
24        COMP_CLK <= `HIGH;
25    end else if (state == 3'd5) begin
26        DIGITAL_OUT_BUF <= {DIGITAL_OUT_BUF[`BIT_ADC-2:0], COMP_OUT};
27        SDAC_NEXT <= next_SDAC (COMP_OUT, ADCount, SDAC);
28    end else if (state == 3'd6) begin
29        if (ADCount == `BIT_ADC-1) begin
30            DIGITAL_OUT <= DIGITAL_OUT_BUF;
31            EOC <= `HIGH;
32        end
33    end
34 end
```

Verilog -> GDS変換の方法

- 2種類
 - TT(TinyTapeout)でのやり方を踏襲した方法
 - メリット
 - 投稿先標準のやり方のため、トラブルがない
 - デメリット
 - サイズやピンの位置が固定のため、無駄なエリアが出来てしまう
 - Chiplgniteでのやり方を踏襲した方法
 - メリット
 - サイズやピンの位置を変更可能なため、最小のサイズに出来る
 - デメリット
 - TTが利用するファブではあるが、バージョンの違いなどがある可能性がある



TT : GDSを生成する

- source ~/ttsetup/venv/bin/activate
- verilogファイルなどを追加した場合はcreate-user-configから実行すること

```
> cd ~/tt[TT_runname]-tt_um_[username]_[projectname]  
> ./tt/tt_tool.py --create-user-config
```

※Dockerが動いている必要がある

○ GDSの生成

```
> ./tt/tt_tool.py --harden
```



TT : GDSを表示する

- OpenRoadやKlayoutで実行する

```
> cd ~/tt[TT_runname]-tt_um_[username]_[projectname]
```

○ OpenRoadをGUIで実行

```
> ./tt/tt_tool.py --open-in-openroad
```

○ Klayoutで実行

```
> ./tt/tt_tool.py --open-in-klayout
```

[illegible]

The image shows a terminal window with a large number of lines of system logs. The logs are organized into columns, with the first column containing timestamps and the second column containing log messages. The text is dense and appears to be a mix of system boot logs and network-related messages. The terminal has a dark background with light-colored text.

GDSの結果

[illegible]

Chiplgnite : nix-shellのセットアップ

- インストール方法
 - https://librelane.readthedocs.io/en/stable/installation/nix_installation/installation_linux.html

```
> sudo apt-get install -y curl
> curl --proto '=https' --tlsv1.2 -fsSL https://artifacts.nixos.org/nix-installer | sh -s -- install --
no-confirm --extra-conf "
  extra-substituters = https://nix-cache.fossi-foundation.org
  extra-trusted-public-keys = nix-cache.fossi-
foundation.org:3+K59iFwXqKsL7BNu6Guy0v+uTlwsxYQxjspXzqLYQs=
  extra-experimental-features = nix-command flakes
"
```

Chiplgnite : LibreLaneのセットアップ

- SAR-Logicのサンプル
 - <https://github.com/ishi-kai/librelane-sar-logic>

```
> cd ~/
> git clone -recursive https://github.com/ishi-kai/librelane-sar-logic

※失敗する場合は、~/librelane-sar-logic/submodules/ 内で
> git clone https://github.com/librelane/librelane
> cd librelane
> git checkout 1e4f4d5bf9d2693798b12dc0c1cd0337ad266a0d
```

Chiplgnite : ビルドする

- SAR-LOGICのソースコード
 - ~/librelane-sar-logic/src/SAR-LOGIC.v
 - 設定
 - ビルド用の各種設定
 - ~/librelane-sar-logic/config/config.json
 - ピンの設定
 - ~/librelane-sar-logic/config/pin_order.cfg

○ nix-shellを起動する

```
> cd ~/librelane-sar-logic/submodules/librelane  
> nix-shell shell.nix
```

○ ビルドする

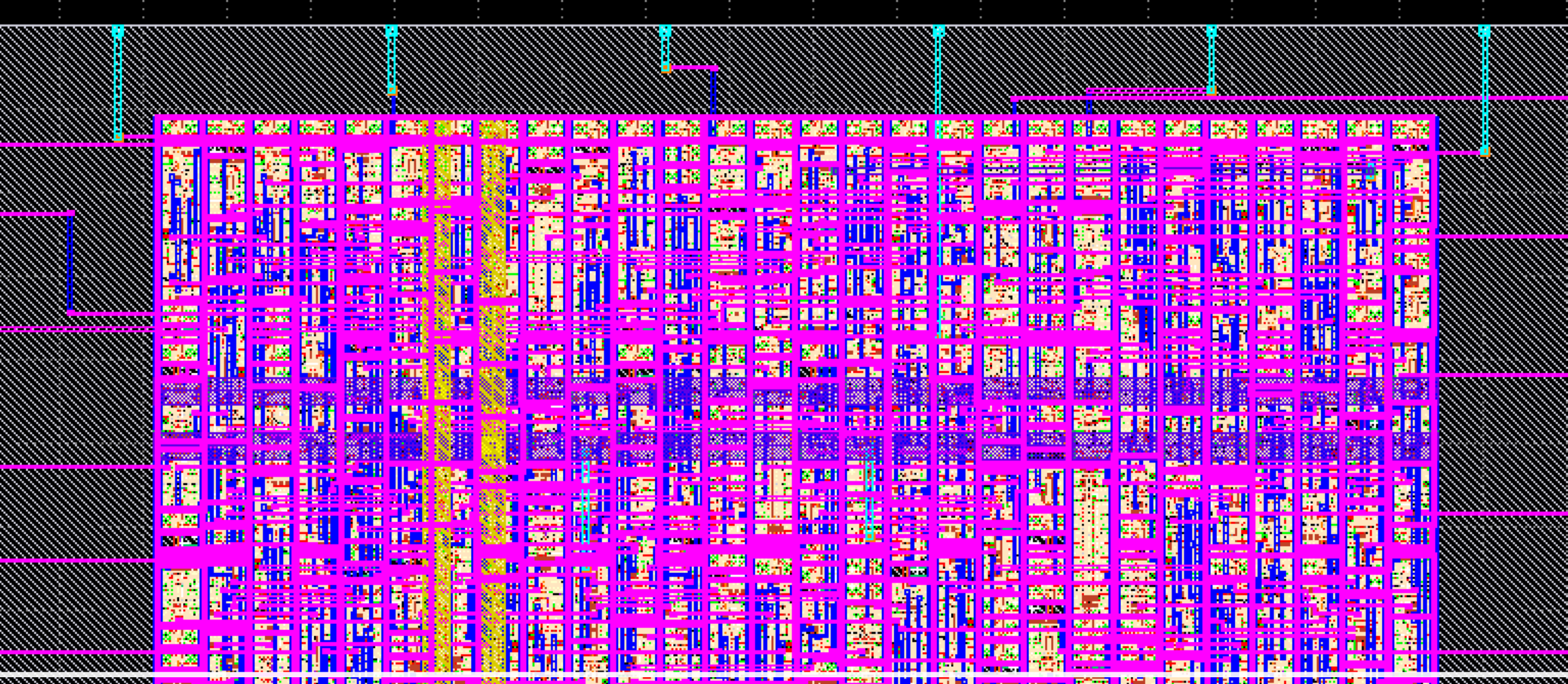
```
> cd ~/librelane-sar-logic/config/  
> librelane config.json -j 12
```


Chiplgnite : ビルドされたファイル

- ビルドされたファイルができる場所
 - ~/librelane-sar-logic/config/runs/RUN_[ビルドした日時]/final/
- ビルドされたファイル
 - GDSファイル（どちらを使ってもよい）
 - gds/SAR_LOGIC.gds
 - klayout_gds/SAR_LOGIC.klayout.gds
 - spiceファイル
 - spice/SAR_LOGIC.spice
 - 最初に空のsubcktは削除してください。
 - このファイルはスタンダードセルの情報が入っていません。下記コマンドで結合してください。

○ ビルドされたspiceファイルに必要なファイルを結合する

```
> cat ~/pdk/sky130A/libs.ref/sky130_fd_sc_hd/spice/sky130_*.spice SAR_LOGIC.spice > SAR_LOGIC_FULL.spice
```



GDSの結果



各種検証を実施する方法

- LVS
 - スタセルと統合したSAR_LOGIC.spiceを~/xschem/simulations/ にコピー
 - メニュー「sky130 PDK」->「Make netlist for Klayout LVS」を実行
 - メニュー「sky130 PDK」->「Run Klayout LVS」を実行
 - DRC
 - メニュー「Tools」->「DRC」->「[klayout dir]/drc/sky130A.lydrc」を実行
 - PEX
 - メニュー「sky130 PDK」->「Run Magic PEX(RC extraction)」を実行
 - C extractionはキャパシタのみを対象としたPEX
 - ※メニュー「Elabless sky130」がある場合はそちらにあるメニューを利用してもよい
-

比較器編

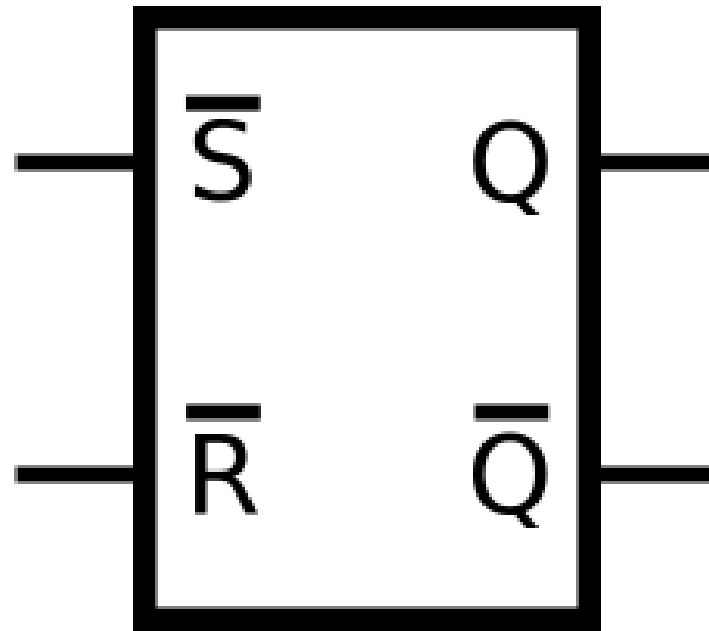


ラ ッ チ 回 路

ラッチ回路

入力を停止しても直前の状態を
保持し続けることができる

「記憶」の最小単位



Dラッチ

安藤さん記事より

■ Dラッチ

図3にDラッチの回路例を示します。蝶ネクタイのようなシンボルはトランSMissionゲートを、インバータの上下に線が伸びたシンボルはクロックドインバータを表しています。

制御信号 E の入力がトランSMissionゲートとクロックドインバータで逆になっている点に注目してください。

E がHighの時、入力側のトランSMissionゲートが導通し、入力 D が内部ノード X にそのまま伝わります。このとき、フィードバック側のクロックドインバータはハイインピーダンスになっているため、信号の衝突は起きません。

E がLowの時、トランSMissionゲートが遮断され、入力が切り離されます。代わりにクロックドインバータが動作し、出力 $\bar{Q}$ を反転して再び X に戻すループが形成されます。こうして、直前のデータが回路内で保持（ラッチ）されます。

なお、入力段のトランSMissionゲートをクロックドインバータに置き換える設計もあります。トランジスタ数は増えますが、入力信号を再駆動（バッファ）して受け取るため、動作が安定します。

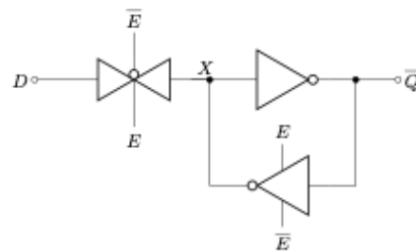


図3 Dラッチの回路図

■ トランSMissionゲート

図2左はトランSMissionゲートです。制御信号 E がHighの時は入力 A をそのまま出力 Q へ通過させ、 E がLowの時は Q をハイインピーダンス状態にします。

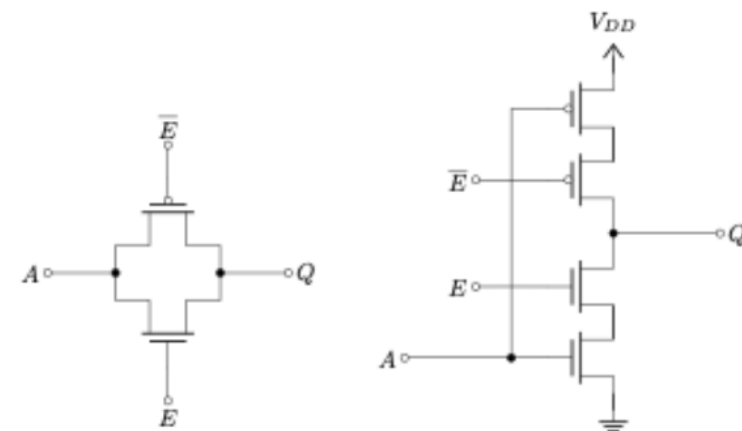
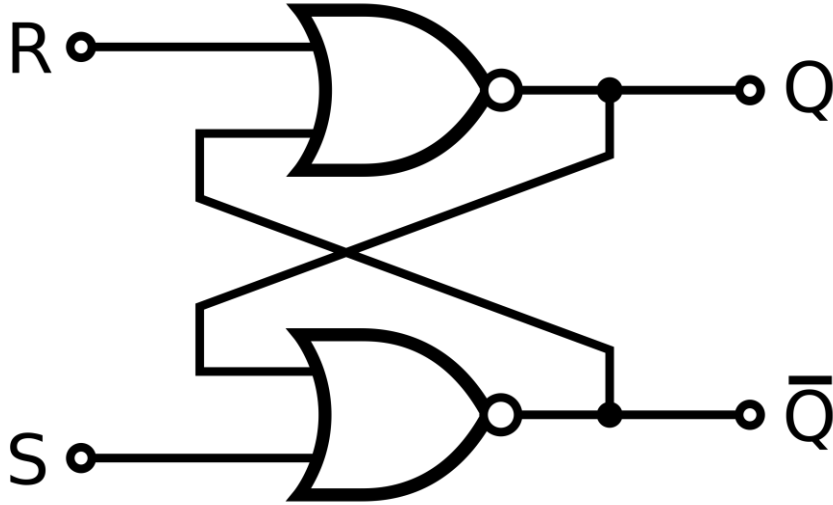


図2 トランSMissionゲートの回路図（左）とクロックドインバータの回路図（右）



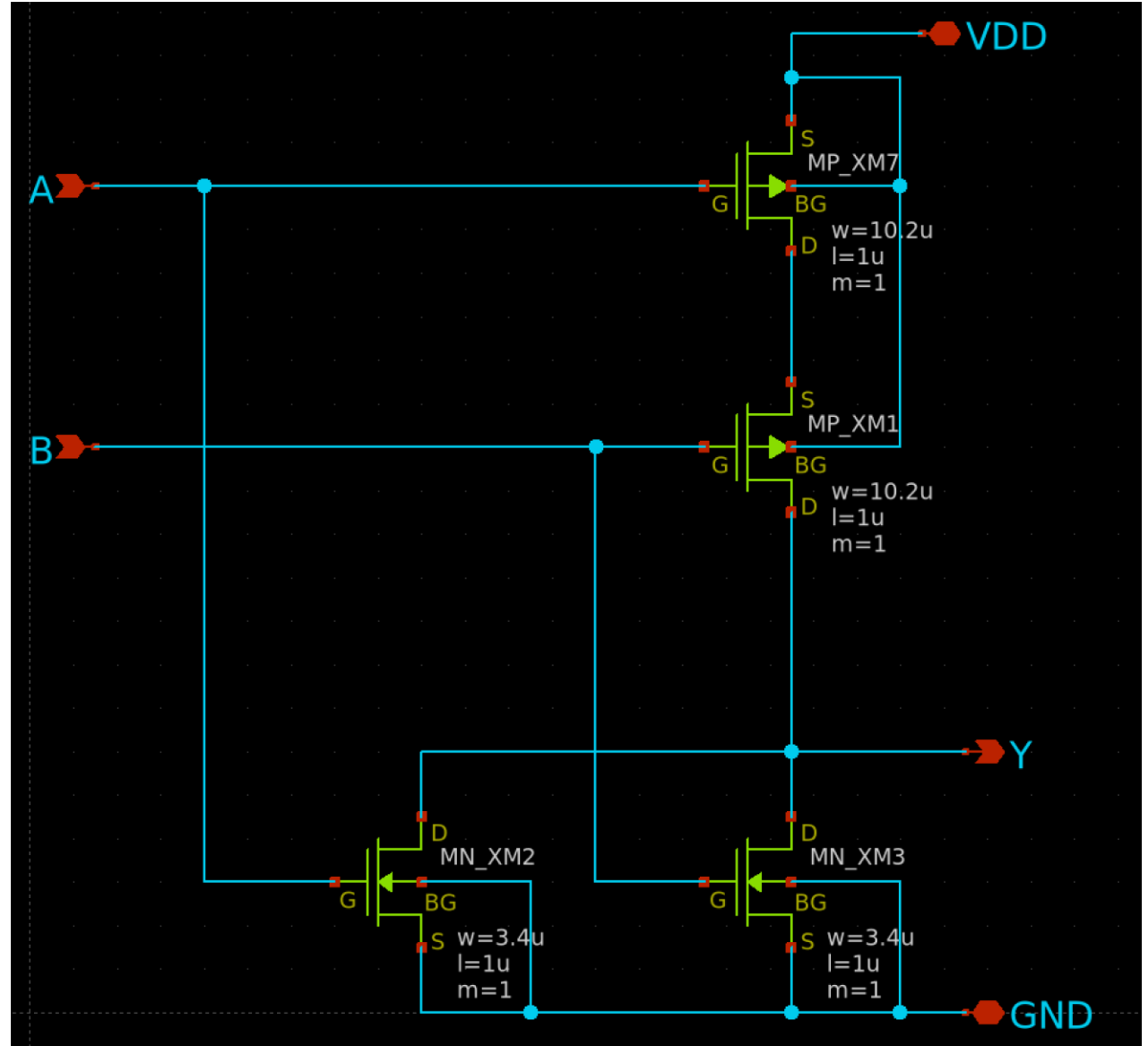
R (Reset)	S (Set)	Q (出力)	状態
0	0	前の状態を保持	保持 (Hold)
0	1	1	セット (Set)
1	0	0	リセット (Reset)
1	1	0 (不安定)	禁止 (Invalid)

SRラッチ

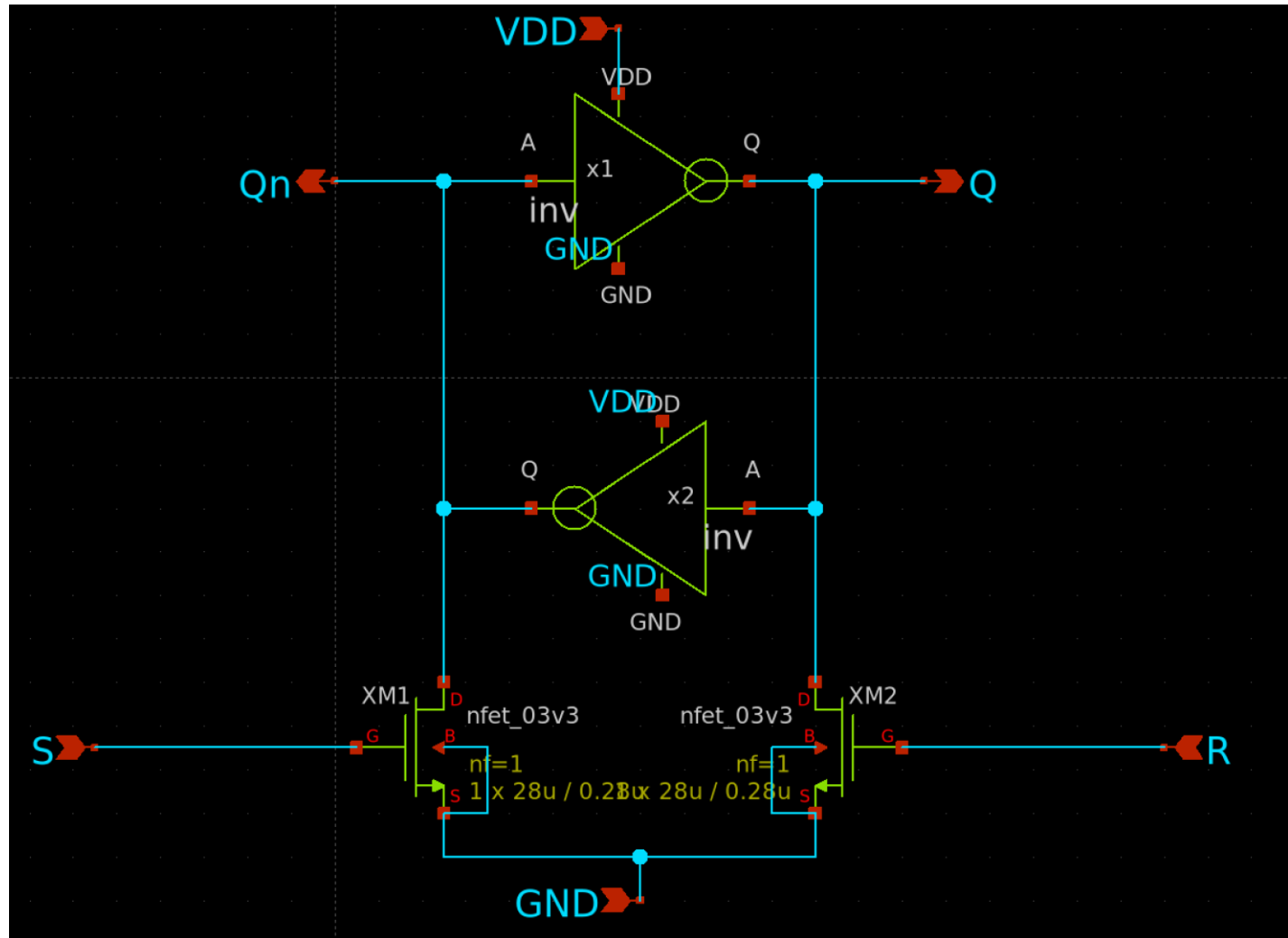
一般的な論理ゲートでの実装としては、たすきがけになったペアのNORゲートで構成する。

状態は、Qの記号を付した端子から出力される。

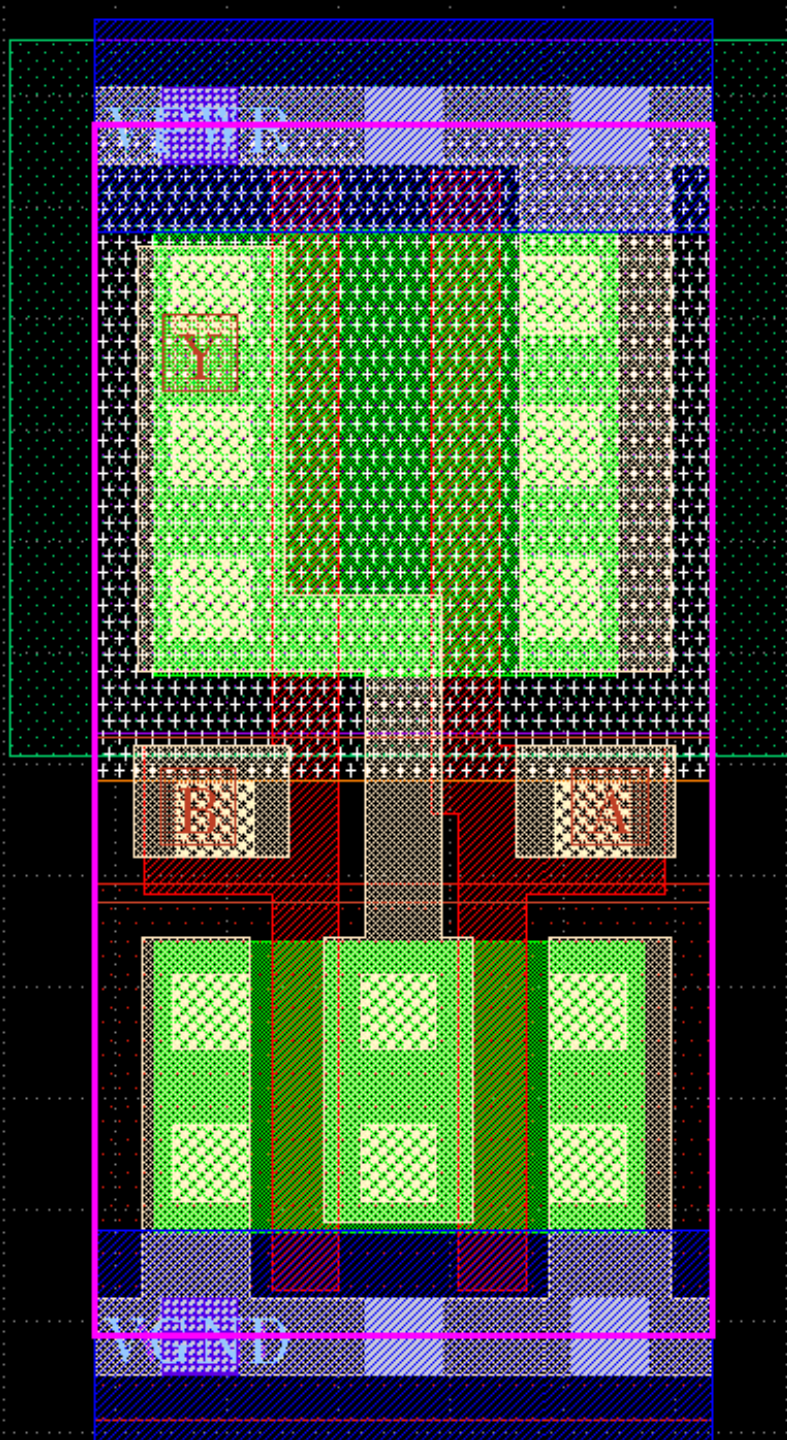
NOR回路



インバータ
を使う
パターン



NORスタセル



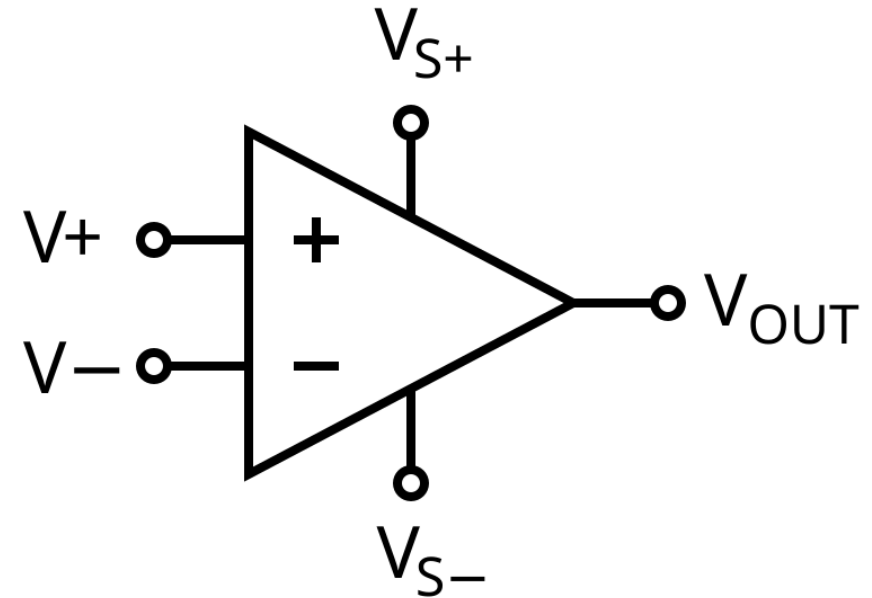
sky130_fd_sc_hd.sky130_fd_sc_hd_nor2_1



比較器（コンパレータ）

一般的なコンパレータ

- 二つの電圧または電流を比較し、どちらが大きいかで出力が切り替わる素子である。
 - 最もシンプルな構成は、負帰還をかけない状態のオペアンプ





問題点

- 0-5V動作できない
 - 対策：Rail-to-Rail化する
 - 常に動作しているため消費電力が大きい
 - 対策：クロックドコンパレータにする
-

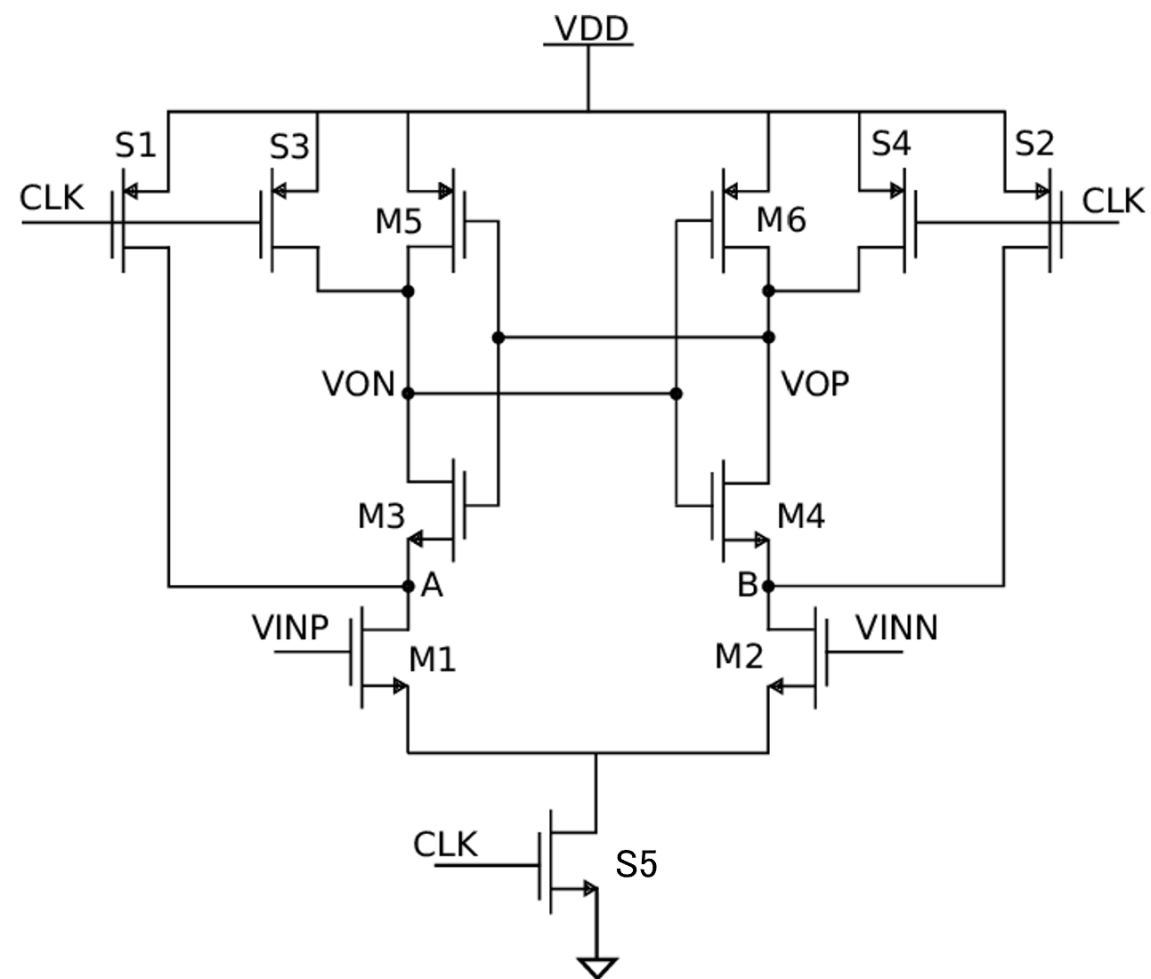


クロックドコンパレータ

クロックドコンパレータ

- クロック信号に同期して動作し、正帰還を利用して微小な差電圧を極めて短時間でロジックレベルまで増幅する回路
 - 高速・低消費電力ADCのSAR-ADCにおいて、最も多用される回路形式の一つ

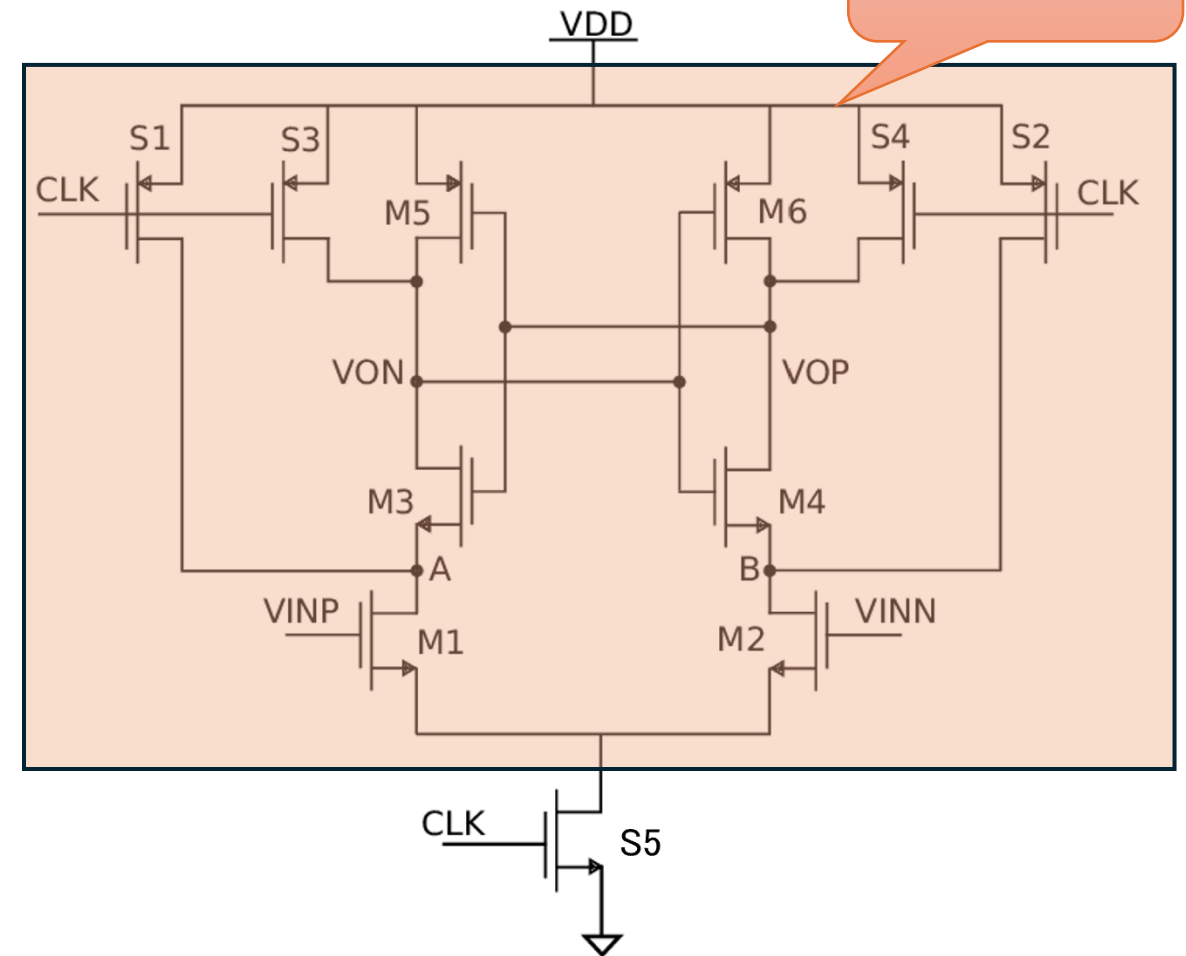
StrongARM型 クロックド コンパレータ



動作内容： Clock=Low

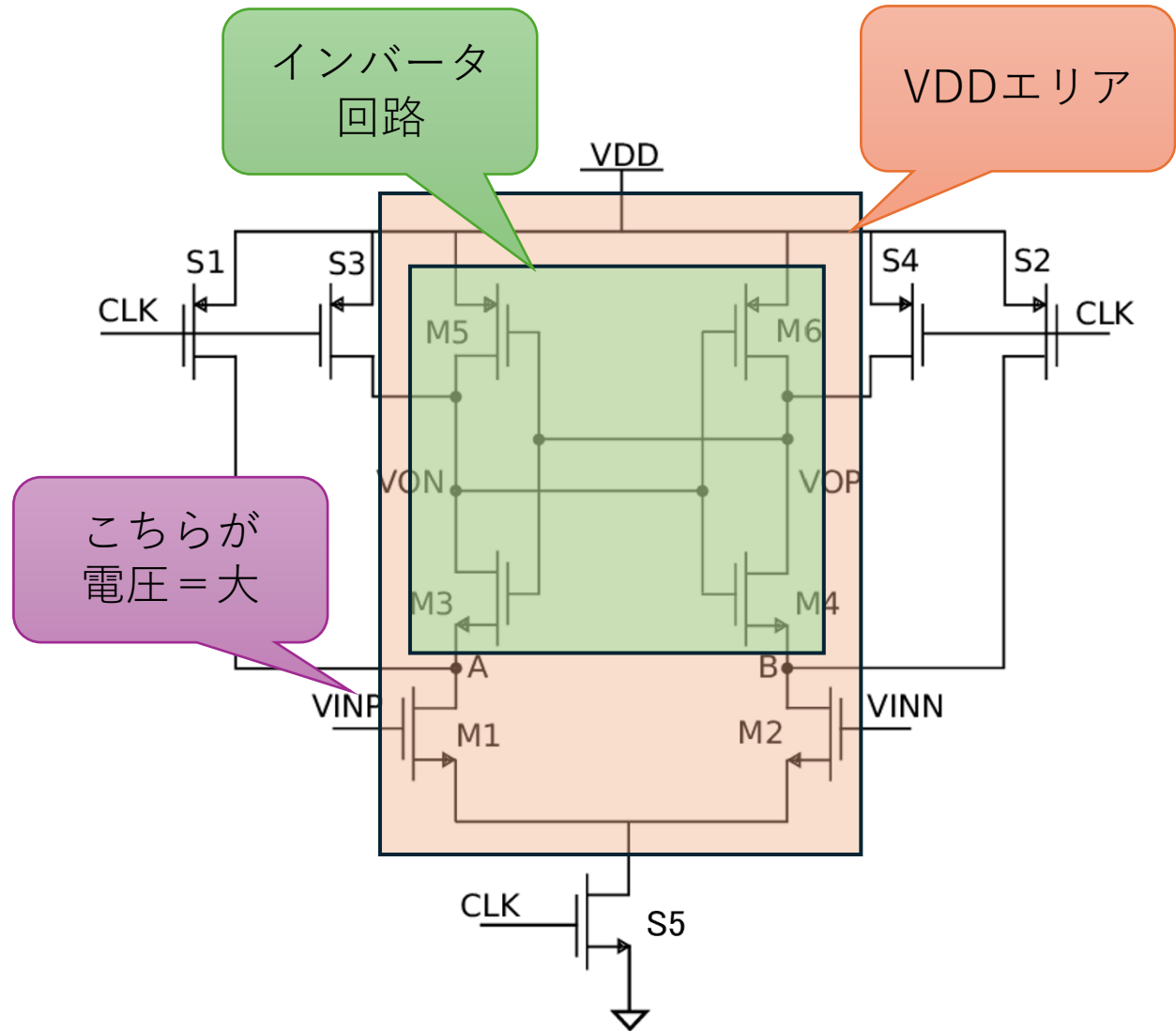
1. S5がOffになるため、VSS(GND)が切り離される。
2. S1～S4がOnになるため、VON,VOP,A,BがVDDになる。
 1. プリチャージ動作
3. 全体がVDDになるため動作が停止する。

※Sがスイッチ的な動作をするFET



動作内容：
Clock=High

1. S1～S4がOffになり、VDDが切り離される。
2. S5がOnになり、VSSへ接続される。
 1. この瞬間は、まだ回路はVDDであることがポイント
3. もし $V_{INP} > V_{INN}$ であれば、M1の方がM2よりも多く電流が流れる。その結果、ノードAの電圧がノードBよりも早く下がることになる。
4. ノードA, Bが下がっていくと、M3, M4がオンになり、次に出力VON, VOPも放電され始める。
5. わずかに早く下がった方の電圧（この場合VON）が、対向するインバータのPMOS（M6）をオンにし、NMOS（M4）をオフにします。これにより、VOPがVDD、VONがVSSで確定する。



設計方針

このペアは同サイズ
ずれるとオフセットとなっ
てしまう

インスタンス	種類	役割・設計意図
M1, M2	NMOS	最重要： 感度向上とオフセット低減のため最大に。
M3, M4	NMOS	ラッチ速度を稼ぐため、M1,M2の半分程度。
M5, M6	PMOS	強すぎると反転を阻害するため、NMOS(M3,M4)の1～1.5倍くらい。
S1, S2	PMOS	寄生容量を抑えつつ、リセット期間内に充電完了させるため、最小サイズでよい。
S3, S4	PMOS	同上。
S5 (Tail)	NMOS	回路全体の電流を引くため、低抵抗（大サイズ）に。



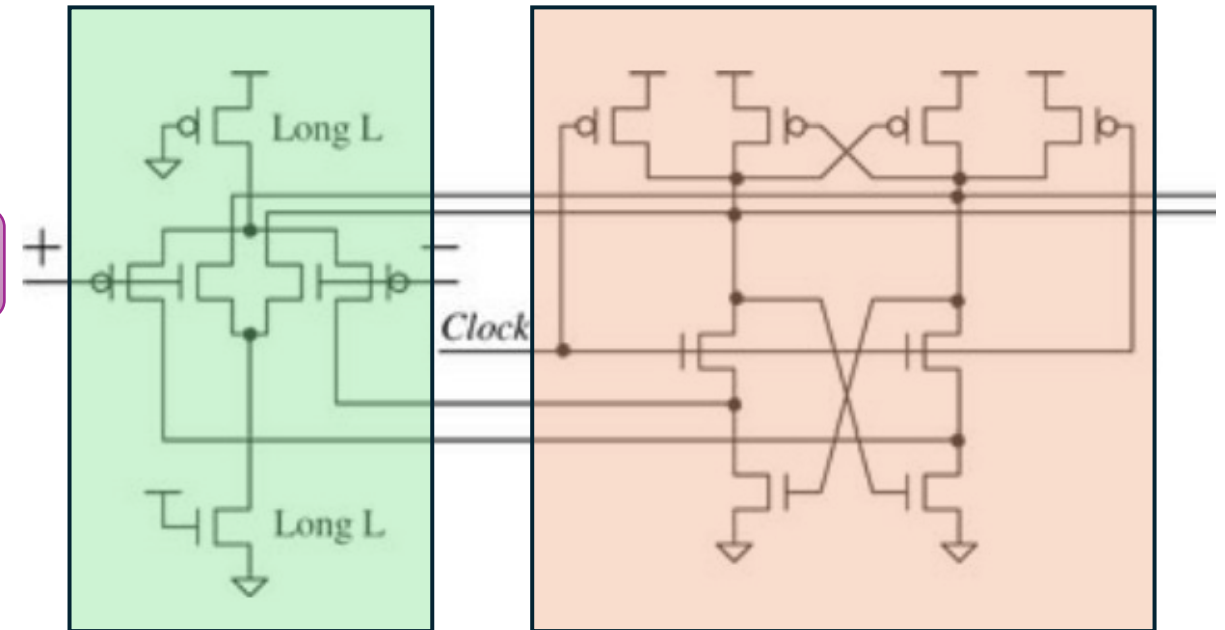
問題点

- キックバックノイズ
 - 内部ノードの大きな電圧変化が、入力端子側へノイズとして逆流する
 - Rail-to-Rail動作ではない
 - 下位1bitがつぶれてしまう
-

Rail-to-Rail入力差動対 クロックコンパレータ

- オレンジの部分
 - 先ほど解説したものと同一役割の部分
- 緑の部分
 - 中央のP,NMOSペアの動作
 - 入力が低い時: PMOSペアが動き、NMOSペアはお休み
 - 入力が高い時: NMOSペアが動き、PMOSペアはお休み
 - 入力が中間の時: 両方のペアが協力して動く
 - 中央のラッチ段との動作
 - PMOS側: 上側から電流を流し込む（充電を加速させる）役割
 - NMOS側: 下側へ電流を引き抜く（放電を加速させる）役割
 - により、高速にコンパレータとして動作する
 - 「Long L」部分 = 「テール電流源」
 - 高インピーダンスのためキックバックノイズを低減する

Rail-to-Rail部



設計方針

インスタンス	種類	役割・設計意図
電流源	Long L (P & N)	W=必要な電流が流せるサイズ L=短チャンネル効果対策などを考慮した最小値の3~5倍
入力対	PMOS Pair	W=必要なクロックで動作するサイズ L=短チャンネル効果対策などを考慮した最小値
入力対	NMOS Pair	P/Nバランスを考慮した値
ラッチ	クロス結合部	L=短チャンネル効果対策などを考慮した最小値 W/L比=電流源と同じにしてラッチ動作すればOK。しない内調整する。



Rail-to-Rail入力差動対 クロックドコンパレータ設計：スペック

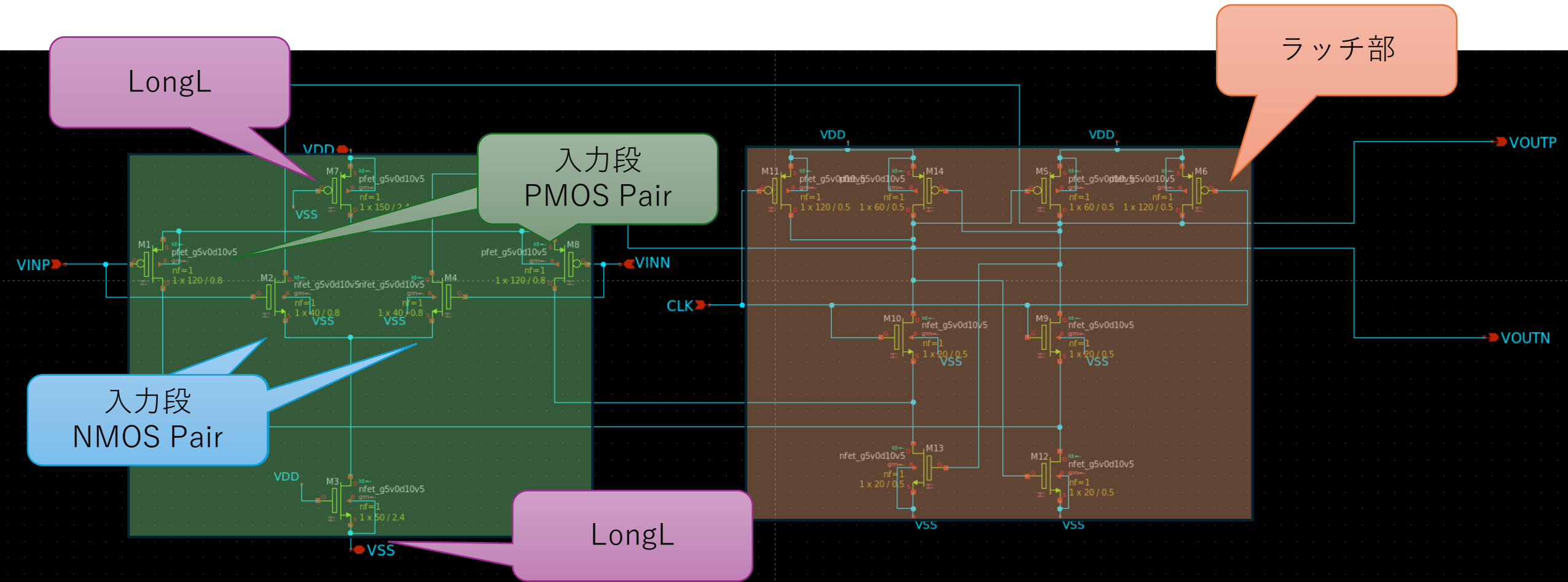
可変Ref電圧：
1.8V-5.0V

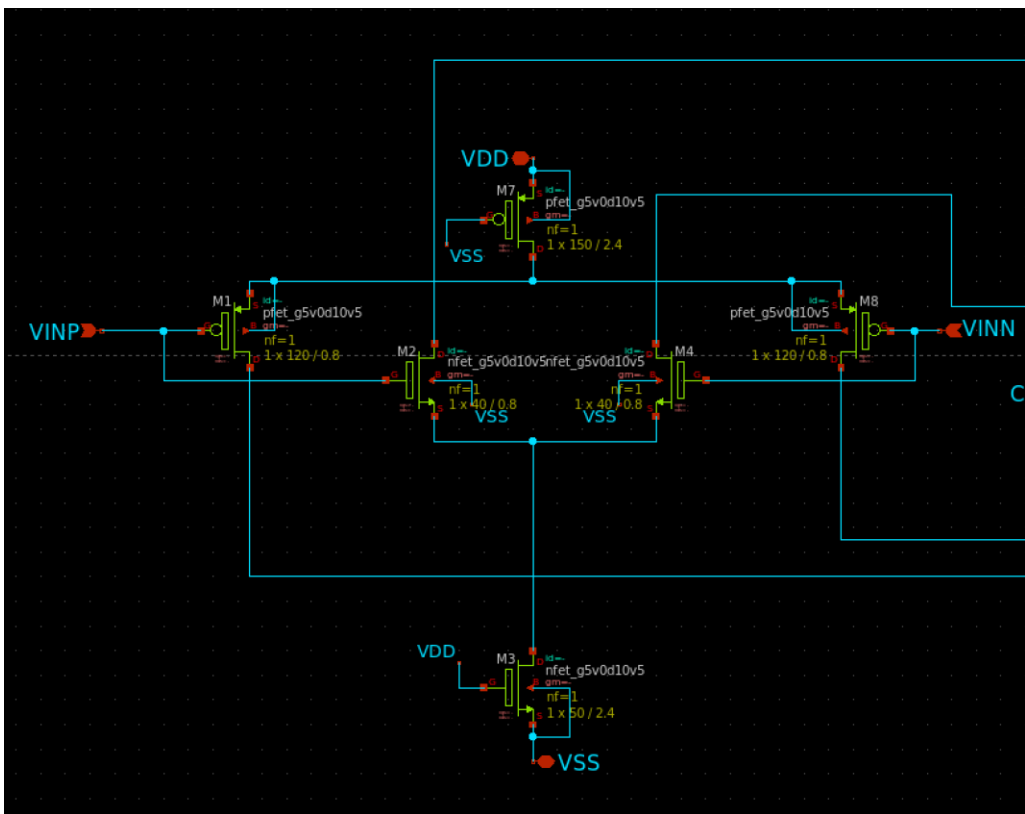
Rail-to-Rail動作：
0-5V

ビット数：6bit

クロック：1.2MHz

シミュレーション回路例

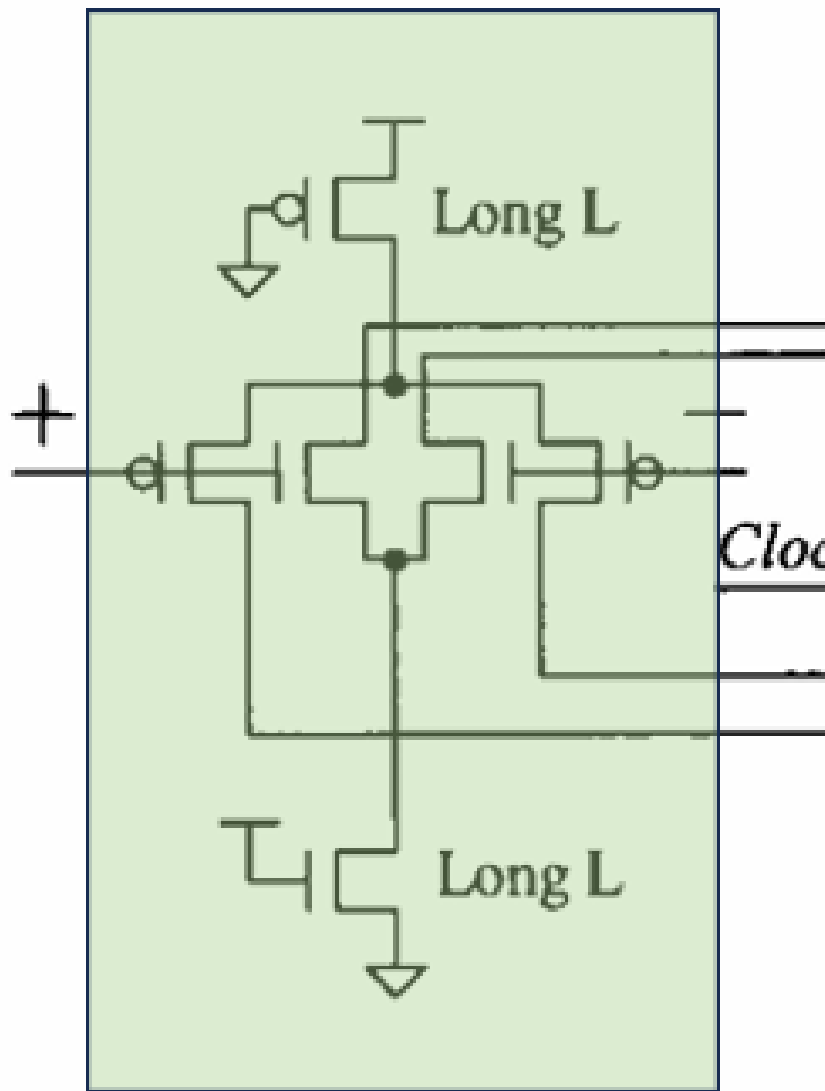




入力共通モード電圧	主に動作する入力対
低い領域	PMOS入力差動対
中間領域	NMOS + PMOS入力差動対
高い領域	NMOS入力差動対

入力段： Rail-to-Rail入力差動対

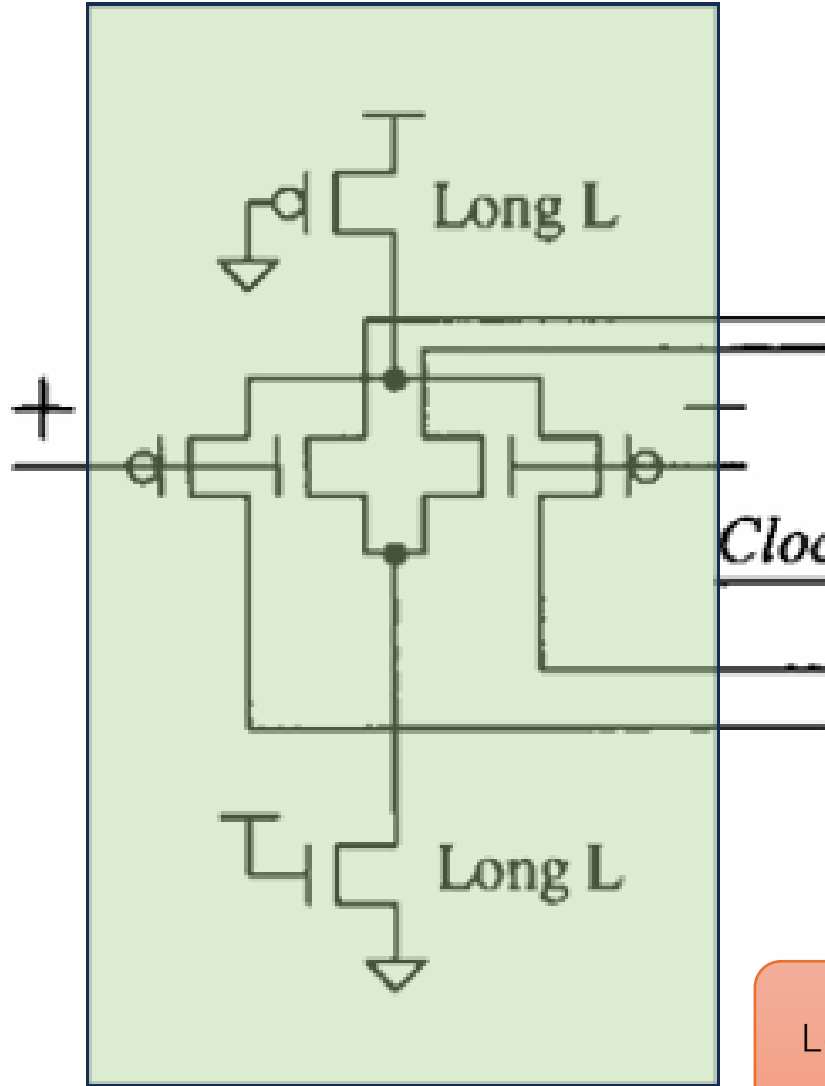
- SAR ADCでは、CDAC出力が広い電圧範囲を動く場合がある。
- そのため、NMOS入力差動対だけ、またはPMOS入力差動対だけでは、入力範囲が不足することがある。
- そこで、NMOS入力差動対とPMOS入力差動対を組み合わせたRail-to-Rail入力差動対を用いる。



入力段：LongL

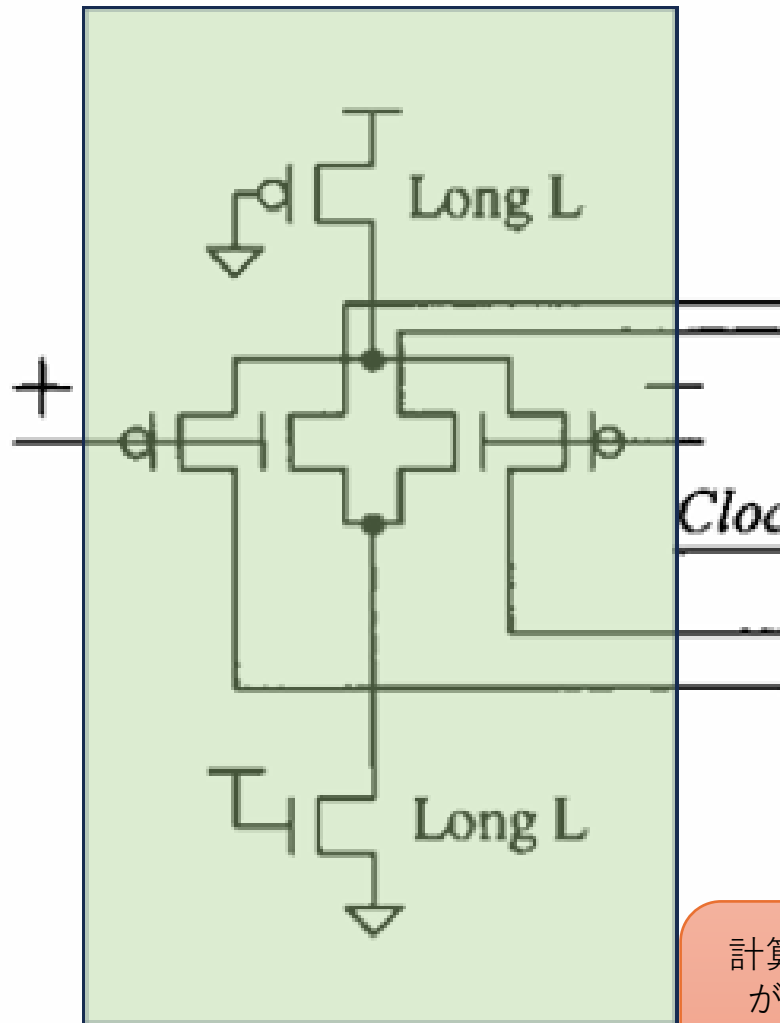
- FETの出力インピーダンスは $r_o \propto L$ であり、 L が長いほど飽和領域での特性がフラットになる。よって、
 - メリット
 - しきい値ばらつきを抑えやすい
 - 入力換算オフセットを低減しやすい
 - チャネル長変調が小さくなる
 - アナログ特性が安定しやすい
 - レイアウトばらつきの影響を抑えやすい
 - デメリット
 - ゲート容量が増え、速度は低下しやすい
- したがって、入力段ではオフセットと速度のバランスを見ながら L を決める

入力段：PairのLの設計



- 基本的には速度を重視する場合は短めのLを用いる
 - 入力差動対としてoffsetやmatchingを改善したい場合は、少し長めのLを使う
 - Pairを大きくしすぎたり、Lを長くしすぎたりすると、寄生容量が増加して速度が低下する点に注意する
- 経験則的には下記のあたりから設計を始めるとよい
 - $L = L_{min} \sim 2L_{min}$
- 本設計ではoffsetやmatchingを考慮し $L = 0.8$ とする
 - 設計幅が無いいため決め打ちで問題無い

LongLより短い必要があるため



入力段： LongLのLの設計

- LongLのLの値は、シミュレーション的に決定する
- そこで、経験則的には下記のあたりから設計を始めるとよい
 - $LongL = 3 \sim 5 \times L$
- 本設計では3倍の2.4uとする
- ゲート容量低減のため最小とする

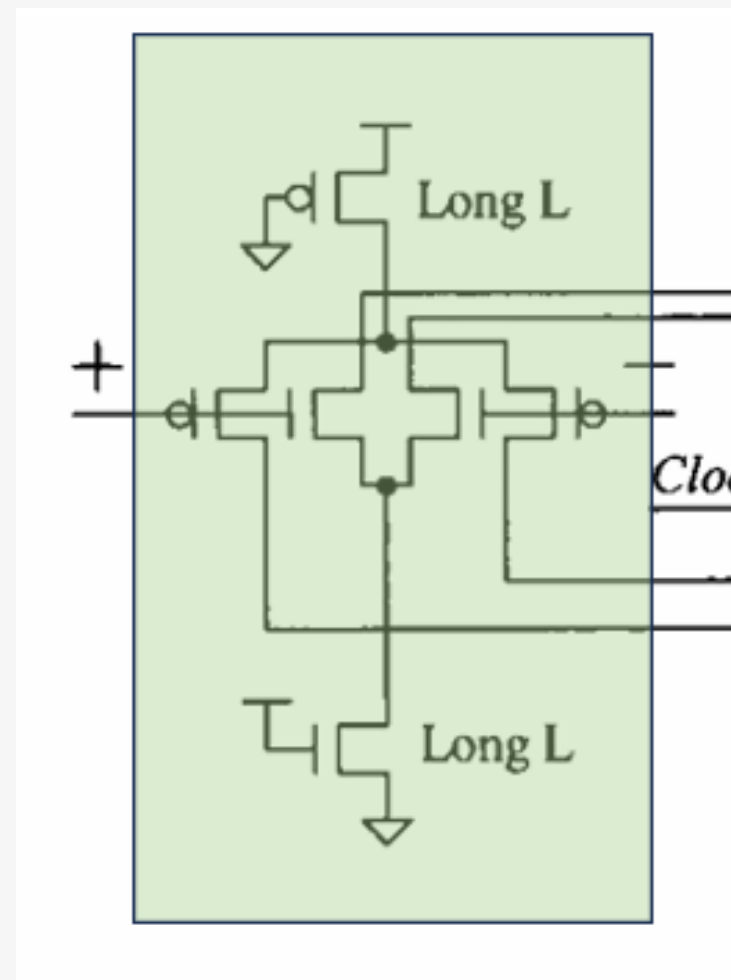
計算から求める手法もあるが、選択幅が小さいためシミュレーションによる調整が楽である

入力段：最小の L_{min} の決定

- 入力段はアナログ的な動作をする
 - 短チャネル効果やチャネル長変調を考慮
 - 5.0V素子の最小Lは0.5 μ であり、本スペックではこのサイズで問題ない
 - 設計方法はOPAMPハンズオン資料を参照
 - 1.8V～5.0Vまでの入力を考慮
 - 1.8Vの場合、5.0V時に比べLSBが1/3まで小さくなる
 - 誤差を減らすために、ペルグロムの法則（後述）より $W \times L$ の面積を増やす必要がある
 - FETの性能は W/L で決めるため、Lを大きくするとWも大きくなる点に注意
 - 以上から最小の L_{min} は、5.0V素子の最小単位の0.5 μ とする
-

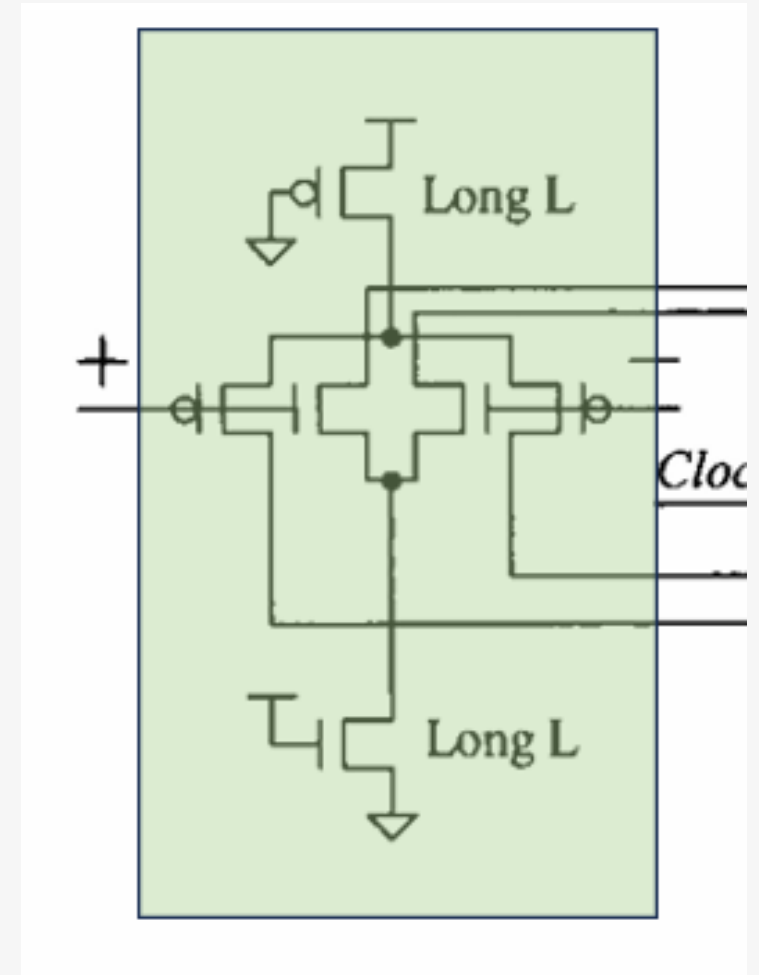
入力段：LongLのWの設計

- **W** は必要な動作電流を流せるサイズに設定する
 - 電流が小さすぎる場合
 - 入力差動対やラッチ部に十分な電流が供給されず、再生動作が遅くなる
 - 電流が大きすぎる場合
 - 消費電力が増加する
 - ラッチ動作が強くなりすぎることで、入力側へ戻る **kickback noise** が大きくなる
- したがって、Wは **速度を満たす最小限の電流** になるように調整する



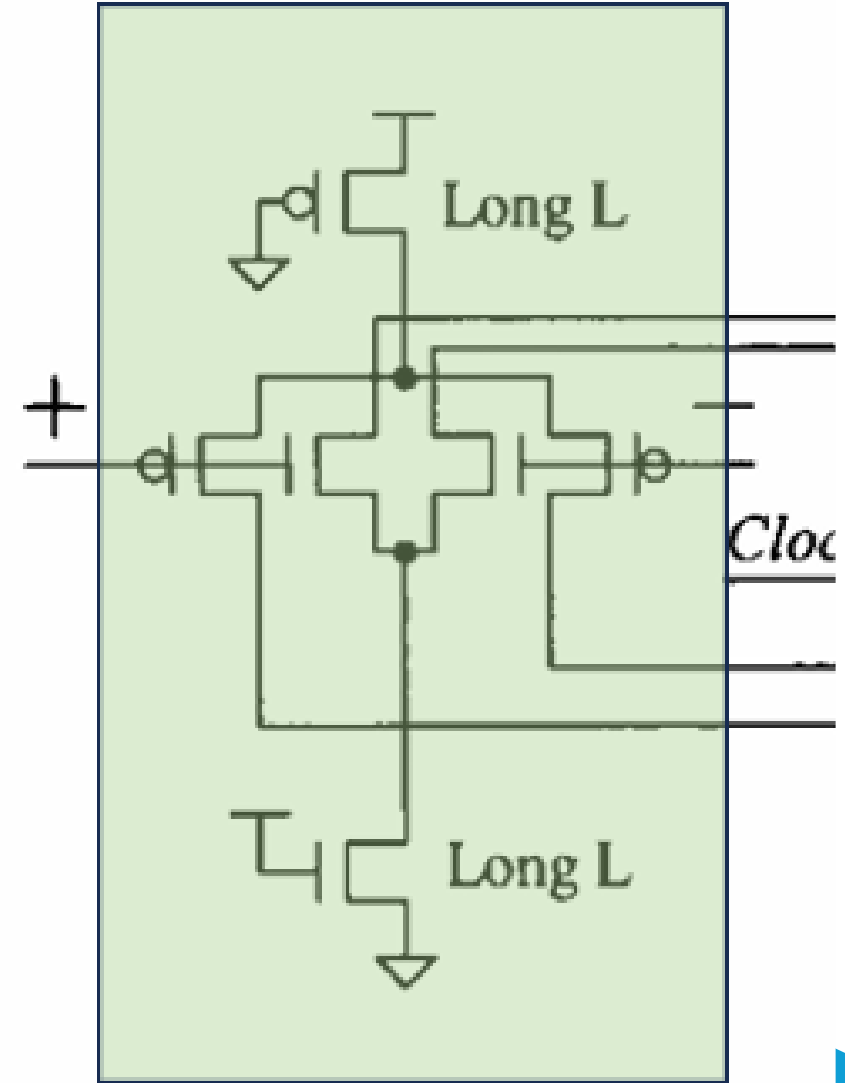
入力段：LongLのWの設計

- 必要なテール電流 I_{tail} の設計
 - $I_{tail} \approx C_{total} \times \frac{\Delta V}{\Delta t} = 150fF \times \frac{5.0V}{7.5ns} = 100\mu A$
 - C_{total} = 寄生容量
 - 5.0V素子はゲート酸化膜が厚いため150fFとした
 - Δt = 評価相での判定時間
 - 1.2MHz(約416ns)の1/100で計算のしやすさで7.5ns
- W_n の設計
 - $I_{tail} = \frac{1}{2} \mu C_{ox} \frac{W}{L} V_{OV}^2$
 - NMOSの $\mu C_{ox} = 60 \frac{\mu A}{V^2}$ 、 $V_{OV} = 0.4V$
 - ngspiceのモデルから求める方法はOPAMPハンズオン参照
 - $W_n = \frac{2I_{tail}L}{\mu C_{ox}V_{OV}^2} = 50.0\mu m$
- よって、本設計では $W_n = 50\mu m$, $W_p = 150\mu m$ とする
 - 電子の移動速度の違いより、 $W_p:W_n \doteq 3:1$



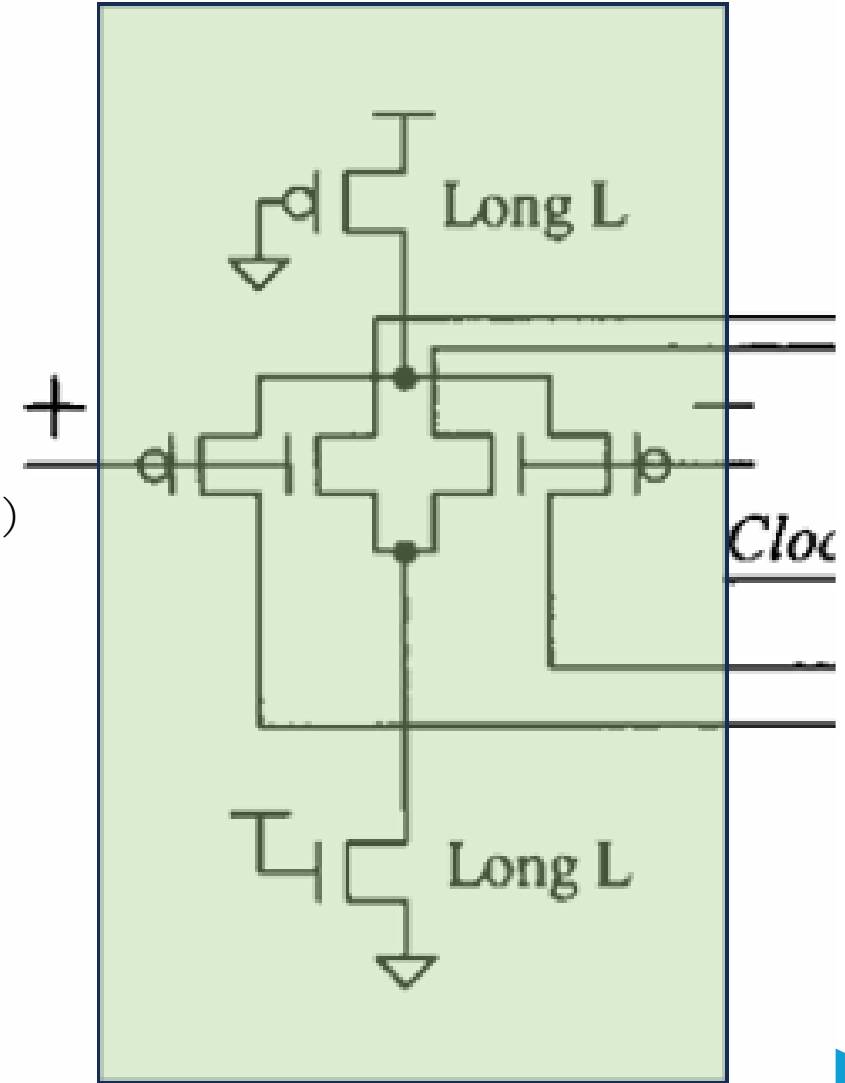
入力段：PairのWの設計 周波数から求める

- 起動までの時間（遅延時間 $\div$ 1/周波数） $t_{delay} \propto \frac{C_{load}}{g_m}$
 - C_{load} = 内部寄生容量, $g_m = \sqrt{2\mu C_{ox} \frac{W}{L} I_D}$, $I_D = \frac{I_{tail}}{2}$
 - 近似式 $g_m \approx \frac{C_{load}}{t_{delay}} \approx \frac{200fF}{4ns} = 50\mu S$
 - 1.2MHz（周期833ns）の評価相416nsの1/100を目標値とした
- $W = \frac{Lg_m^2}{\mu C_{ox} I_{tail}}$
 - $W_n = \frac{Lg_m^2}{\mu C_{ox} I_{tail}} = \frac{0.8 \times 50\mu^2}{60\mu \times 100\mu} = 0.33\mu m$
- 以上から、最小単位より小さいのでこのクロックにおいては考慮の必要はない



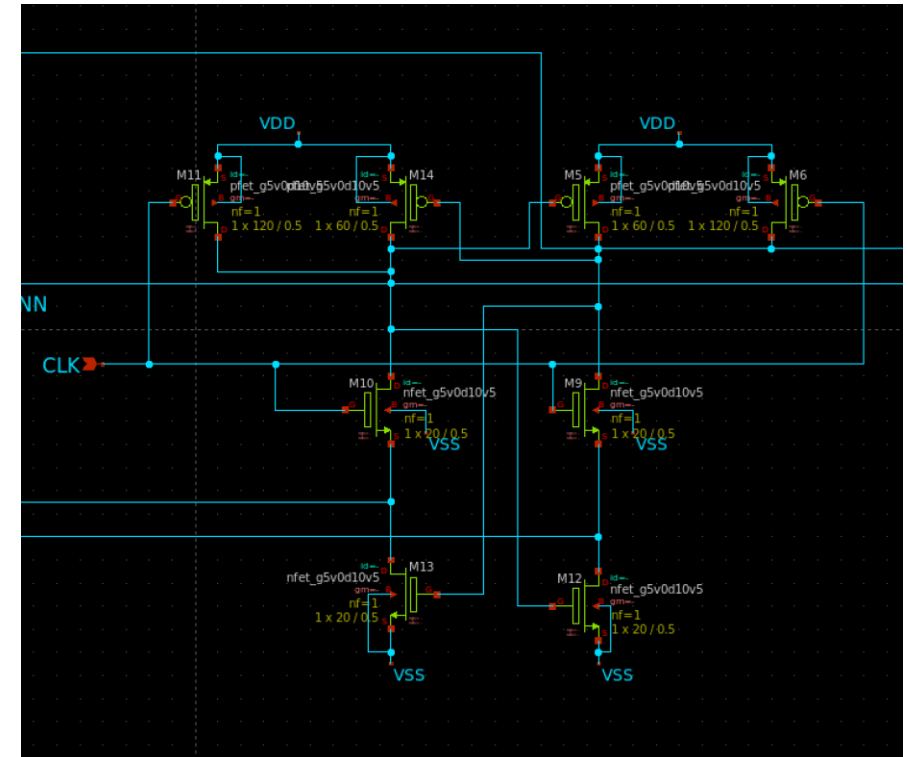
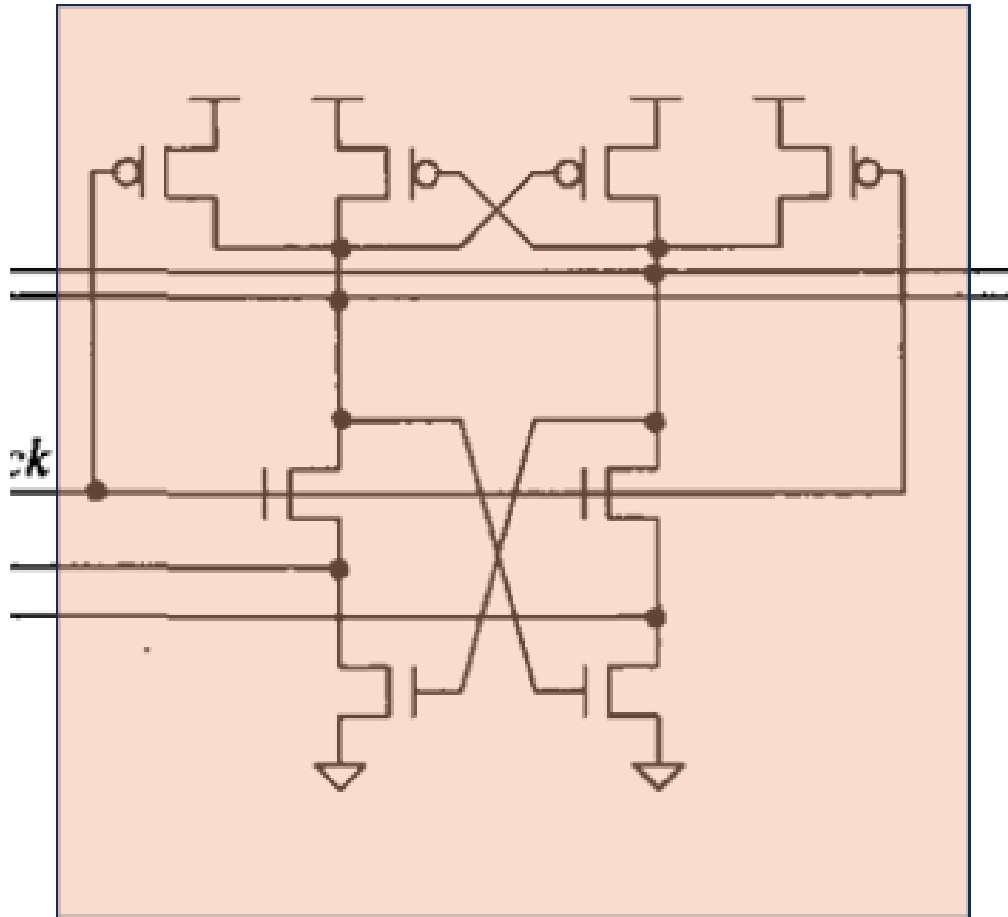
入力段：PairのWの設計 精度から求める

- ペルグロムの法則 $W = \frac{A_{V_{th}}^2}{L\sigma_{off}^2}$
 - $A_{V_{th}}$ = プロセス固有のミスマッチ係数 (PDK記載)
 - 5.0V NMOSの $A_{V_{th}} = 15mV \cdot \mu m$ (1.8Vは $4.5mV \cdot \mu m$ のため5.0Vを採用)
 - σ_{off} = 許容出来るオフセット電圧の標準偏差
 - $1LSB(1.8V) = \frac{1.8V}{2^6} = 28.1mV$, $3\sigma_{off} = \frac{1}{2}LSB = 14.05mV$, $\sigma_{off} = 4.68mV$
- $W_n = \frac{0.015^2}{0.8 \times 0.00468^2} \approx 12.84\mu m$
- Long L の $W >$ Pairの W とする必要がある
- 電子の移動速度の違いより、 $W_p:W_n \cong 3:1$
- 以上から、3倍の余裕を見て $W_n = 40\mu m, W_p = 120\mu m$ とする



ラッチ部：Lの設計

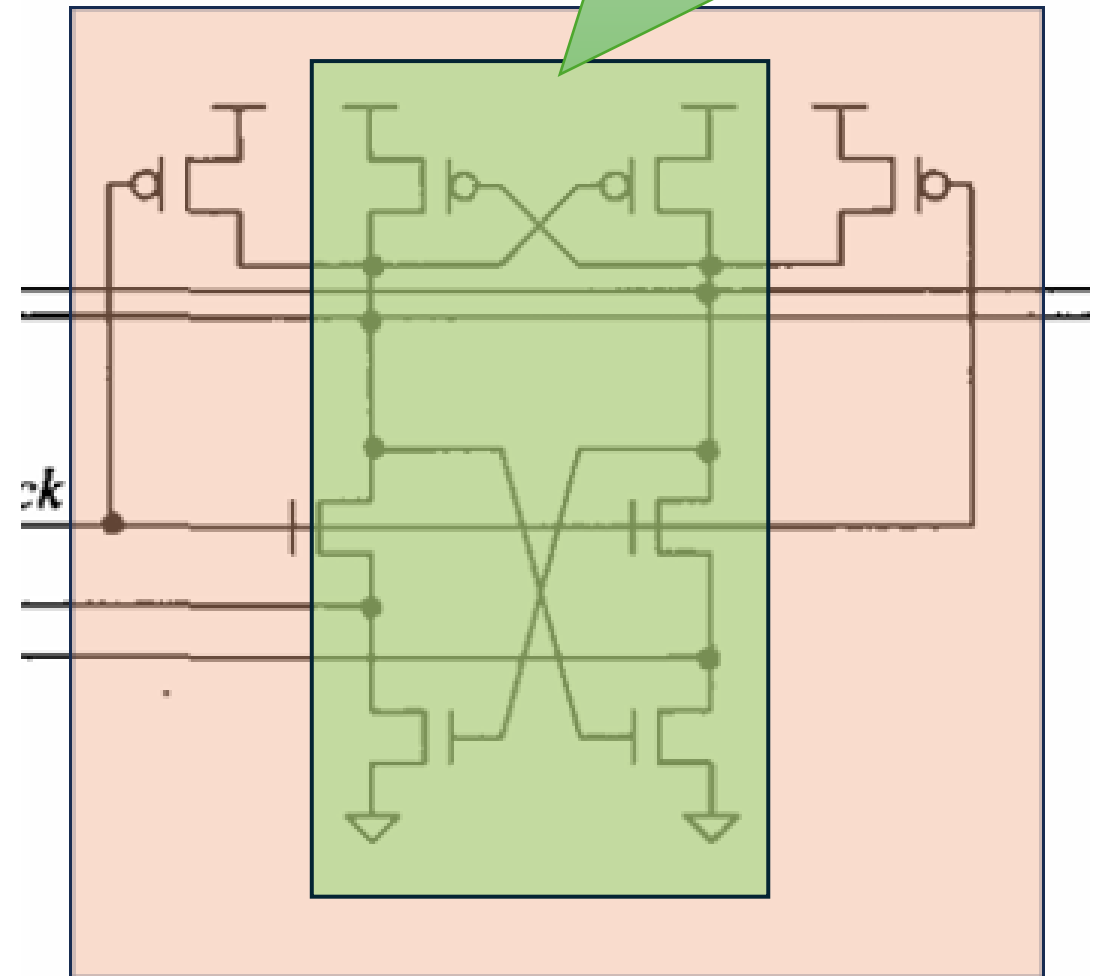
- ラッチ部の基本構造は「クロス結合インバータ」であり、ロジック回路的な動作をする
- そのため、Lは最小値とする
 - $L=0.5\mu\text{m}$



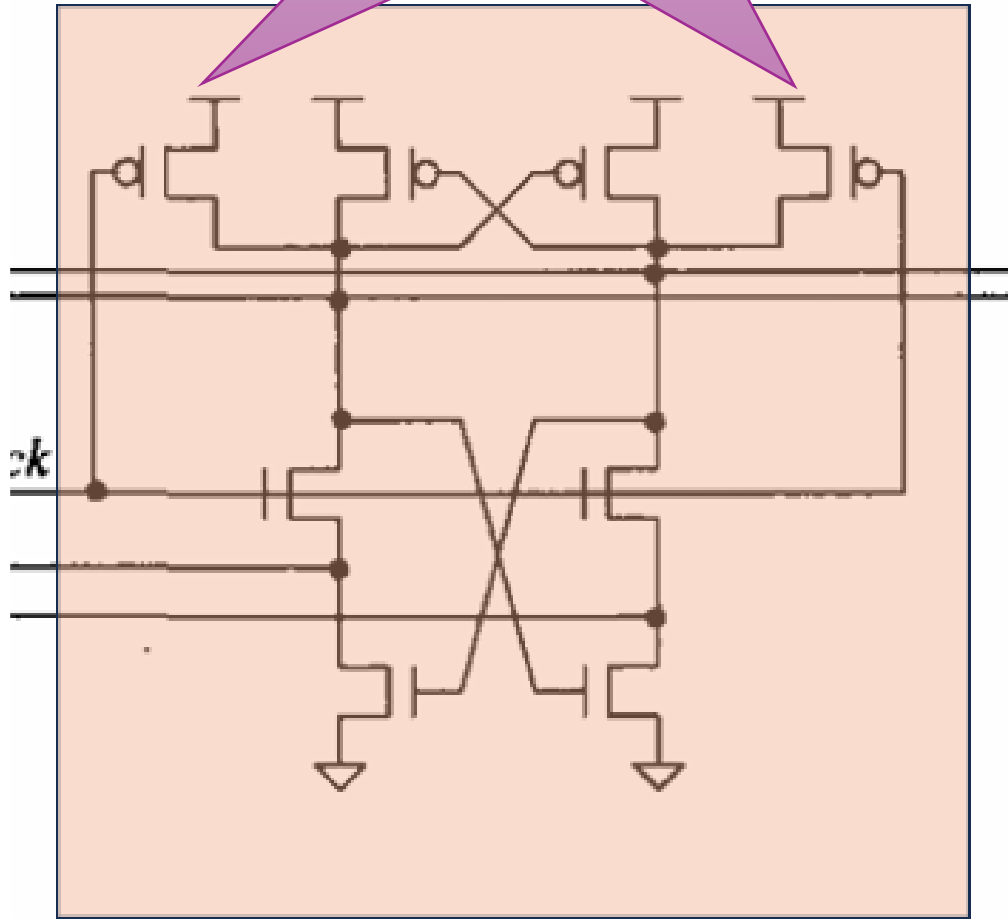
クロス結合インバータ部

ラッチ部： クロス結合部のWの設計

- 全体
 - Wを大きくすると g_m が増加し、動作速度は向上する。しかし、Wを大きくしすぎると内部ノード容量、消費電力、kickback noiseが増加する。
 - 左右のバランスが崩れると、offsetや判定非対称性の原因になる。
- 下側NMOS
 - 小さすぎると、放電速度が遅くなり、ラッチ再生が始まるまでに時間がかかる。
- 設計
 - LongLの0.5~0.8倍サイズがあれば十分だと考えらえる。
 - Lのサイズが違うことに注意
 - 今回は、 $W_n = 20\mu m, W_p = 60\mu m$ とする
 - 電子の移動速度の違いより、 $W_p:W_n \approx 3:1$



CLKにつながる
2つのPMOSが
プリチャージ部

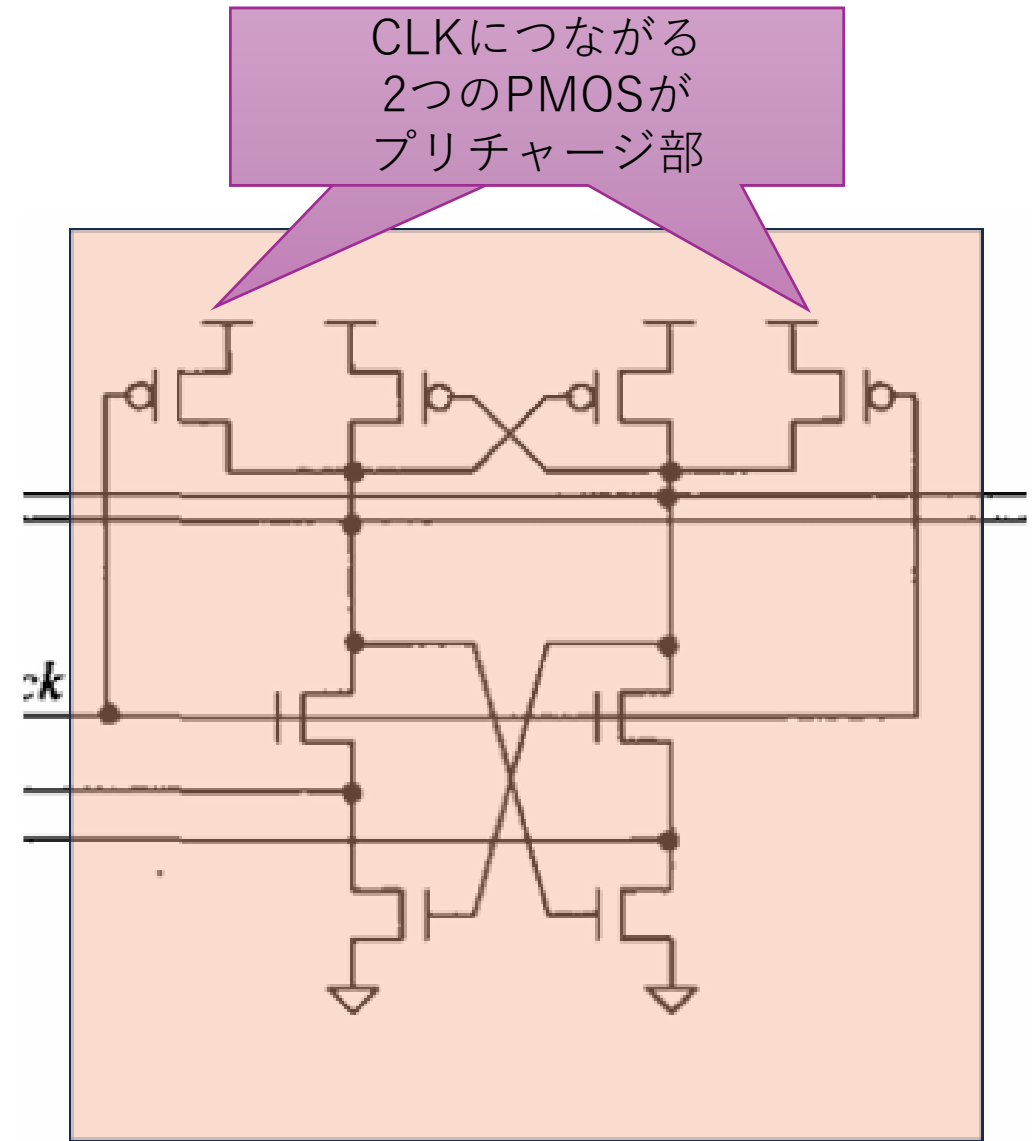


ラッチ部： プリチャージ部の設計

- クロックドコンパレータのプリチャージと同じ
 - プリチャージが不十分だと、前回の比較結果が内部ノードに残り、次の判定に影響する。その結果、オフセット増加、判定誤り、メタスタビリティ、左右非対称動作の原因となる
 - プリチャージPMOSを大きくしすぎると、内部ノード容量が増加し、評価時の放電速度が低下する。また、クロックに同期した電荷注入や kickback noiseが増える可能性がある
- したがって、プリチャージPMOSは、評価開始前に内部ノードを十分にVDDへ戻せる範囲で、必要最小限のサイズにする

ラッチ部： プリチャージ部のWの設計

- 全体をチャージできる最小サイズのWでよいため、
倍のサイズの $W_p = 120\mu\text{m}$ とした。
 - 正確に求めるならすべての消費電流を計算して求めるべきであるが、さほどのズレは無いと考えてこの値とする





Rail-to-Rail OPAMP

カスコード接続

- 構成
 - 2つのトランジスタを直列に接続し、単一のトランジスタでは実現が難しい優れた性能を得るための回路
- 原理
 - M2がM1に対して「**電圧スタビライザ**」および「**電流ブースター**」として機能
 - M1 (下段、コモンソース)
 - 役割: 入力電圧(V_{in})を電流変化に変換する。基本的な増幅動作の起点。
 - M2 (上段、コモンゲート)
 - ゲート: 固定バイアス電圧(V_b) - M2の動作点を定める
 - 役割: M1のドレイン電圧を安定させる

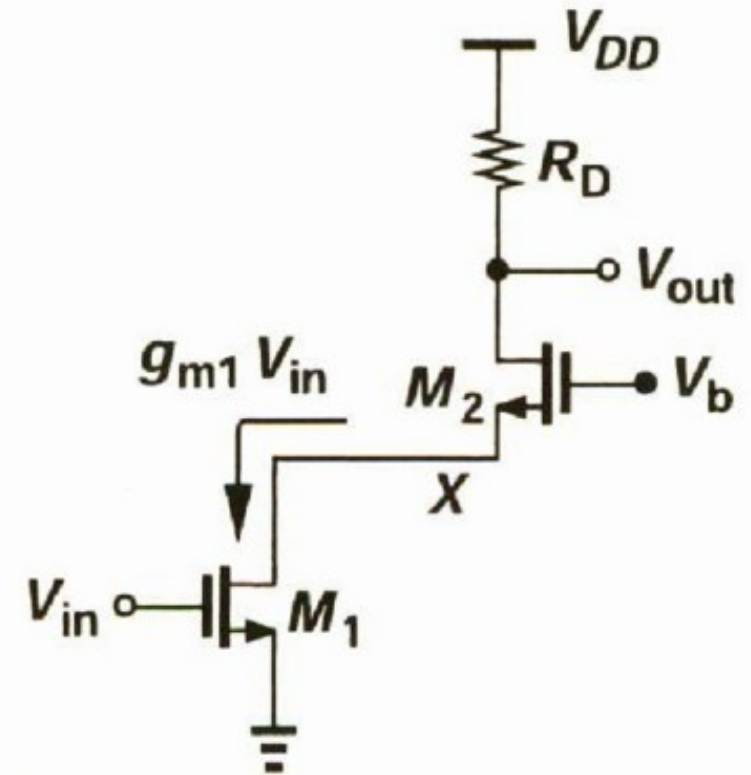
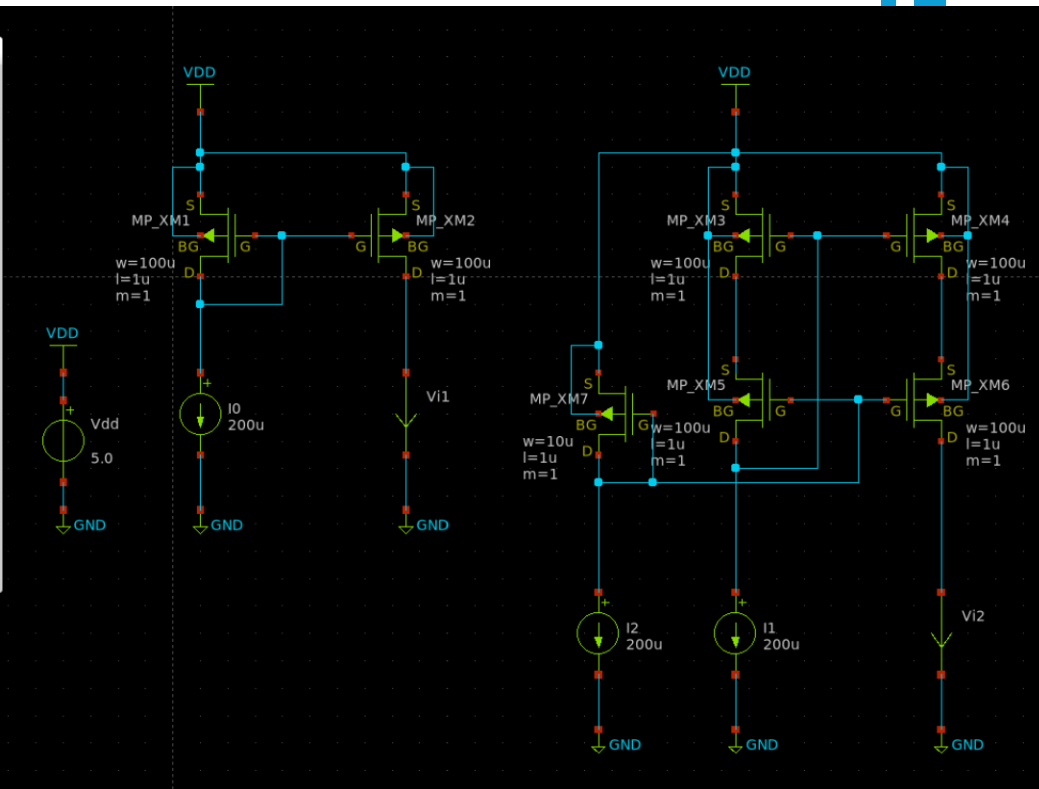
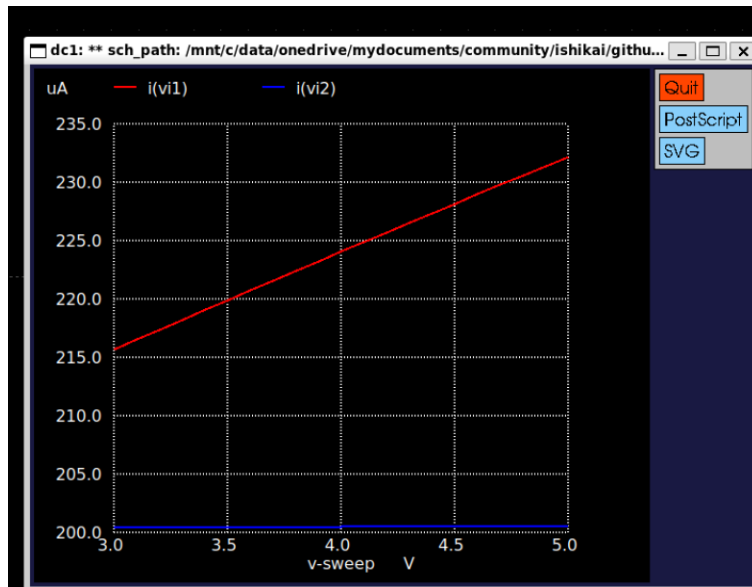


図 3.50 カスコード増幅段.

利点：短チャネル効果の改善

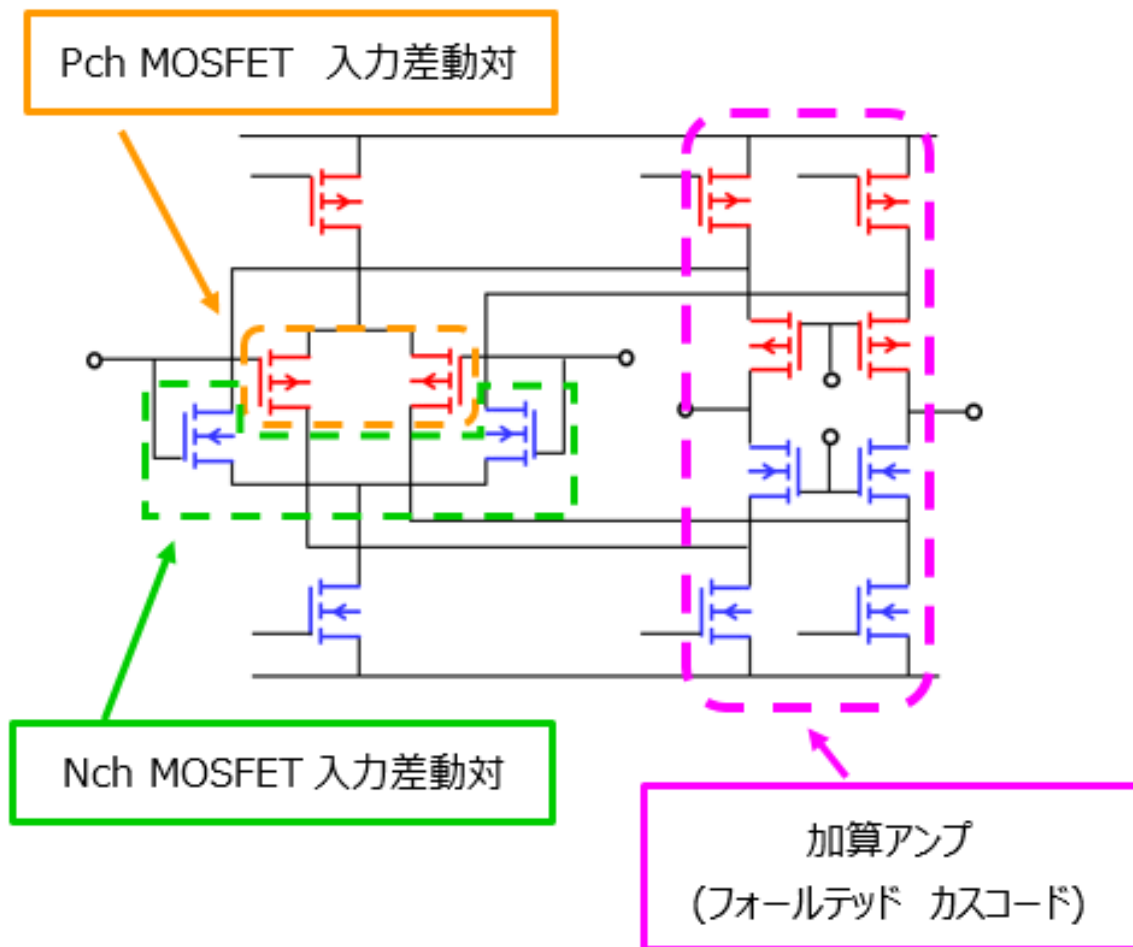
- 原理

- 下段トランジスタのドレイン電圧がほぼ一定に保たれることを利用
 - cascode.sch



応用：Rail-to-Rail

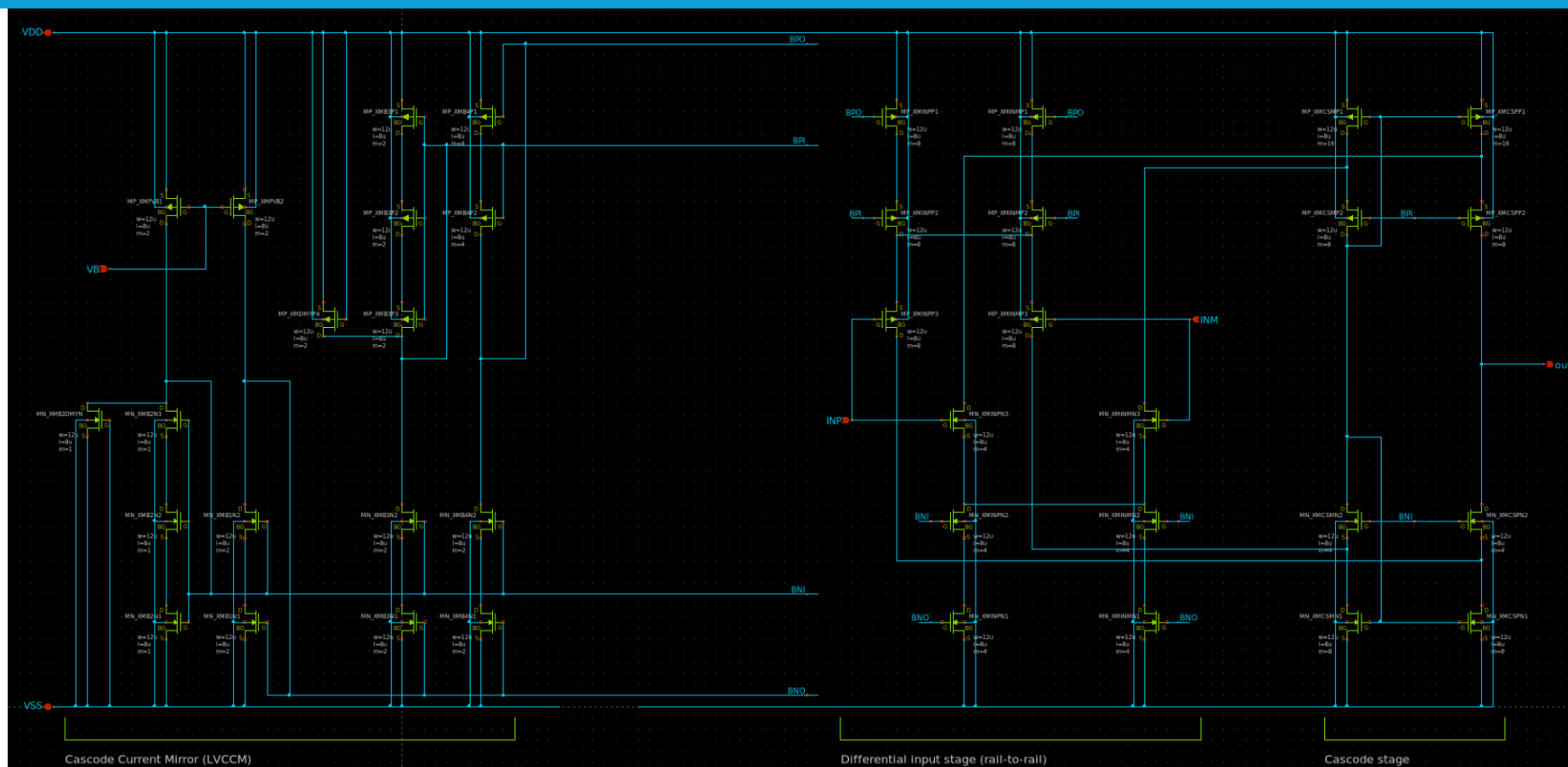
- 構成
 - コンプリメンタリ差動対
 - フォールデッドカスコード
 - プッシュプル構成
- 内容
 - VDD-VSSまでフルレンジで出力させる
- 構造
 - 差動対をpMOS,nMOS構造にする
 - カレントミラーをカスコード接続する





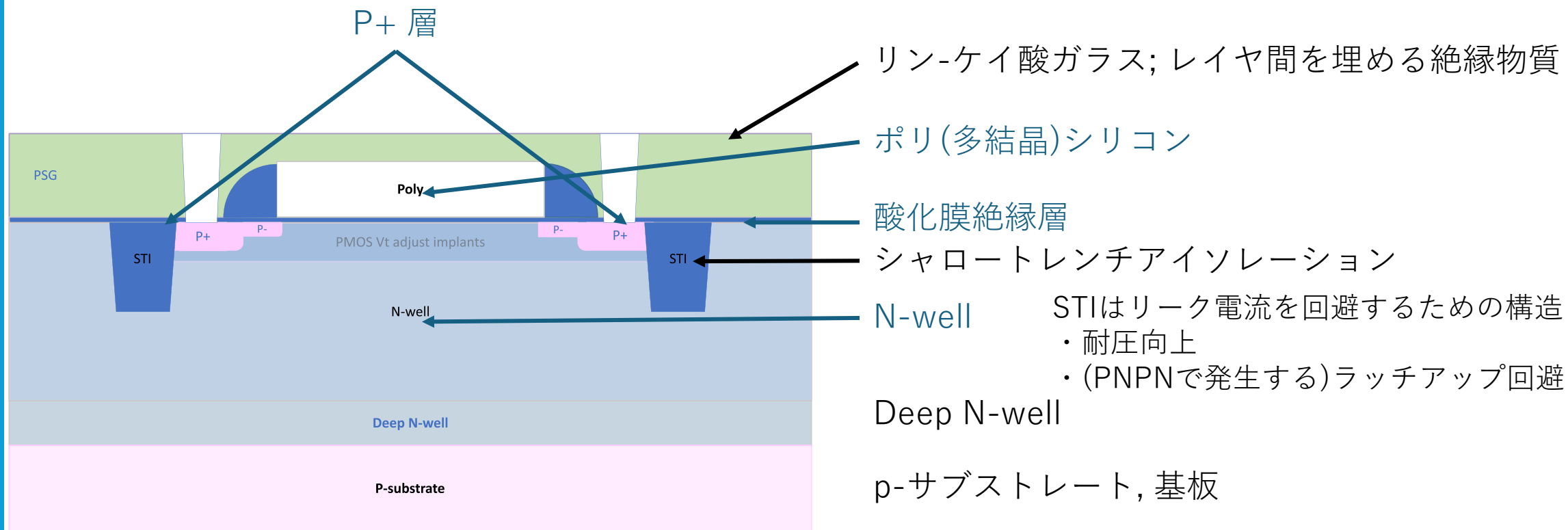
Rail-to-Rail OPAMP型コンパレータ

Rail-to-Rail OPAMP型コンパレータ

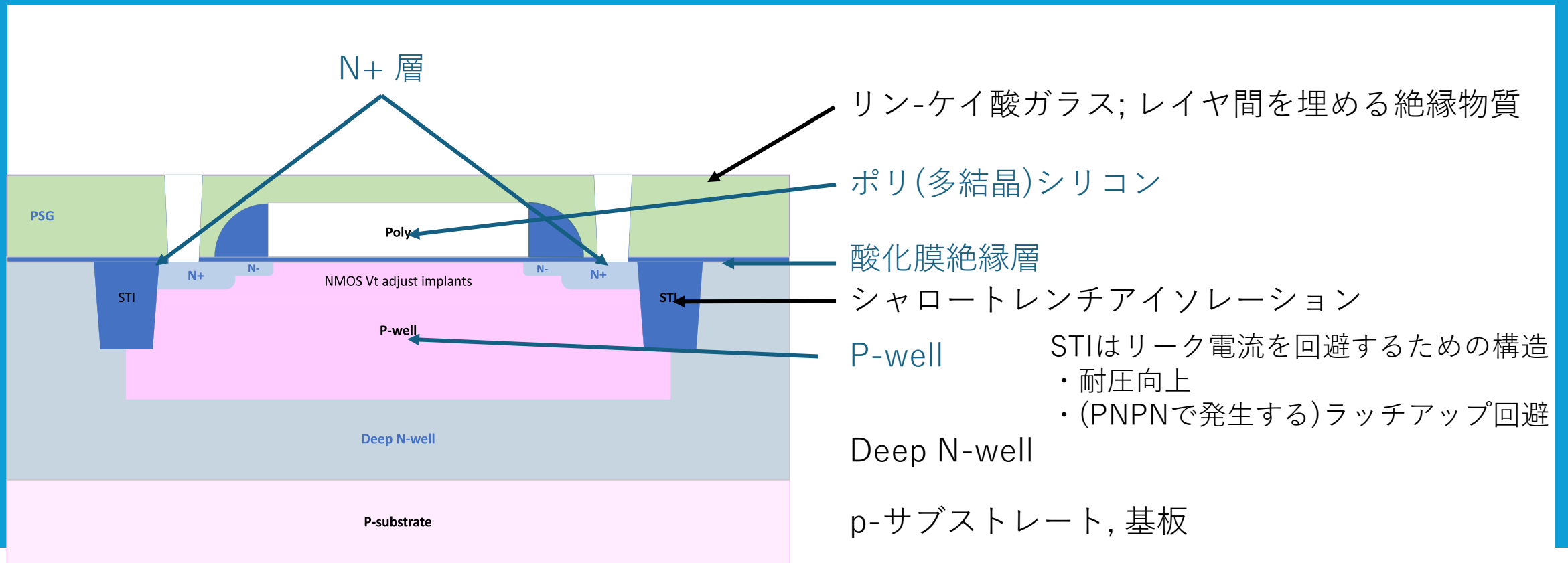


基板バイアス効果 (body effect)

- ボディにかかるバイアス電圧によってトランジスタの動作が変化する
 - スレッシュホルド電圧 (V_{th}) が変化
 - ゲート電圧を固定して比較した時、ドレイン・ソース間電流(I_{ds})が変化



プレーナ型pMOSトランジスタ (トリプルウェル)



プレーナ型nMOSトランジスタ（トリプルウェル）

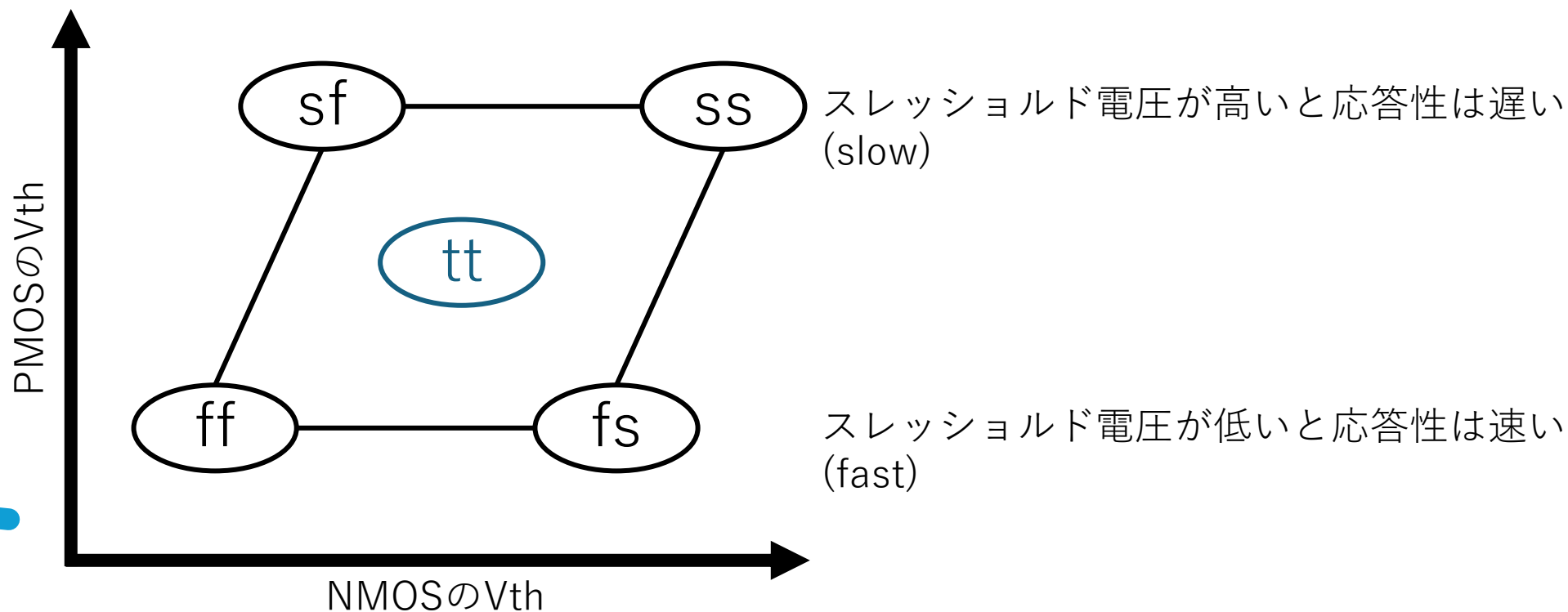


シミュレーション編



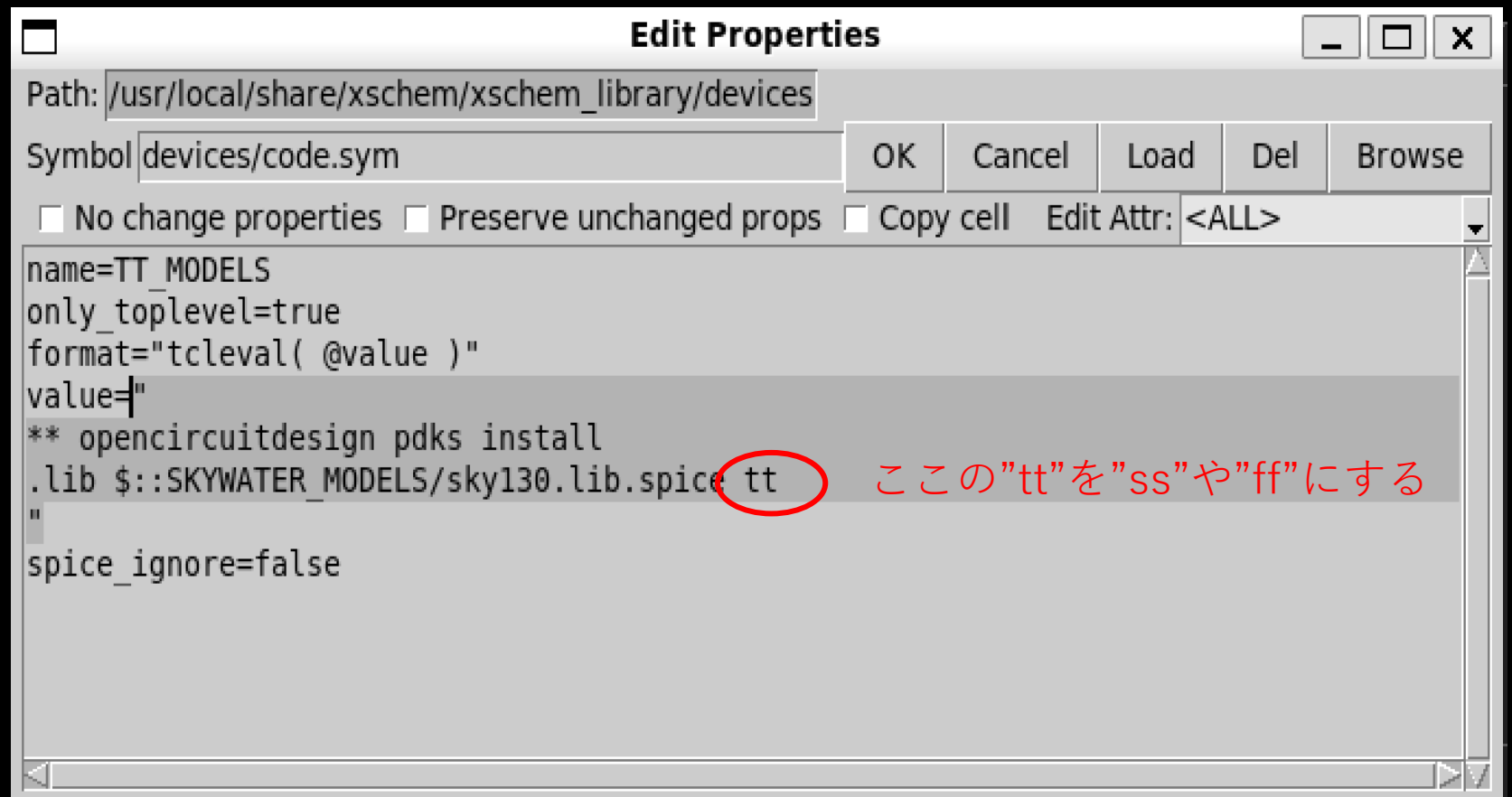
コーナーモデルシミュレーションについて

- ssモデル – 応答性が最悪(slow-slow)の時のモデル
- ttモデル – 応答性がtypical-typicalなモデル
 - https://github.com/3zki/lsi1_analog1/



モデル変更

MODELS



最低限必要なシミュレーション

温度=temp

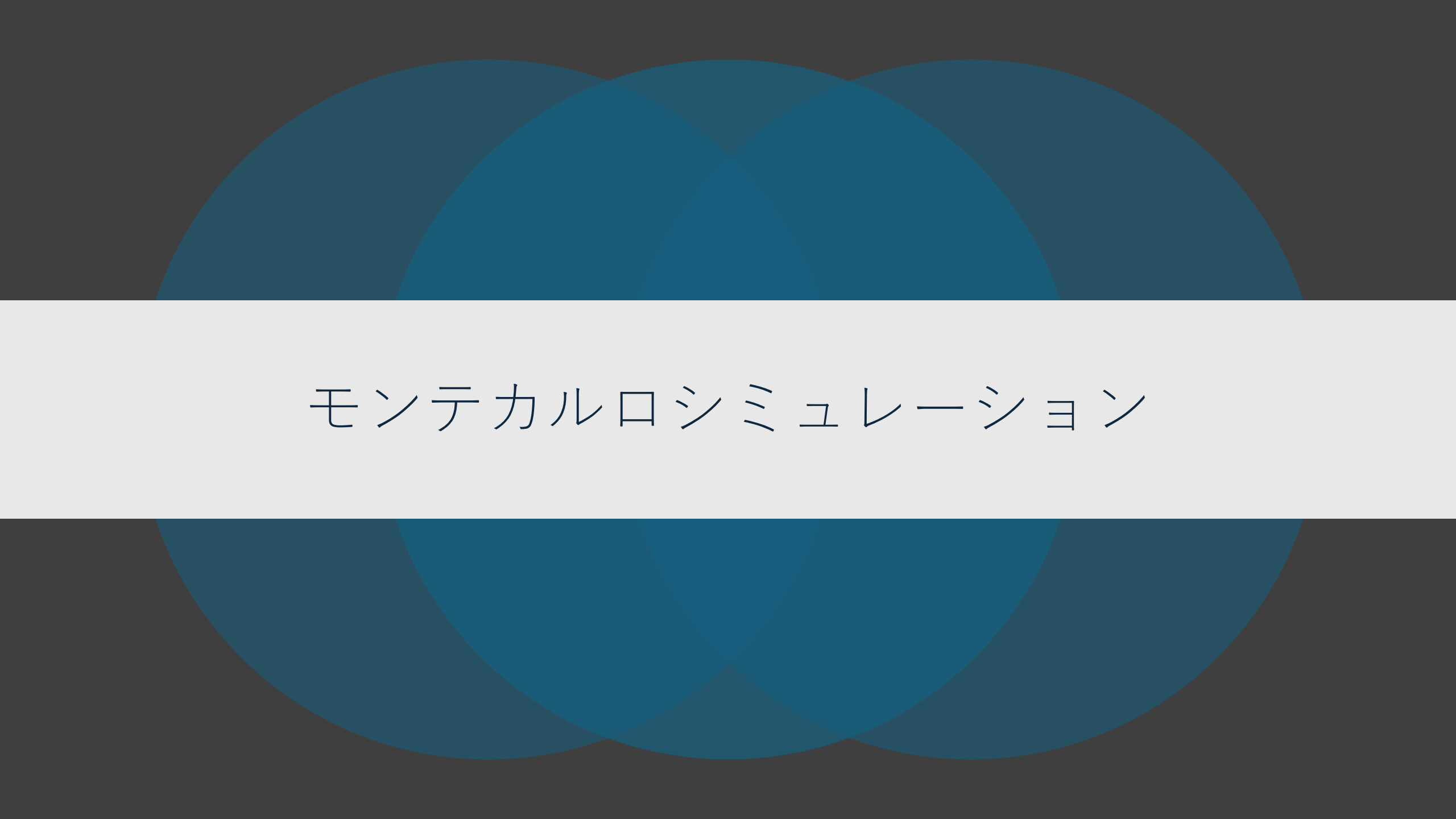
- 27度

NGSPICE

```
.temp 27  
.control  
save all  
tran 1n 30n  
plot V(Vout) V(CLK)  
write trans_gate_tb.raw  
.endc
```

コーナーモデル

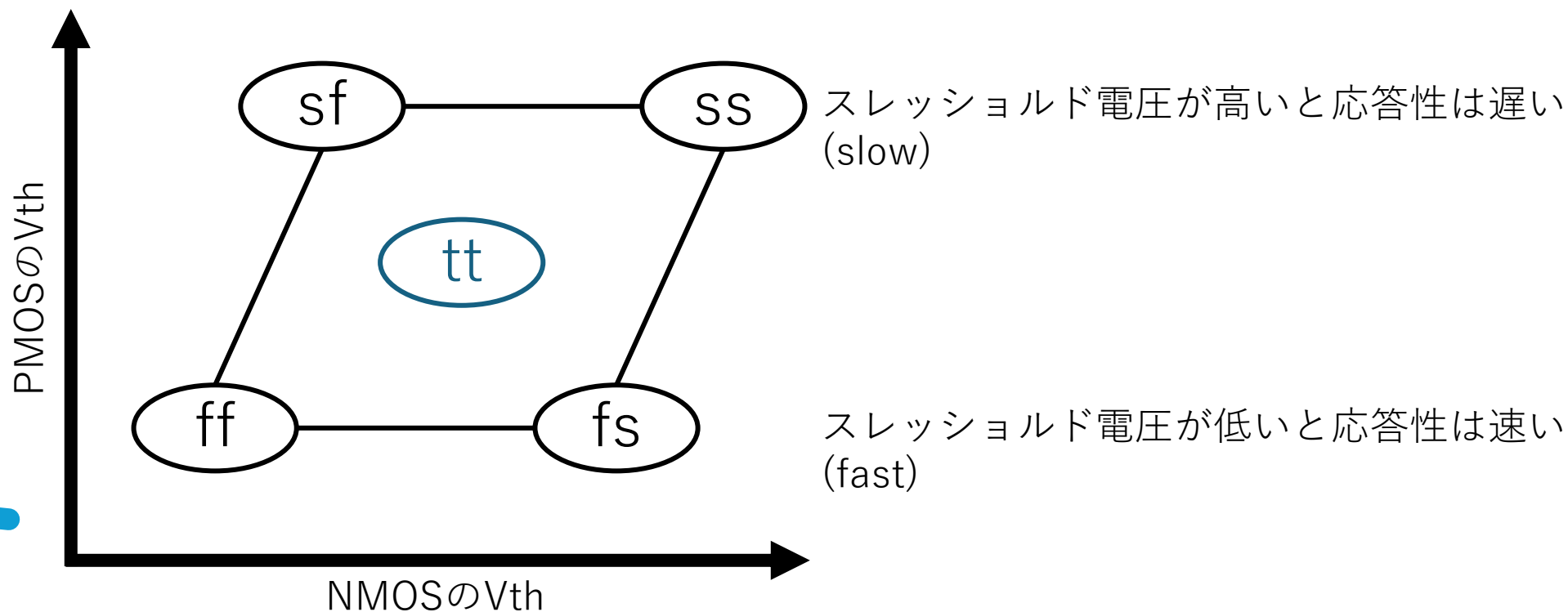
- ff, tt, ss



モンテカルロシミュレーション

モンテカルロシミュレーション

- シミュレーションを実行する際、どのコーナのモデルを使用するか指定する必要アリ
- モンテカルロシミュレーションではランダムなコーナでシミュレーションを複数回行う
 - シミュレーションモデルは“mm” を利用する
 - https://github.com/3zki/lsl1_analog1/

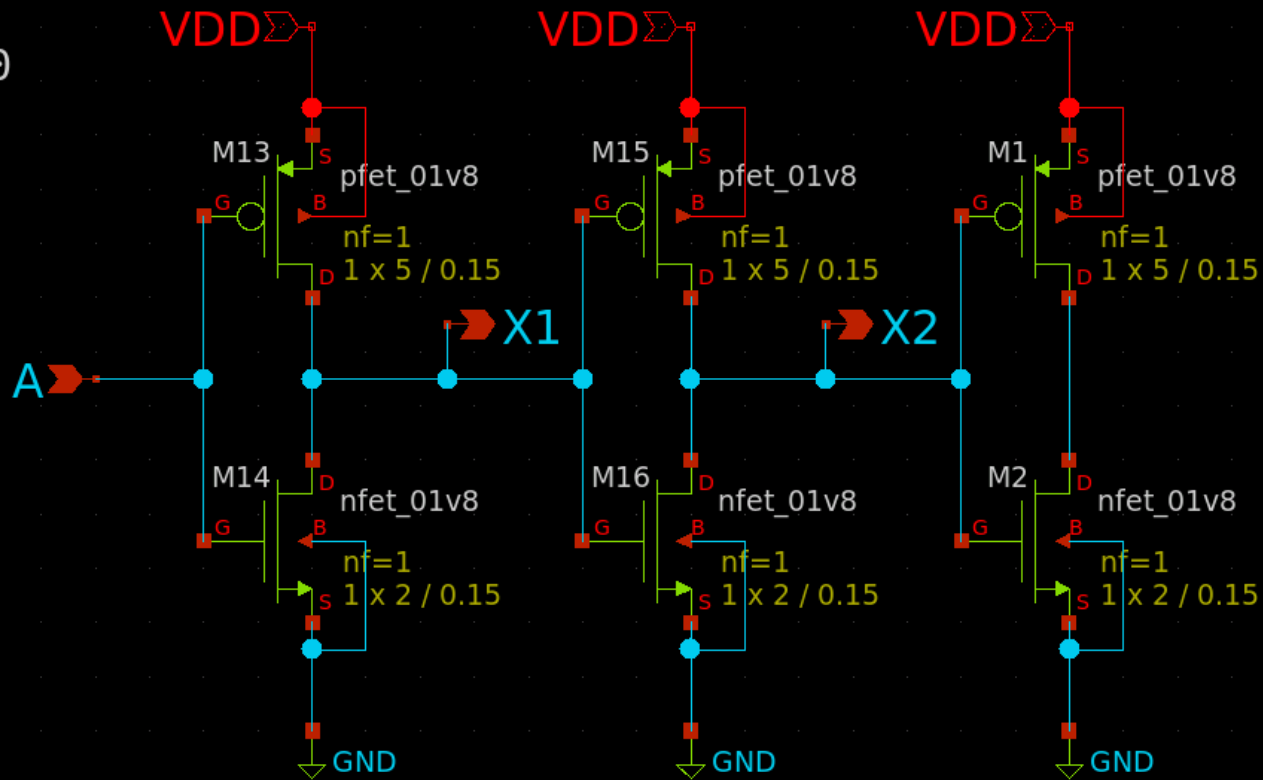
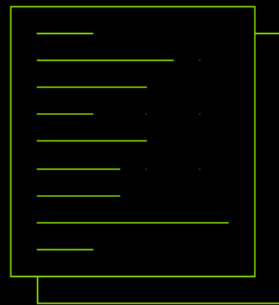


Xschem 構成例 10回 ($L=0.15\ \mu\text{m}$)

ngspice

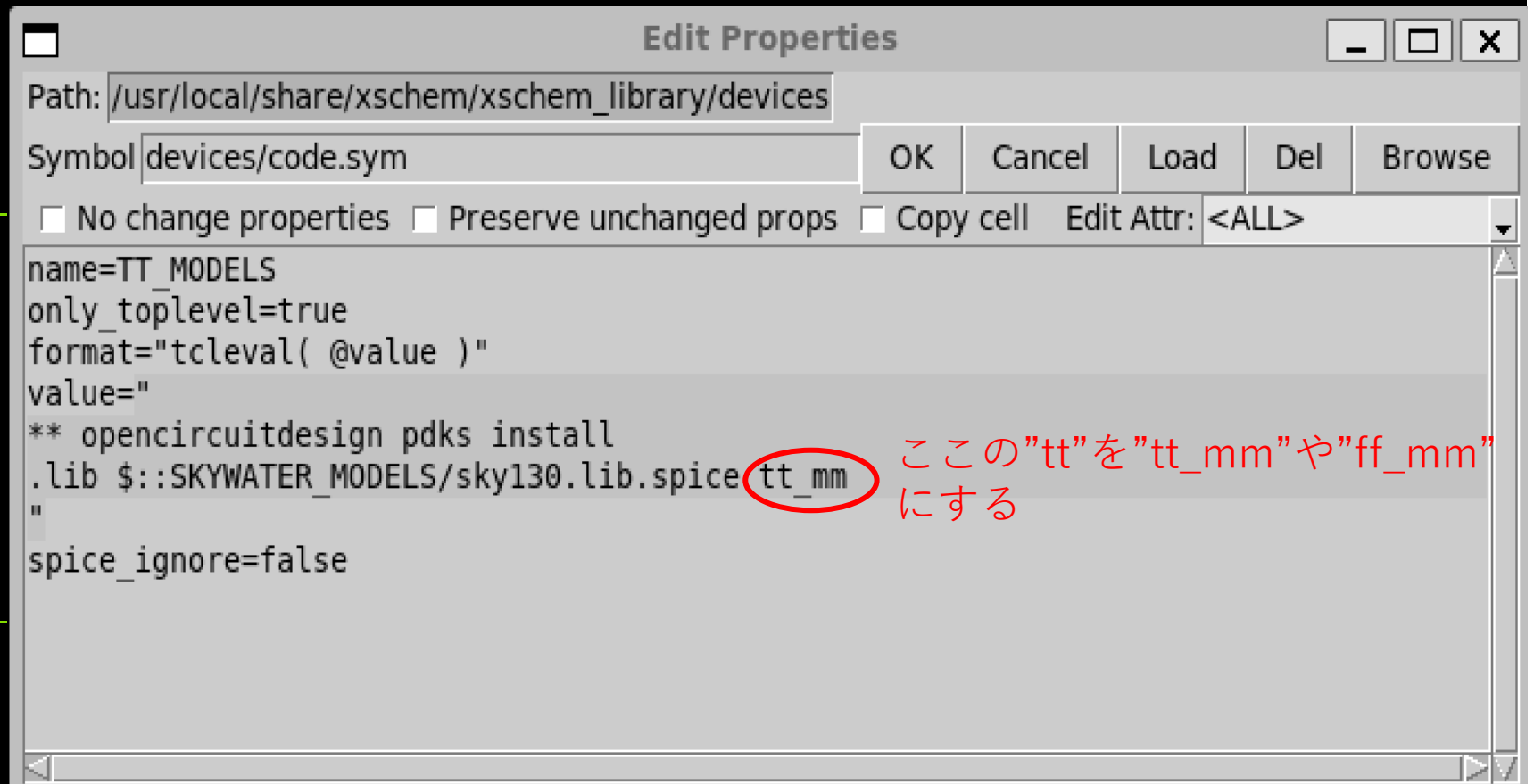
```
VA A 0 pulse(0 1.8 0 2p 2p 0.5n 1n) dc 0
VD VDD 0 dc 1.8
.control
set wr_vecnames
set wr_singlescale
let count=0
dowhile count < 10
  tran 1p 2n
  write ~/mc_{$&count}.raw
  reset
  let count = count + 1
end
.endc
```

MODELS



モデル変更

MODELS



Edit Properties

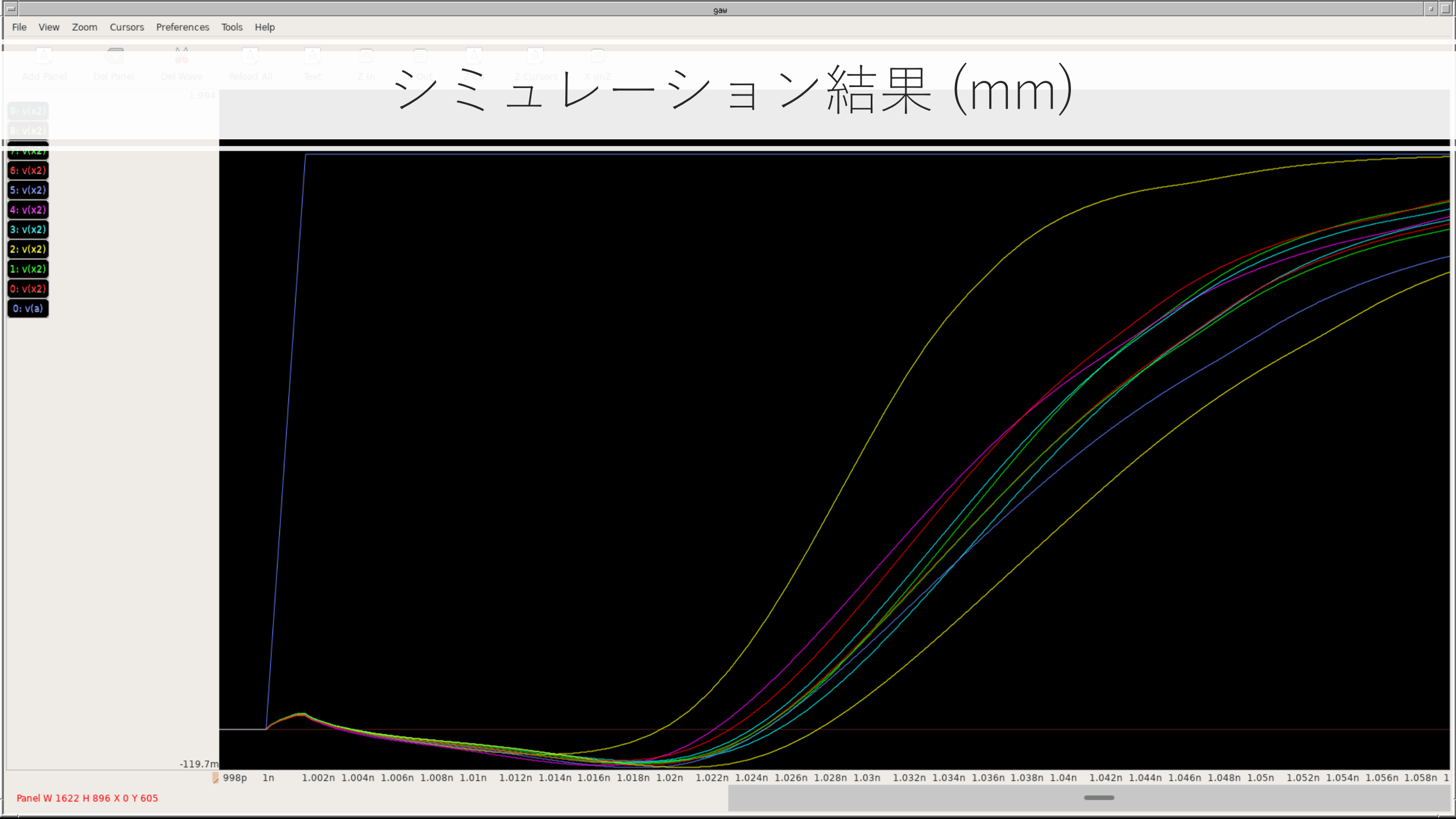
Path: /usr/local/share/xschem/xschem_library/devices

Symbol: devices/code.sym

☐ No change properties ☐ Preserve unchanged props ☐ Copy cell Edit Attr: <ALL>

name=TT_MODELS
only_toplevel=true
format="tclevel(@value)"
value="
** opencircuitdesign pdks install
.lib \$::SKYWATER_MODELS/sky130.lib.spice tt_mm
"
spice_ignore=false

この"tt"を"tt_mm"や"ff_mm"にする





SAR-ADCのシミュレーション



最低限必要なシミュレーション

コンパレータ

- 全ビットに対応する電圧でのオフセットとノイズ
- スルーレート

トランスミッションゲート

- スルーレート

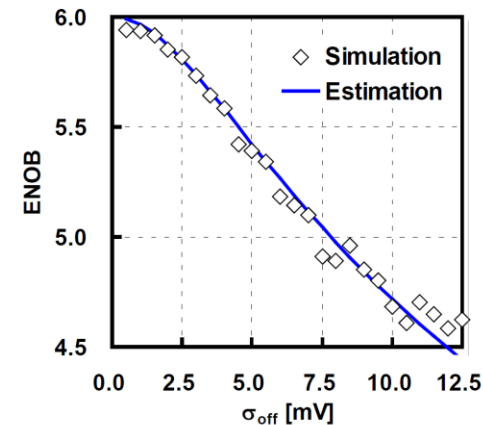
CDAC

- 雑音
- 許容誤差
- 電力効率は考慮しない

オフセットとノイズ

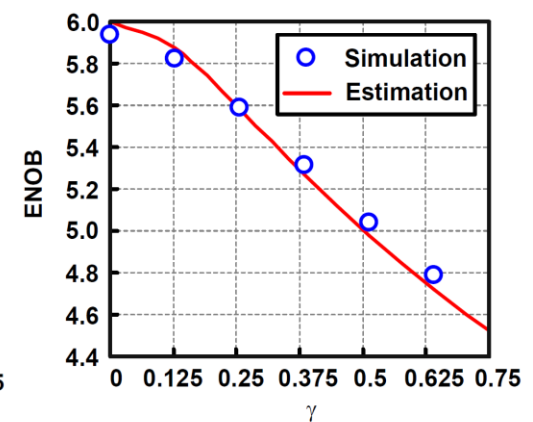
- 各ビットに対応する電圧に対して何VのズレでOnになるかチェックする
 - $3.3\text{V}/6\text{bit} = 0.0515625\text{V}$ ごと
 - 後述のAIの章を参照
 - 目標： $N = 6\text{bit}$ で $\text{ENOB} = 5.5$ を目指す場合、
 - $\alpha \doteq 0.25$ となり、 $V_q = 0.0515625\text{V}$ なので、 $V_{\text{off}} \doteq 0.0129\text{V}$
 - 対策：W/Lサイズを大きくする
- PEX後もやること

$$\text{ENOB} = N - \frac{1}{2} \log_2(1 + 12\alpha^2) \quad \alpha = \frac{V_{\text{offset}}(\sigma)}{V_q}$$



オフセット電圧による性能劣化

$$\text{ENOB} = N - \frac{1}{2} \log_2(1 + 12\gamma^2) \quad \gamma = \frac{v_n(\sigma)}{V_q}$$



ノイズによる性能劣化

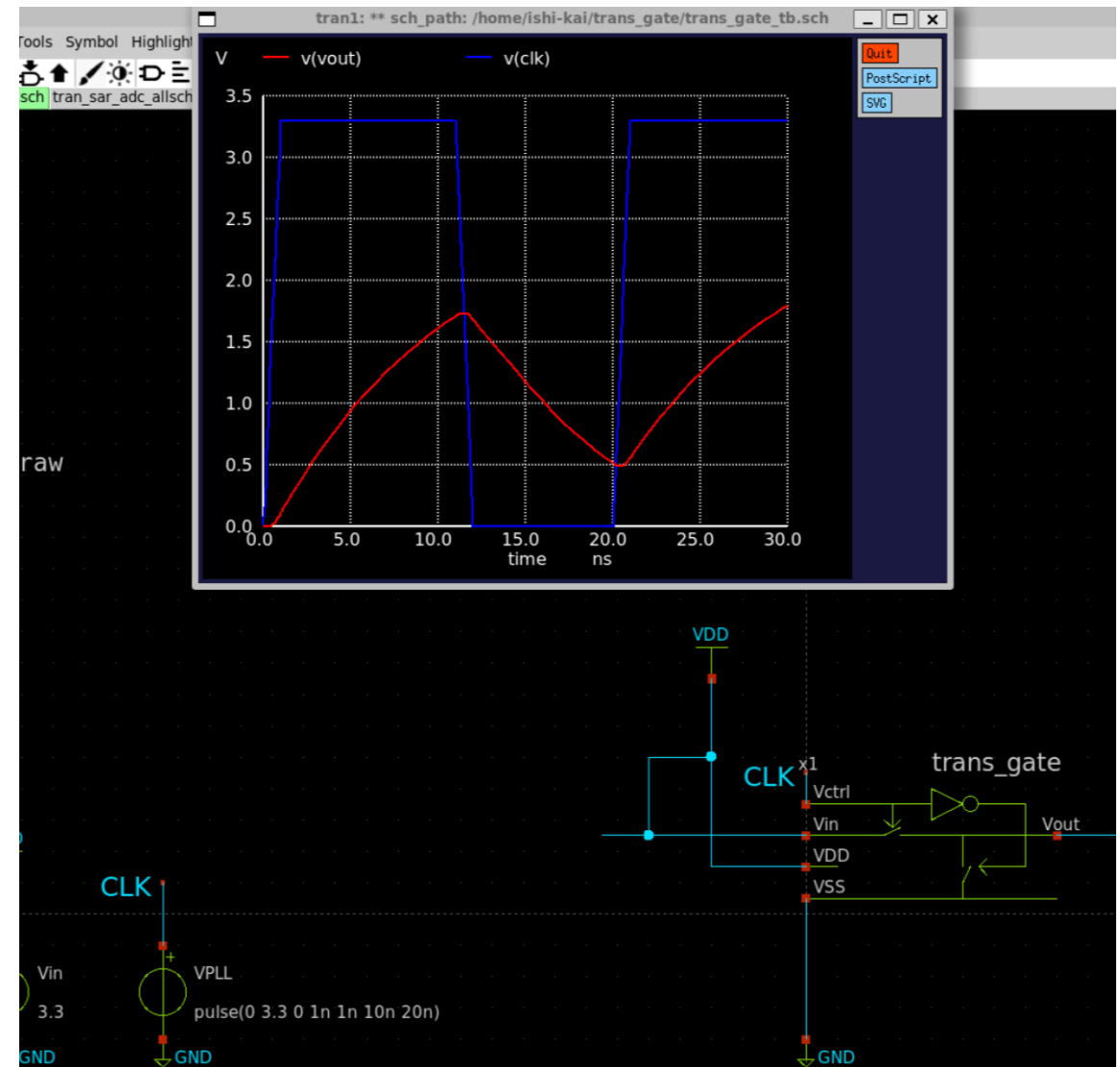


スルーレート (OPAMPより)

- 出力電圧が単位時間あたりに変化できる最大速度
 - テール電流の大きさに影響される
 - オーディオ用であるのであれば、可聴周波数の20kHzでナイキスト周波数を考えて2倍の40kHzあれば最低限の要求は満たせるので、それほど気にしなくとも良い
 - 性能劣化を考慮しても、100kHzを超えていれば十分である
-

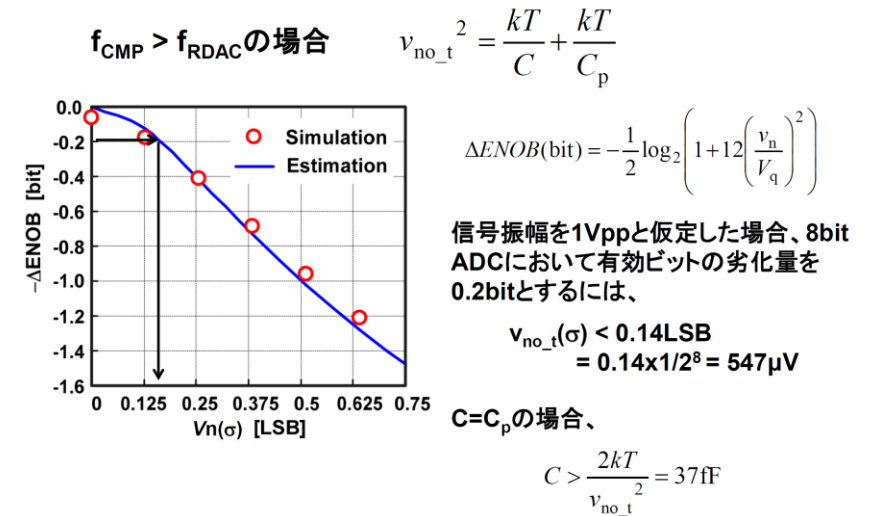
スルーレートの例

- ダメな例
 - 目標値：目標とするクロック
 - PEX後はかなり遅くなるので20~30%は余裕が欲しい
 - 対策：W/Lのサイズを大きくする
- PEX後もやること

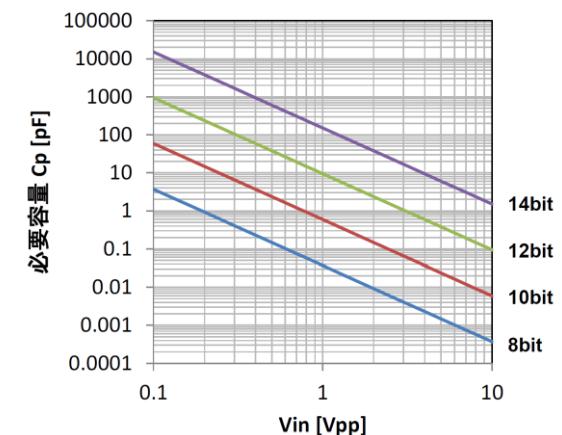


CDACの雑音

- 計算で必要容量が求められる
 - 6bitなので、100fFが安全圏となる
 - 対策：容量を大きくする



ADCの分解能が2bit上がると必要な容量値は16倍になる。
信号振幅を2倍にすると容量値は1/4に減じられる。



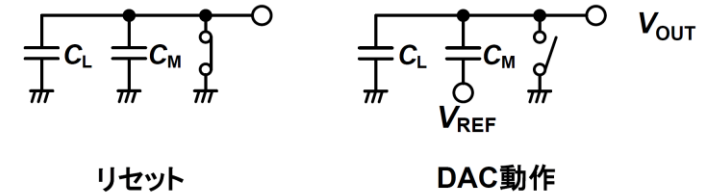
CDACの許容誤差

- 計算で許容誤差が求められる

- 6bitで100fFの場合

- 1σ : $2.08\% = 2.08\text{fF}$
- 3σ : $6.24\% = 6.24\text{fF}$

MSBの変換の場合を考える



出力電圧 $V_{\text{OUT}} = \frac{C_M}{C_M + C_L} V_{\text{REF}}$

ばらつき感度
$$\Delta V_{\text{OUT}} = \frac{\partial V_{\text{OUT}}}{\partial C_M} \Delta C_M + \frac{\partial V_{\text{OUT}}}{\partial C_L} \Delta C_L = \frac{C_L \Delta C_M - C_M \Delta C_L}{(C_M + C_L)^2} V_{\text{REF}}$$
$$= \frac{(\Delta C_M - \Delta C_L)}{4C_S} V_{\text{REF}} \quad \because C_M = C_L = C_S$$

DNL < 1/4LSBを達成するためには、

$$\frac{(\Delta C_M - \Delta C_L)}{4C_S} V_{\text{REF}} < \frac{V_{\text{REF}}}{2^{N+2}} \quad \text{より、} \quad \frac{\Delta C_M - \Delta C_L}{C_S} < \frac{1}{2^N}$$

ΔC_M と ΔC_L が無相関なばらつきを持っている場合、

$$\frac{\Delta C}{C_S} < \frac{1}{2^{N+0.5}}$$

3 σ の範囲で1/4LSBを達成するためには

$$\frac{\Delta C}{C_S}(\sigma) < \frac{1}{3} \frac{1}{2^{N+0.5}}$$

```

COMMANDS
SM=ngspice
include ~/Chipathon2023_ADC/maple0705/SAR_Logic/layout/user_proj_sarlogic_pex_extracted.spice
include ~/Chipathon2023_ADC/cdac/min_cap_array_8x8/rev001/TOP_pex_extracted.spice
include ~/Chipathon2023_ADC/latch/TOP_pex_extracted.spice
include ~/Chipathon2023_ADC/gitefu/comp_20240331/TOP_pex_extracted.spice
temp 27
.control
save all
alter V1 3.3

let start_vin = 0.0515625 / 2
let stop_vin = 3.3
let delta_vin = 0.0515625
let vin_act = start_vin
alter V2 vin_act

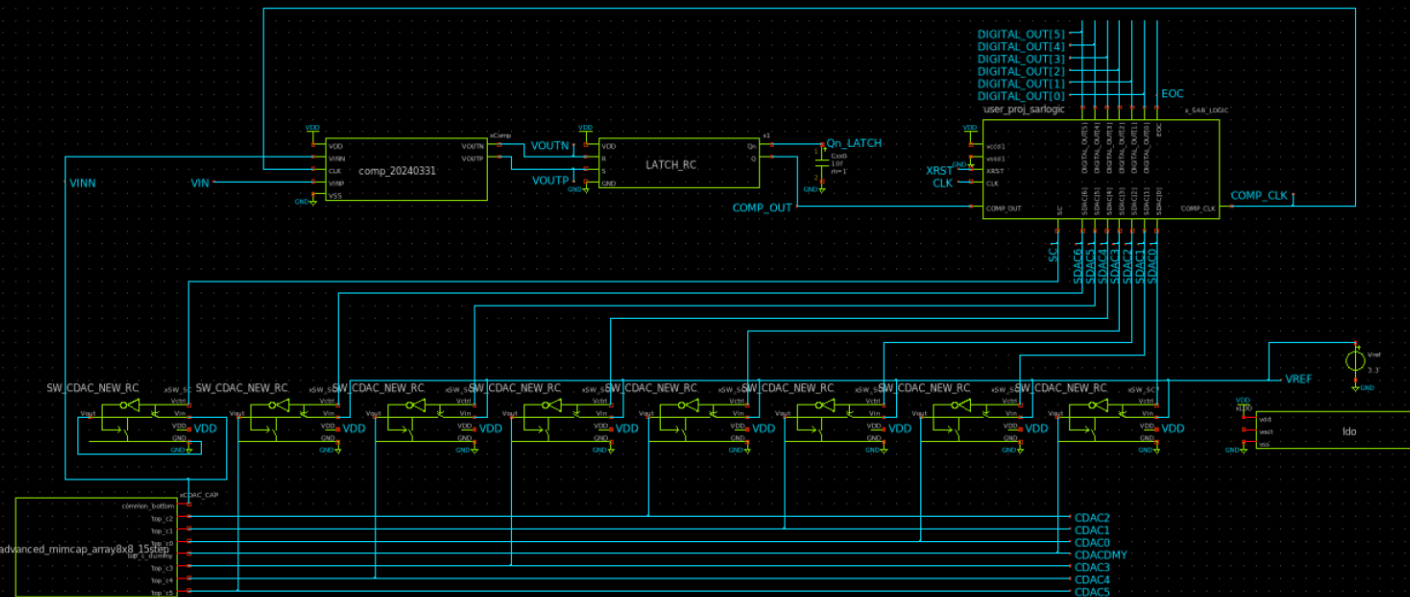
while vin_act le stop_vin
  tran 100n 1u
  write sar_adc_tran.raw
  meas tran vin FIND v(VIN) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout5 FIND v(DIGITAL_OUT[5]) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout4 FIND v(DIGITAL_OUT[4]) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout3 FIND v(DIGITAL_OUT[3]) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout2 FIND v(DIGITAL_OUT[2]) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout1 FIND v(DIGITAL_OUT[1]) WHEN v(EOC)=1.65 FALL=LAST
  meas tran vout0 FIND v(DIGITAL_OUT[0]) WHEN v(EOC)=1.65 FALL=LAST
  print vin >> sar_adc_tran.out.typ.txt
  print vout5 >> sar_adc_tran.out.typ.txt
  print vout4 >> sar_adc_tran.out.typ.txt
  print vout3 >> sar_adc_tran.out.typ.txt
  print vout2 >> sar_adc_tran.out.typ.txt
  print vout1 >> sar_adc_tran.out.typ.txt
  print vout0 >> sar_adc_tran.out.typ.txt
  let vin_act = vin_act + delta_vin
  alter V2 vin_act
end
.endc

```

```

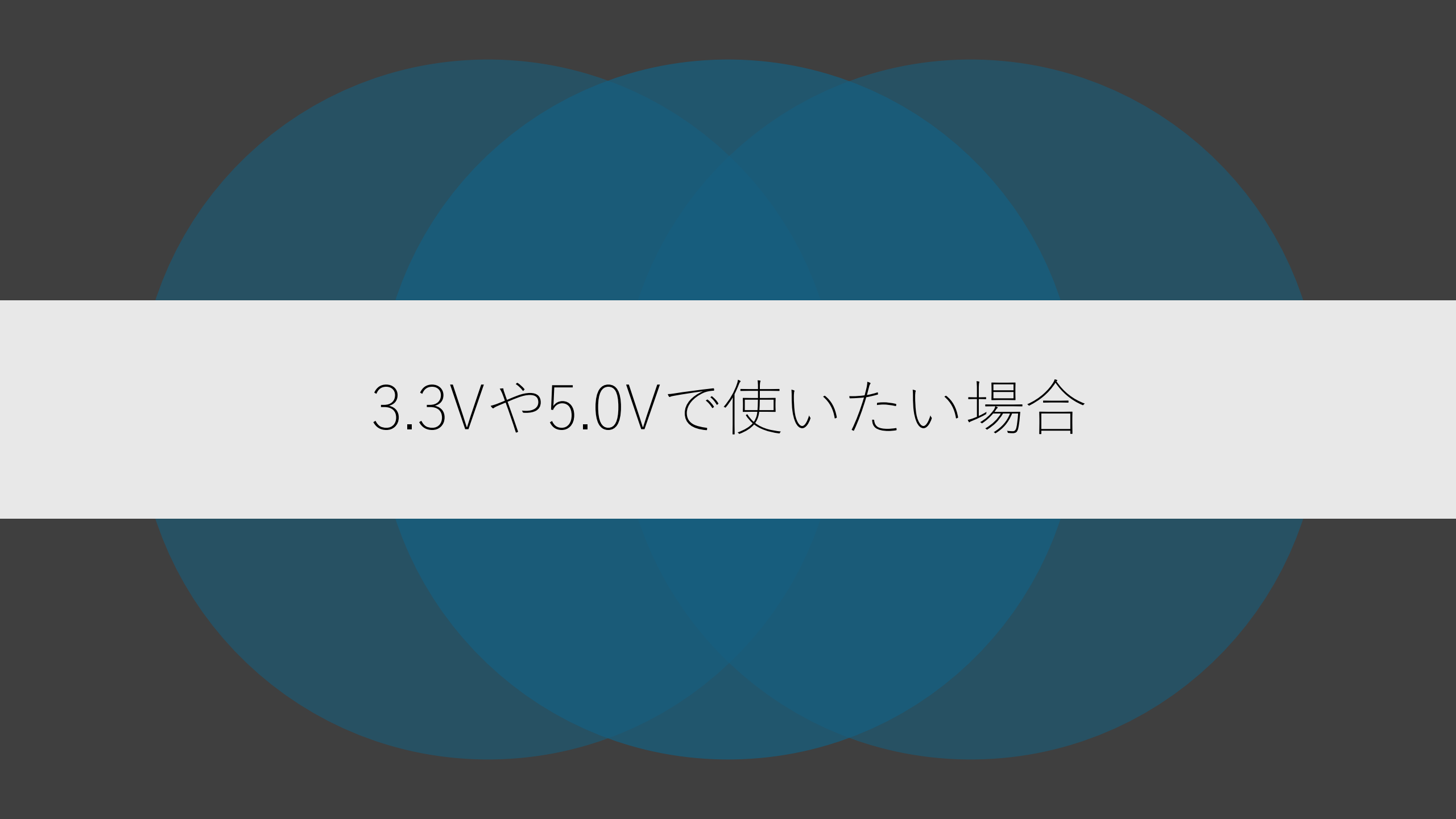
al
include $:/180MCU_MODELS/design.ngspice
lib $180MCU_MODELS/sm141064.ngspice typical
lib $180MCU_MODELS/sm141064.ngspice cap_min
lib $180MCU_MODELS/sm141064.ngspice bjt_typical
lib $180MCU_MODELS/sm141064.ngspice res_typical
lib $180MCU_MODELS/sm141064.ngspice mscap_typical
lib $180MCU_MODELS/sm141064.ngspice mimcap_typical
lib $180MCU_MODELS/sm141064.ngspice diode_typical
include $:/180MCU_STDKELLS/gf180mcu_fd_sc_mcu7t5v0.spice

```



CDACの許容誤差

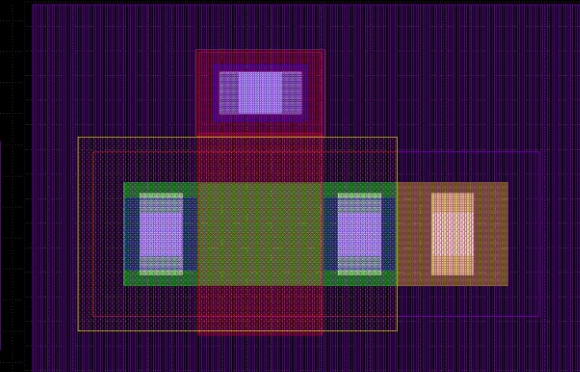
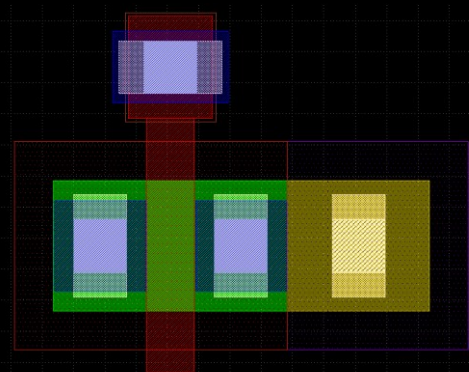
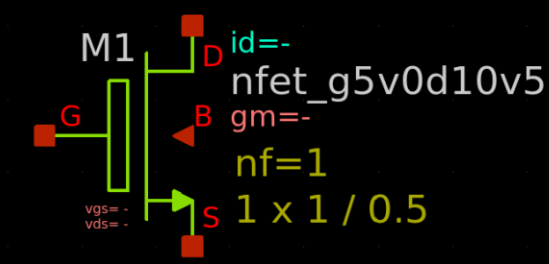
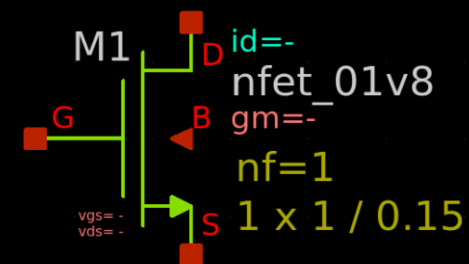
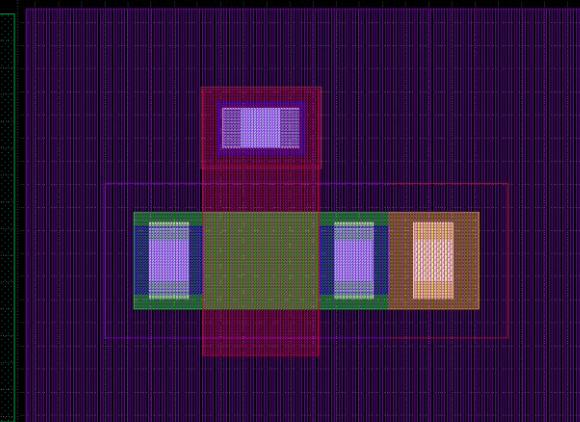
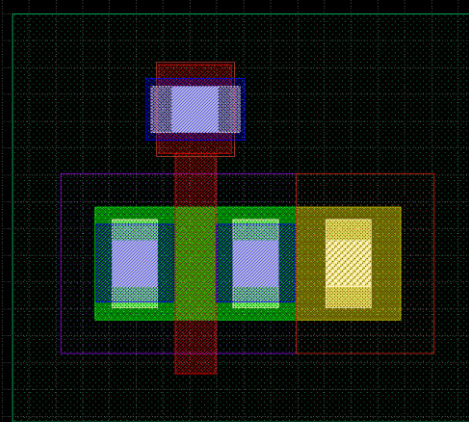
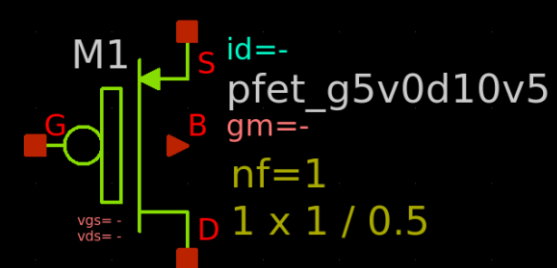
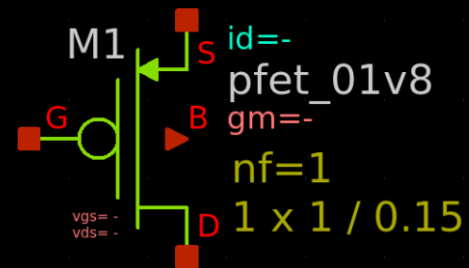
- シミュレーションする場合は、PEXしないとほぼ意味は無い
- そのため、実施する場合はすべてのレイアウト後となる
- 対策：レイアウトに対称性を持たせる



3.3Vや5.0Vで使いたい場合

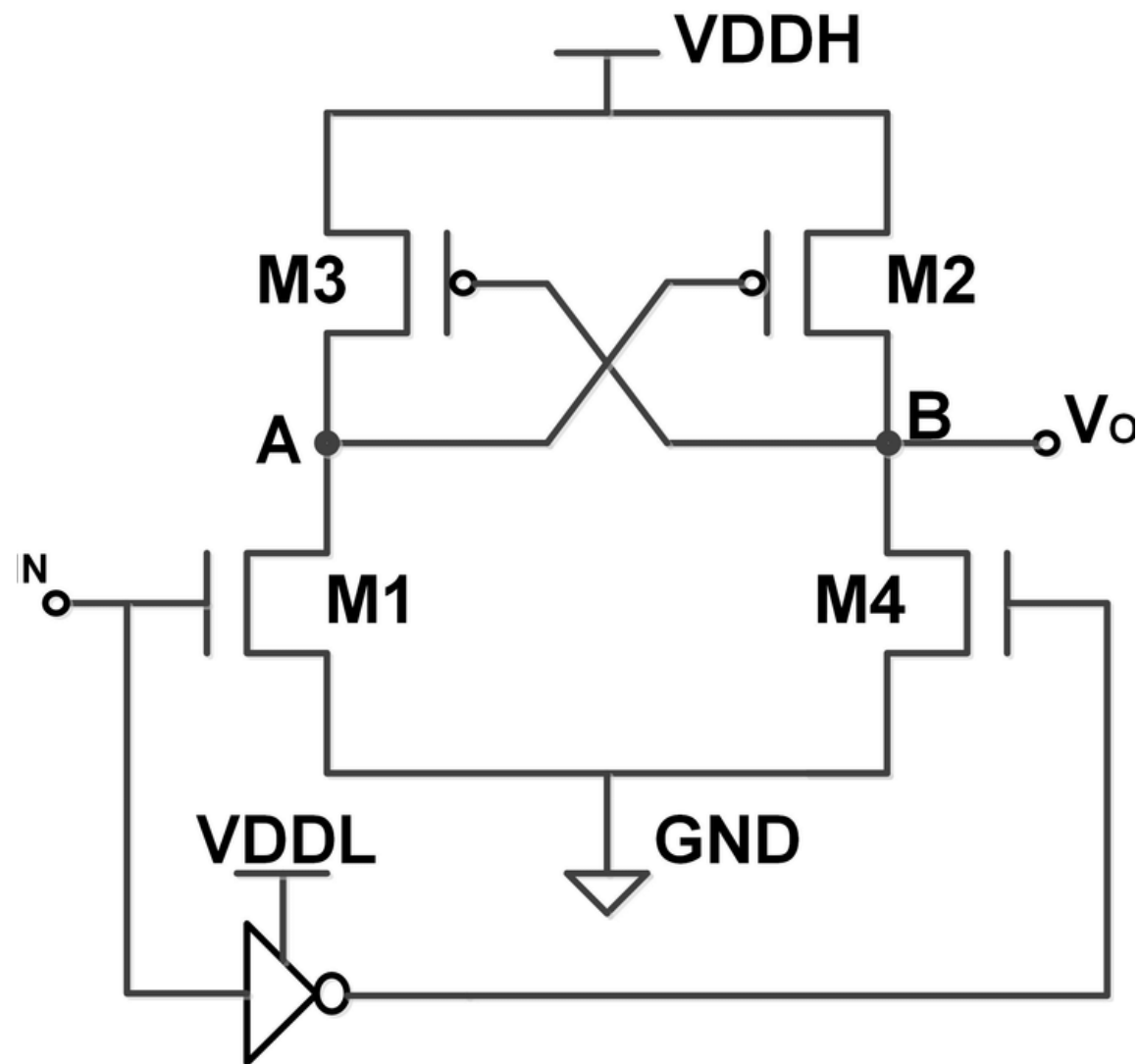
FETの種類： 1.8V低圧FETと5.0V耐圧FET

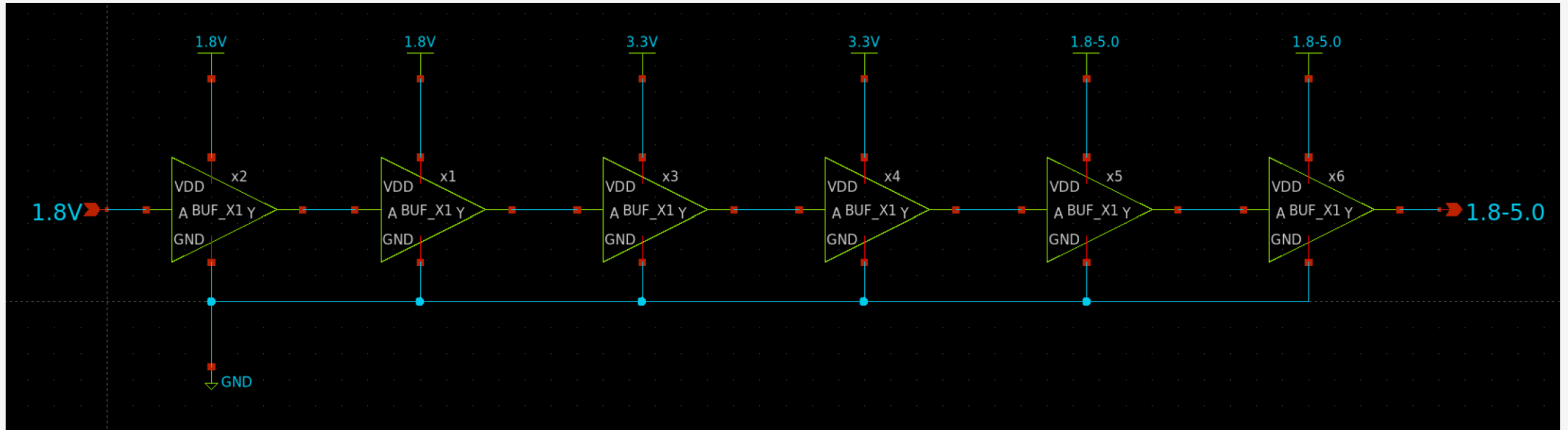
- Sky130nmは耐圧によってFETの種類が違う
 - 下記は1.8Vと5.0VのFETの回路シンボルとPCell



混載回路での注意点

- デジタル（ロジック）回路部分が1.8Vロジックである
 - 3.3Vや5.0Vを利用するにはレベルシフト回路が必要となる
- 回路例
 - 差動型レベルシフタ（Cross-coupled Level Shifter）
 - 可変電圧にする場合、バランスをミスると動かない可能性あり





安全策：インバータ回路レベルシフト

- TinyTapeoutは3.3V電源(= 中間電圧)ピンがある
 - 1.8V = VDPWRピン、3.3V = VAPWRピンから供給する
- Padが使える
 - アナログ電源(1.8-5.0V)はピンから入力する

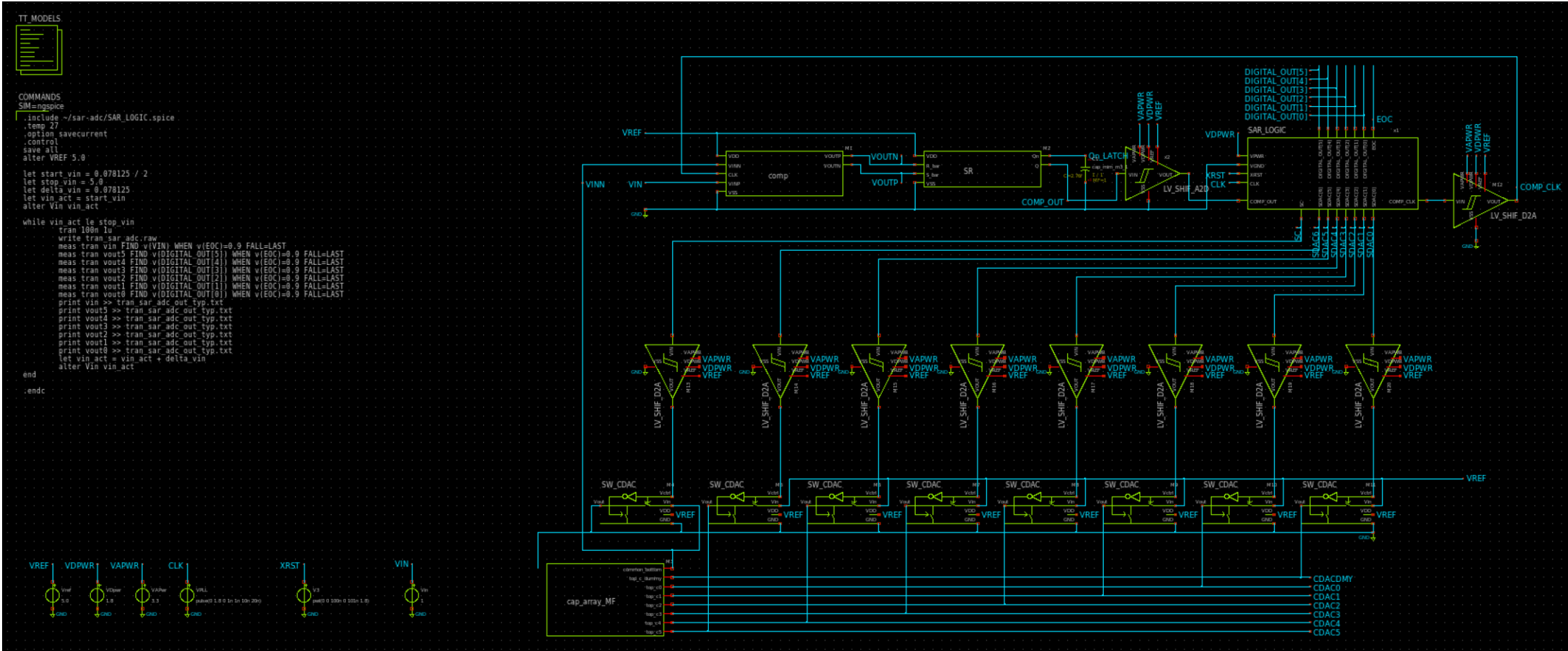
The background features three overlapping teal circles of varying shades, creating a Venn diagram-like effect. A horizontal white band cuts across the middle of the image, containing the title text.

全統合シミュレーション

シミュレーションで確認すること

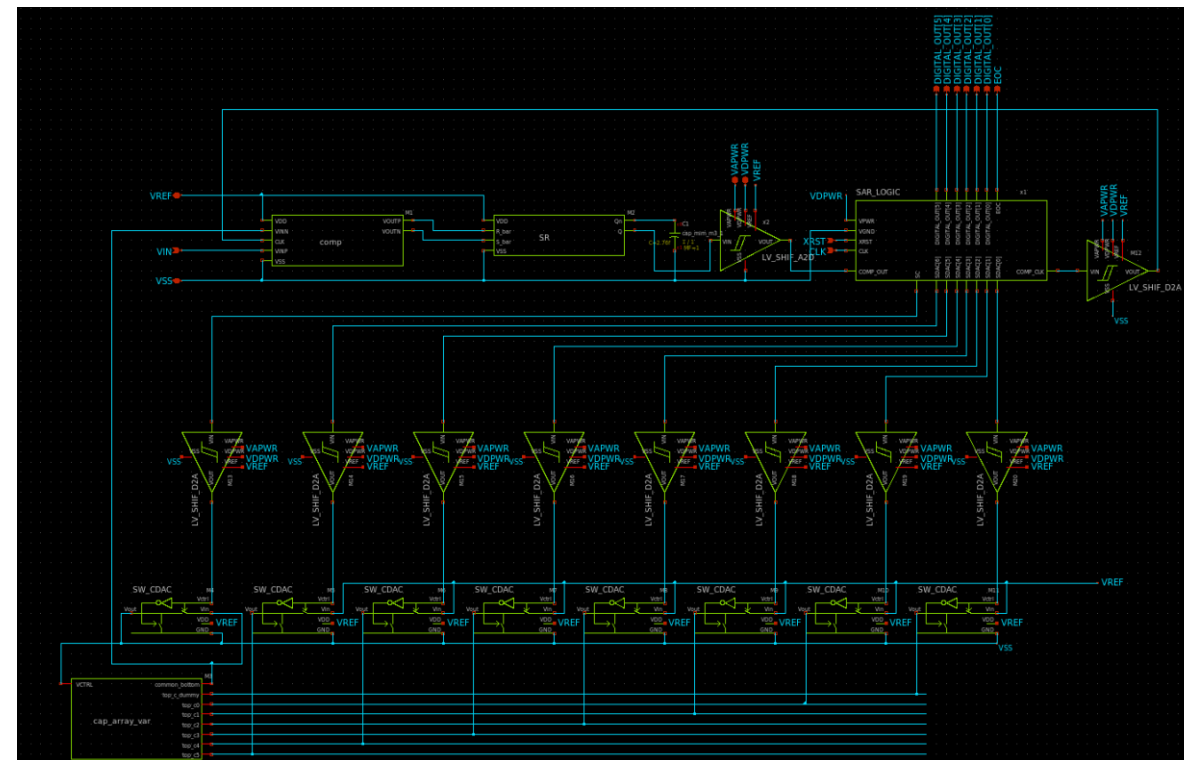
- 6bit(64)分の電圧をvinに入力して、正しいbitのDIGITAL_OUT[0-5]が出力されること
 - 0V~5V(リファレンス電圧)まで6bit(64)分、0VにLSB電圧を足した値をvinとして入力して、それぞれの結果を出力するシミュレーションをすればよい

ファイル名：
tran_sar_adc.sch&tran_sar_adc_pex.sch



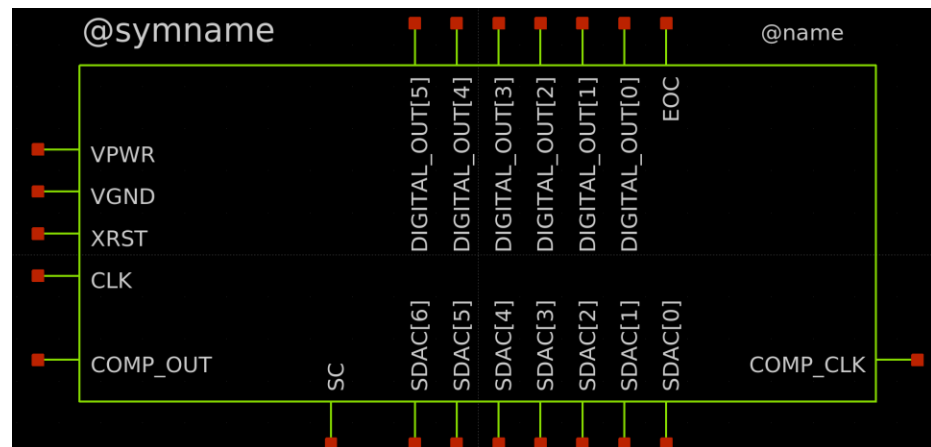
全部の回路を統合

- 問題点
 - SAR-LOGICの回路図が存在しない
 - spiceファイルを直接読み込むことで対応する



Spiceファイルから 読みだす方法1/2

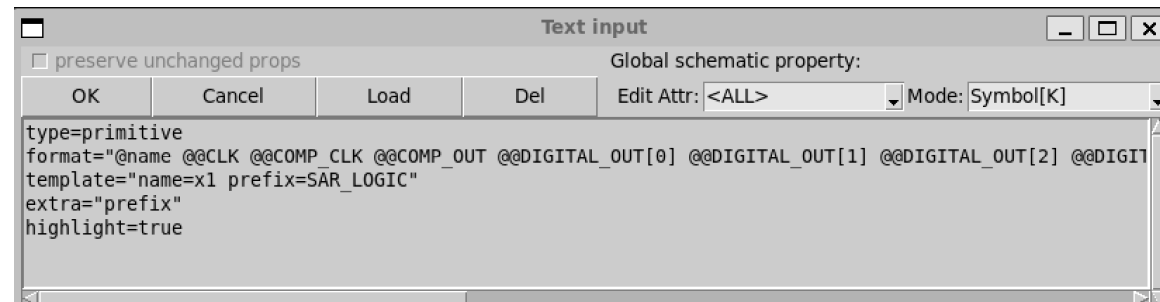
- Schファイル（上側の図）
 - ピン情報のみ
- Symファイル（下側の図）
 - schのピン情報からシンボルを作る



Spiceファイルから読みだす方法2/2

- Symファイルの設定
 - 右上の図を参照

```
format="@name @@CLK @@COMP_CLK  
@@COMP_OUT @@DIGITAL_OUT[0]  
@@DIGITAL_OUT[1] @@DIGITAL_OUT[2]  
@@DIGITAL_OUT[3] @@DIGITAL_OUT[4]  
@@DIGITAL_OUT[5] @@EOC @@SC  
@@SDAC[0] @@SDAC[1] @@SDAC[2]  
@@SDAC[3] @@SDAC[4] @@SDAC[5]  
@@SDAC[6] @@VGND @@VPWR @@XRST  
@prefix"
```



- formatの内容
 - Spiceファイルの「.subckt SAR_LOGIC」の後のピンと同じ並びで記述する

```
.subckt SAR_LOGIC CLK COMP_CLK  
COMP_OUT DIGITAL_OUT[0] DIGITAL_OUT[1]  
DIGITAL_OUT[2] DIGITAL_OUT[3]  
DIGITAL_OUT[4] DIGITAL_OUT[5] EOC SC  
SDAC[0] SDAC[1] SDAC[2] SDAC[3] SDAC[4]  
SDAC[5] SDAC[6] VGND VPWR XRST
```

spiceファイルの読み込み設定

- Spiceコマンド内
 - .include [spiceファイルへの絶対path]

COMMANDS

SIM=ngspice

```
.include ~/sar-adc/SAR_LOGIC.spice  
.temp 27  
.option savecurrent
```

シミュレーション コマンド1/2

- letコマンド
 - 変数を定義する
- マジックナンバー
 - 5.0 = リファレンス電圧
 - 0.078125 = LSB電圧
 - 5.0V / 6bit(64)
 - 1/2は閾値

```
.include ~/sar-adc/SAR_LOGIC.spice
.temp 27
.option savecurrent
.control
save all
alter VREF 5.0
```

```
let start_vin = 0.078125 / 2
let stop_vin = 5.0
let delta_vin = 0.078125
let vin_act = start_vin
alter Vin vin_act
```

```
while vin_act le stop_vin
    tran 100n 1u
    write tran_sar_adc.raw
    meas tran vin FIND v(VIN) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout5 FIND v(DIGITAL_OUT[5]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout4 FIND v(DIGITAL_OUT[4]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout3 FIND v(DIGITAL_OUT[3]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout2 FIND v(DIGITAL_OUT[2]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout1 FIND v(DIGITAL_OUT[1]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout0 FIND v(DIGITAL_OUT[0]) WHEN v(EOC)=0.9 FALL=LAST
    print vin >> tran_sar_adc_out_typ.txt
    print vout5 >> tran_sar_adc_out_typ.txt
    print vout4 >> tran_sar_adc_out_typ.txt
    print vout3 >> tran_sar_adc_out_typ.txt
    print vout2 >> tran_sar_adc_out_typ.txt
    print vout1 >> tran_sar_adc_out_typ.txt
    print vout0 >> tran_sar_adc_out_typ.txt
    let vin_act = vin_act + delta_vin
    alter Vin vin_act
```

```
end
```

```
.endc
```

シミュレーション コマンド2/2

- meas行
 - EOCに変化があったら、その時の各ポートの電圧を変数に保存する
- print行
 - 上記の電圧をファイルに保存する
- while行
 - vin_actがstop_vinまで繰り返す
 - 一回ごとにvin_actにLBS電圧を足す
 - vin_actをvinに設定する

```
.include ~/sar-adc/SAR_LOGIC.spice
.temp 27
.option savecurrent
.control
save all
alter VREF 5.0
```

```
let start_vin = 0.078125 / 2
let stop_vin = 5.0
let delta_vin = 0.078125
let vin_act = start_vin
alter Vin vin_act
```

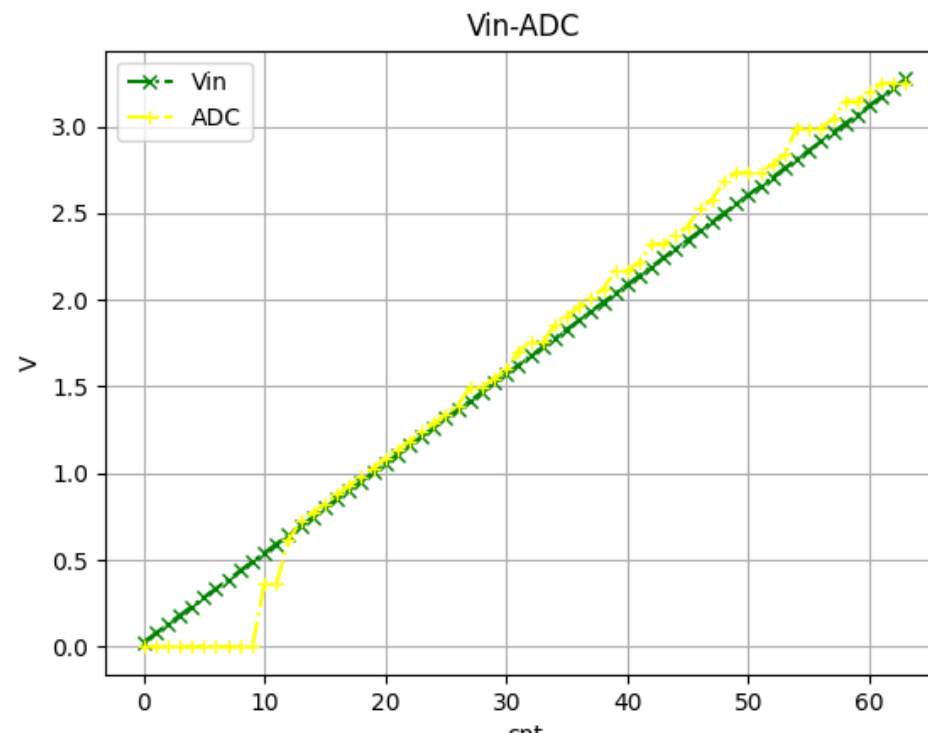
```
while vin_act le stop_vin
    tran 100n 1u
    write tran_sar_adc.raw
    meas tran vin FIND v(VIN) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout5 FIND v(DIGITAL_OUT[5]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout4 FIND v(DIGITAL_OUT[4]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout3 FIND v(DIGITAL_OUT[3]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout2 FIND v(DIGITAL_OUT[2]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout1 FIND v(DIGITAL_OUT[1]) WHEN v(EOC)=0.9 FALL=LAST
    meas tran vout0 FIND v(DIGITAL_OUT[0]) WHEN v(EOC)=0.9 FALL=LAST
    print vin >> tran_sar_adc_out_typ.txt
    print vout5 >> tran_sar_adc_out_typ.txt
    print vout4 >> tran_sar_adc_out_typ.txt
    print vout3 >> tran_sar_adc_out_typ.txt
    print vout2 >> tran_sar_adc_out_typ.txt
    print vout1 >> tran_sar_adc_out_typ.txt
    print vout0 >> tran_sar_adc_out_typ.txt
    let vin_act = vin_act + delta_vin
    alter Vin vin_act
```

```
end
```

```
.endc
```

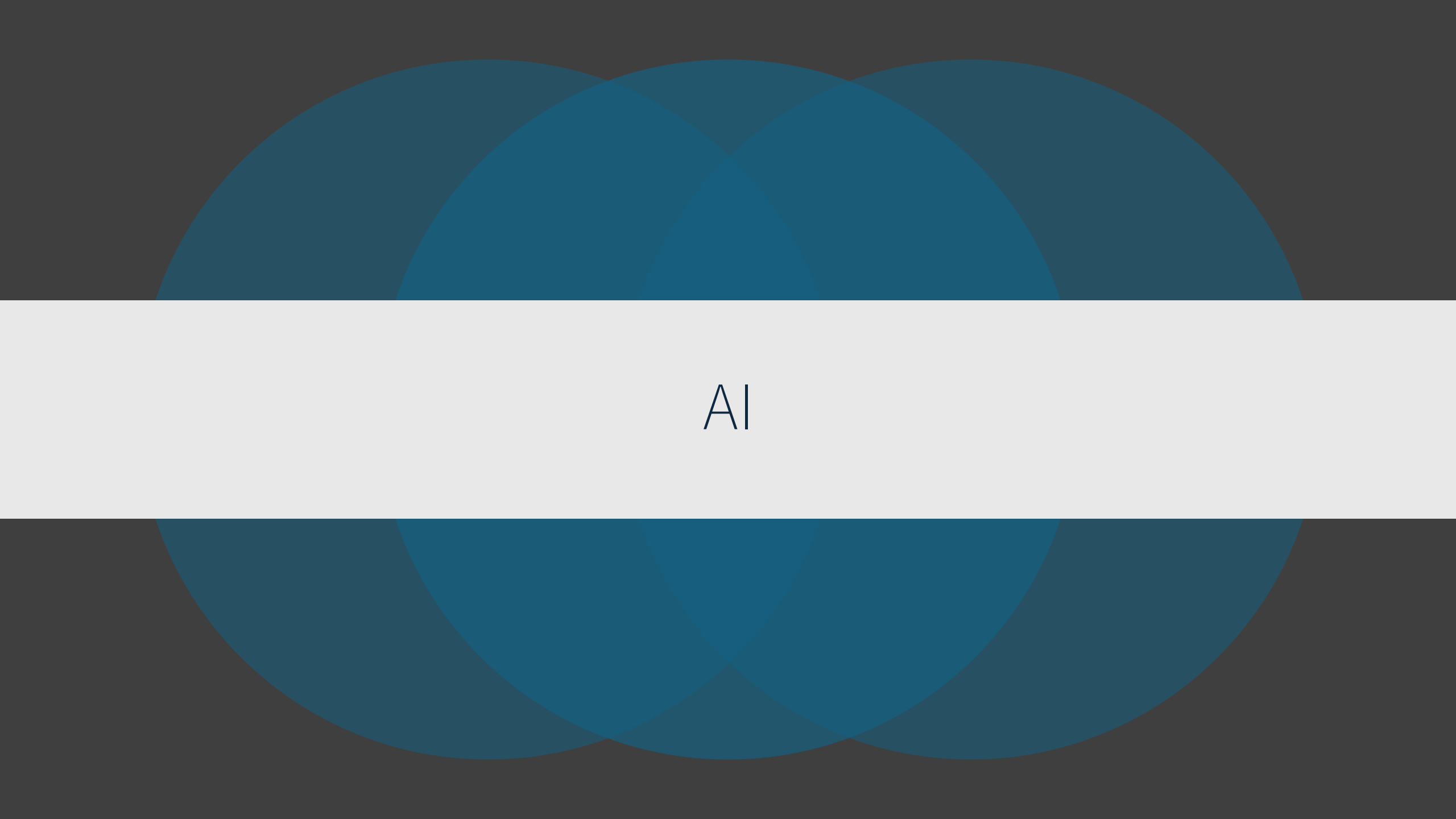
結果の表示

- 結果表示スクリプト
 - tran_sar_adc_result_checker.py
 - シミュレーションにより作成された「tran_sar_adc_out_typ.txt」から「tran_sar_adc_vin_vout_result.png」のグラフを作成する
 - 「tran_sar_adc_out_typ.txt」は「上書き」ではなく「追記」となる点に注意



並列実行スクリプト

- このシミュレーションは、6bit(64)分実行されます。並列で実行したい場合はこちらをお使いください。
- ファイル群
 - tran_sar_adc_1loop_sp.sch
 - ベースとなるシミュレーションファイルです。
 - ngspiceコマンドは変更しないでください
 - tran_sar_adc_1loop_sp_create_spice.py
 - 上記で生成されたspiceを6bit分のspiceに変換します
 - tran_sar_adc_1loop_sp_sim.sh
 - 上記のspiceファイルをCPUコアごとに並列実行します。
 - tran_sar_adc_1loop_sp_result_checker.py
 - 上記のシミュレーションで作成された結果ファイルから結果画像を作成します。



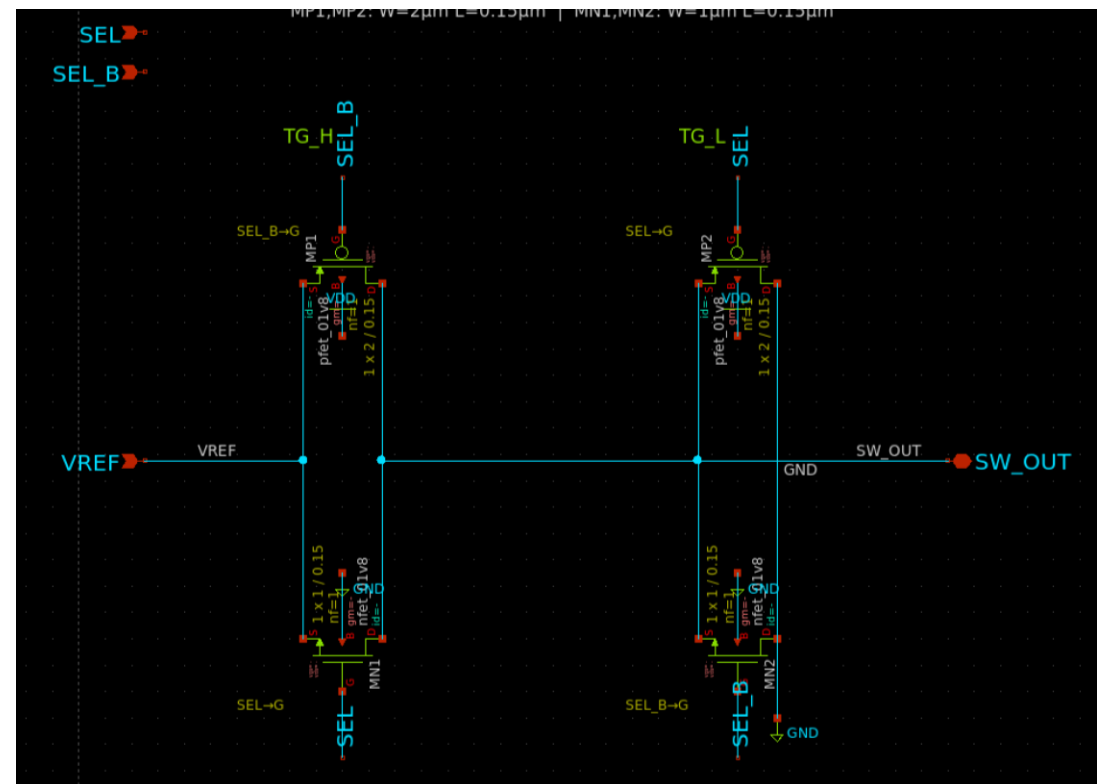
AI

シミュレーション

- ngspice用の0Vと3.3Vのパルス信号を1nsから10nsごとに100nsまで上げ下げする電源を作ってください。
 - `Vpulse vdd 0 PULSE(0 3.3 1n 100p 100p 10n 20n)`
 - `Vpulse vdd 0 PULSE(V1 V2 TD TR TF PW PER)`
 - パラメータの意味：V1=初期値(0V)、V2=パルス値(3.3V)、TD=遅延(1ns)、TR/TF=立上り/立下り時間(100ps)、PW=パルス幅(10ns)、PER=周期(20ns)
- 使い方
 - シミュレーション用のspiceコマンドを自動生成させて、xschemに張り付ける使い方

回路図

- 形はそれっぽい
- 問題点
 - 微妙にあってない
- 使い方
 - だいたいの形を作らせる
 - Symを作らせる



レイアウト

- 形はほぼあっている
- 問題点
 - P-Cellではない
- 使い方
 - 現状使えるイメージはない

